NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 12. Oktober 2026. Phasen 1–4 implementiert; Phase 5 mit enger HE-MCS-0-Kalibrierung und begrenzten CPU-Studien teilweise validiert. R1–R5 sind im jeweils festgelegten CPU-Umfang abgeschlossen. R6 ist implementiert; CPU-Abnahme erfolgreich, RTX-3060-/CUDA-CI-Hardwareabnahme ausstehend. R5 ergänzt getrennte physikalische Sounding-/Sendekanäle mit Bewegung und Doppler.

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858; Auftragsbasis 46d73fa; Sionna und Sionna RT 2.0.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle Stand umfasst die Unterphasen 1.1 bis 1.4 sowie die vollständigen Phasen 2, 3 und 4: den numerischen SU-MIMO-Kern, konfigurierbare Sionna-Arrays, einen dimensionssicheren MIMO-Datenvertrag, explizit kalibrierbare 802.11ax-EESM-/PER-Abbildung und die schmale Einkopplung in den SpectrumWifiPhy-Empfangspfad. SionnaMimoHelper legt den High-Level-API-Vertrag und dessen zentrale Validierung fest und wendet ihn nun konsistent auf Sionna-IPC, Phase-1-Linkabstraktion und SpectrumWifiPhy an. Der MCS-bezogene ApplyForHeMcs-Aufruf prüft eine für SU-MIMO erforderliche EESM-Kalibrierung vor jeder Zustandsänderung. Das reproduzierbare Beispiel konfiguriert 1x1 und Vollrang-2x2 ausschließlich über SionnaMimoHelper und deckt mit dem deterministischen Kanalpfad SpectrumWifiPhy end-to-end ab. Integrationsnahe Negativläufe prüfen widersprüchliche NSS, nicht unterstützte Modi und eine CFR-/PHY-Dimensionsverletzung. Ein dimensionssicherer, benutzergegebener Precoder unterstützt nun frequenzflaches und frequenzselektives festes $W[k]$, einschließlich kontrollierter Streamauswahl mit $N_s < N_{\mathrm{tx}}$. Darauf aufbauend erzeugt ns-3 nun für den eindeutigen MISO-Fall aus perfekter CFR ein leistungsnormalisiertes Ein-Stream-MRT-$W[k]$. Für $N_{\mathrm{tx}} = 1$ ist der bestehende Ein-Stream-MMSE-Pfad als äquivalente MRC-Kombination validiert. Die neue deterministische SVD-Fabrik wählt pro Subcarrier explizit die N_s stärksten numerisch tragfähigen Eigenmoden und erzeugt bei gleicher Leistung oder optionaler räumlicher Wasserfüllung erneut ein festes $W[k]$. Rang, Singularwerte, Rauschleistung, Referenz-SNR, aktive Moden und Leistungsallokation bleiben vollständig sichtbar. Die neue semantische HE-/EHT-SU-Feedbackdarstellung komprimiert gleichleistungs-SVD-Precoder getrennt in Givens-Winkel, quantisiert sie deterministisch mit den groben 4/2- oder feinen 6/4-Bit-Codebooks und rekonstruiert daraus erneut ein auf den festen-$W[k]$-Vertrag normalisiertes Ergebnis. Bitumfang, Normierungskorrektur sowie phaseninvariante Richtungs- und Koeffizientenfehler werden ausgewiesen. MimoCsiAgingModel trennt $\hat{H}[k, t_{\mathrm{csi}}]$ von $H[k, t_{\mathrm{tx}}]$, bildet periodische Aktualisierung und Feedbackverzögerung explizit ab und liefert eine gerichtete, array-, modus- und zeitstempelabhängige Zustandsidentität. SionnaMimoHelper integriert nun festes Precoding, MRT, SVD, Wasserfüllung, quantisiertes Feedback und CSI-Alterung durch eine explizite, unveränderliche Precoderwahl. Apply führt das ausgewählte $W[k]$ ohne geänderte Semantik durch denselben festen SpectrumWifiPhy-Adapterpfad. Das neue CPU-End-to-End-Beispiel vergleicht Open Loop, festen 2x1-Precoder, MRT und SVD bei gleicher Gesamtleistung und gibt Konfiguration, Durchsatz, PER, Stream-SINR, Rang, Precodingnorm, Leistungsanteile, Feedbackdiagnostik und CSI-Alter maschinenlesbar aus.

Der additive Mehrnutzer-Kanalvertrag ordnet nun gerichtete, zeitgleiche CFRs
stabil nach Nutzer-ID, trägt die RX-Geometrie je Nutzer und übernimmt erst
nach vollständiger Snapshotprüfung Einträge in den bestehenden Cache. Darauf
wertet der neue native Downlink-MU-Kern einen festen gemeinsamen Precoder mit
expliziter Spaltenzuordnung aus. Er erhält die gesamte AP-Sendeleistung,
trennt gewünschte Leistung und Inter-User-Interferenz je Nutzer, Subcarrier und
Stream und berechnet EESM sowie PER mit den nutzerspezifischen MCS- und
Paketgrößenangaben. Die neue wertbasierte Precoderwahl erzeugt für genau einen
Stream und eine Empfangsantenne je Nutzer aus gleichzeitiger perfekter CSI
deterministisches ZF oder RZF mit explizitem Regularisierungsparameter. Beide
Ergebnisse werden auf die gemeinsame Phase-4.2-Gesamtleistung normalisiert;
überlastete und numerisch rangdefiziente Gruppen werden abgelehnt. Unterphase
4.4 integriert diesen Zustand für HE-DL-MU-PPDUs in den Spectrumadapter,
ordnet STA-ID, RU, MCS und NSS zu und begrenzt räumliche Interferenz auf
dieselbe RU. Unterphase 4.5 bindet nun den Laufzeit-CSI-Zustand an die
MultiUserScheduler-Ausgabe und ergänzt feste sowie korrelationsbasierte
Gruppen. Unterphase 4.6 ergänzt die Helper-Installation, den konkreten Downlink-Scheduler und
den
deterministischen CPU-Vergleich. Unterphase 4.7 ergänzt den gemeinsamen
Uplink-ZF-/MMSE-Numerikkern und nutzerweise PER. Unterphase 4.8 bindet reale
HE-Basic-Trigger- und HE-TB-PHY-Metadaten an diesen Kern und berücksichtigt
nur exakt gleichzeitig am AP empfangene Nutzer innerhalb derselben RU.
Phase 5 ergänzt eine begrenzte Link-Level-Kalibrierung sowie reproduzierbare
Kanal-, CSI- und Cache-/IPC-Studien. Deren Aussagen bleiben auf die geprüften
Profile, Szenen und Backends beschränkt.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

  - Ubuntu 24.04, GCC 13.3, CMake 3.28
  - ns-3.40 mit Tests und Beispielen
  - ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
  - Python 3.12.12, Sionna 2.0.1, Sionna RT 2.0.1 und NumPy 2.5.2
  - Mitsuba 3.8.0, Dr.Jit 1.3.1 und LLVM-Backend llvm_ad_mono_polarized
  - PyTorch 2.9.1+cpu; TensorFlow ist keine Laufzeitabhängigkeit mehr

Ohne explizite Arraykonfiguration verwendet der Server weiterhin exakt die
bisherige 1x1-Sende- und Empfangsantenne. Die normalisierte SISO-CFR erfüllt

$$
  \frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Unterphase 1.2
verallgemeinert diese Trennung auf alle Antennenpaare: $H[k]$ enthält
Small-Scale-Fading und der skalare Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

  PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.
2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und
   Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

  PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

- CFR-Tensor $H[k]$ mit expliziten Dimensionen $N_\mathrm{sc}\times N_\mathrm{rx}\times N_\mathrm{tx}$.
- Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.
- Gleiche Gesamtleistungsaufteilung über $N_s = N_{\mathrm{tx}}$ Streams.
- Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.
- Post-MMSE-SINR pro Subcarrier und Stream.
- Mutual-Information-basierte Kompression über Subcarrier.
- Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10 reproduziert.

UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3 (IMPLEMENTIERT)

- Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
- Flache Reihenfolge sc/rx/tx verbindlich festlegen.
- Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der Antennendimensionen serialisieren.
- CacheEntry auf MimoChannelMatrix umstellen.
- Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte kommen indexgenau in ns-3 an.

UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION (IMPLEMENTIERT)

- Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite, Rauschzahl und thermischem Rauschen berechnen.
- Effektive SINR mit einer explizit kalibrierbaren EESM-Abbildung je MCS bestimmen; ohne belegte Kalibrierung existieren bewusst keine Default-Betas.
- Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
- SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im SISO-Grenzfall deckungsgleich zur festgelegten Referenz. Alle Kriterien wurden mit den in Abschnitt 7 beschriebenen automatischen Tests erfüllt.

UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL (IMPLEMENTIERT)

- Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
- NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh ablehnen.
- Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
- Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert auf Rang-1-Kanälen nachvollziehbar gegen 1x1.
Die automatisierten Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam konfiguriert:

- Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
- Ausrichtung: Azimut, Elevation und Rollwinkel
- Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
- Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp

  - Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass
Beispiele keine internen Objekte manuell verdrahten müssen.

Phase 2 wird in drei Unterphasen nach API-Vertrag, interner Integration und
Benutzbarkeit getrennt. Damit bleiben API-Entscheidungen ohne Änderung des
PHY-Verhaltens prüfbar, während Kompatibilitäts- und End-to-End-Regressionen
den nachfolgenden Integrationsschritten eindeutig zugeordnet werden können.

  UNTERPHASE 2.1: API-VERTRAG (IMPLEMENTIERT)

  - SionnaMimoHelper sowie Konfigurationstypen, dokumentierte Defaults und
  unterstützte Betriebsarten definieren.
  - Arraygeometrie, knotenspezifische Ausrichtung, TX-/RX-Antennenzahlen,
  maximale NSS, Empfängertyp, Kanalnormalisierung, Rauschzahl und
  Effective-SINR-Modell in einem konsistenten Vertrag zusammenführen.
  - Zentrale Validierung für vollständige, endliche und untereinander
  konsistente Parameter bereitstellen.
  - In Phase 2 nur SISO und SU-MIMO Spatial Multiplexing als funktionale Modi
  unterstützen. Beamforming und MU-MIMO werden entweder noch nicht angeboten
  oder brechen, falls sie für API-Stabilität bereits repräsentiert werden,
  eindeutig als nicht unterstützt ab.
  - Reine API-Unit-Tests für Defaults sowie gültige und ungültige
  Konfigurationen hinzufügen, ohne das bestehende PHY-Verhalten zu ändern.

Abnahmekriterium: Der API-Vertrag ist unabhängig von einer Simulation
testbar, rückwärtskompatible Defaults sind festgeschrieben und kein Modus aus
Phase 3 oder 4 erscheint fälschlich funktionsfähig.

  UNTERPHASE 2.2: INTEGRATION (IMPLEMENTIERT)

  - SionnaMimoHelper mit den vorhandenen Phase-1-Komponenten für Kanal,
  Link Budget, MMSE, EESM und SpectrumWifiPhy verbinden.
  - Knotenbezogene Arraygeometrie und Ausrichtung über den Sionna-IPC- und
  Serverpfad anwenden.
  - Antennenzahlen, maximale Spatial Streams, Rauschzahl, Empfängertyp,
  Normalisierung und Effective-SINR-Konfiguration konsistent an die
  beteiligten ns-3-Objekte weiterreichen.
  - Den bisherigen SionnaHelper-Pfad und die globalen 1x1-Defaults unverändert
  nutzbar halten.
  - Integrationstests ergänzen, welche manuelle Phase-1-Konfiguration und
  Helper-Konfiguration für 1x1 und 2x2 vergleichen.

Abnahmekriterium: Äquivalente manuelle und High-Level-Konfigurationen liefern
dieselben Phase-1-Ergebnisse; bestehende 1x1-Beispiele und Tests bleiben
unverändert erfolgreich.

  UNTERPHASE 2.3: BENUTZBARKEIT UND VALIDIERUNG (IMPLEMENTIERT)

  - Minimale 1x1- und 2x2-Beispiele ausschließlich über die High-Level-API
  anlegen.
  - Integrationsnahe Negativtests für widersprüchliche Laufzeitparameter und
  Fehlerweitergabe vom Helper bis zum PHY ergänzen.
  - End-to-End-Tests vom Helper über Sionna beziehungsweise den
  deterministischen Kanalpfad bis SpectrumWifiPhy ausführen.
  - API, Defaults, unterstützte Modi, Fehlermeldungen und Migration von der
  manuellen Konfiguration dokumentieren.

Abnahmekriterium: Die 1x1-/2x2-Beispiele laufen ohne manuelle Verdrahtung
interner Objekte, ungültige Konfigurationen brechen früh und eindeutig ab und

Dokumentation sowie PDF beschreiben den abgeschlossenen Stand von Phase 2.
Die helperbasierten End-to-End- und Laufzeit-Negativtests in Abschnitt 7
erfüllen dieses Kriterium.

PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Sionna liefert weiterhin ausschließlich die komplexe CFR $H[k]$. Auswahl und
Anwendung des Precoding $W[k]$, Streamauswahl, Leistungsaufteilung und
CSI-Zeitbezug bleiben vollständig in ns-3. Die bestehende skalare Grenze zu
SpectrumWifiPhy bleibt erhalten: Erst nach der matrixwertigen Kanal-,
Precoding- und Detektorberechnung wird auf eine Effective-SINR beziehungsweise
äquivalente Empfangs-PSD komprimiert.

Phase 3 ist inhaltlich größer als Phase 1 oder Phase 2 jeweils für sich: Sie
erweitert gleichzeitig Datenvertrag, numerischen Kern, High-Level-API und
PHY-Integration und führt zusätzlich rückgemeldete sowie zeitabhängige CSI
ein. Die folgenden fünf Unterphasen beschreiben deshalb vertikale,
abnehmbare Funktionen; die einzelnen Aufzählungspunkte bleiben interne
Implementierungsschritte. Unterphasen 3.1 bis 3.3 bilden den idealen
Perfect-CSI-Kern, Unterphase 3.4 ist der eigene Realismus- und
Standardisierungsblock und Unterphase 3.5 schließt die Phase end-to-end ab.

UNTERPHASE 3.1: FESTES PRECODING ALS VOLLSTÄNDIGE GRUNDLAGE (IMPLEMENTIERT)

- Einen dimensionssicheren Vertrag für
  $W[k] \in \mathbb{C}^{N_{tx} \times N_s}$ mit verbindlicher sc/tx/stream-Reihenfolge
  definieren; frequenzflache Matrizen werden explizit auf alle Subcarrier
  abgebildet.
- Das Signalmodell auf
  $y[k] = \sqrt{P/N_s}\, H[k]W[k]x[k] + n[k]$ verallgemeinern und den bestehenden
  MMSE-Kern auf den effektiven Kanal $H_{\text{eff}}[k] = H[k]W[k]$ anwenden.
- N_s unabhängig von N_tx im Bereich $1\leq
  N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$ führen.
- Die Leistungsnormierung $\|W[k]\|_F^2 = N_s$ festlegen, sodass P weiterhin die
  gesamte Sendeleistung bezeichnet und kein unbeabsichtigter Precoding-Gewinn
  entsteht.
- Analytische Tests für SISO, Identitäts-Precoding, Streamauswahl,
  komplexwertige feste Matrizen und Dimensions-/Normierungsfehler ergänzen.

Numerisches Zwischenkriterium: $W[k] = I$ reproduziert den Phase-1-Kern
innerhalb 1e-10;
synthetische $H[k]W[k]$-Referenzen liefern indexgenau dieselben
Post-MMSE-SINRs und die Gesamtleistung bleibt invariant.

Integrationsschritte innerhalb derselben Unterphase:

- Benutzergegebenes frequenzflaches und frequenzselektives $W[k]$ durch
  SionnaMimoHelper, Link-Abstraktion und SpectrumWifiPhy-Adapter führen.
- WifiTxVector::GetNss an die Spaltenzahl von $W[k]$ statt weiterhin zwingend
  an N_{tx} koppeln und $N_s < N_{tx}$ erstmals kontrolliert zulassen.
- Subcarrierzahl, Antennenmaße, NSS, endliche Koeffizienten und
  Leistungsnormierung vor jeder Zustandsänderung zentral validieren.
- Ohne explizites Precoding den bisherigen SISO-/Open-Loop-Pfad byte- und
  funktional kompatibel halten.
- API-, Adapter- und End-to-End-Tests für 1x1, 2x2-Identität sowie
  $N_{tx} = 2$, $N_{rx} = 2$, $N_s = 1$ ergänzen.

Abnahmekriterium: Manuell und über SionnaMimoHelper konfiguriertes festes
$W[k]$ liefern dieselben SINR-, EESM- und PER-Ergebnisse; alle Phase-2-Tests
bleiben unverändert erfolgreich. Die analytischen, Adapter- und
SpectrumWifiPhy-End-to-End-Ergebnisse in Abschnitt 7 erfüllen dieses

Kriterium.

  UNTERPHASE 3.2: EIN-STREAM-MRT/MRC (IMPLEMENTIERT)

  - MRT für den eindeutig definierten MISO-Fall $N_{rx} = 1$ aus perfekter CFR
  berechnen und auf Einheitsnorm normieren.
  - Für $N_{tx} = 1$ die bestehende Ein-Stream-MMSE-Lösung als äquivalente
  MRC-Kombination ausweisen und validieren.
  - Den allgemeinen Fall $N_{tx} > 1$ und $N_{rx} > 1$ nicht mehrdeutig als MRT/MRC
  bezeichnen; dessen optimale Ein-Stream-Lösung gehört zur SVD-Unterphase.
  - Analytische MISO-/SIMO-Kanäle, gemeinsame Phasendrehung, Nullkanal und
  erwarteten Arraygewinn testen.

Abnahmekriterium: MRT/MRC erreicht auf analytischen Ein-Stream-Kanälen die
Post-Combining-SNR $\rho\|h\|_2^2$, bleibt leistungsnormalisiert und fällt im
1x1-Grenzfall exakt auf das bestehende Ergebnis zurück. Der analytische
MRT-/MRC-Test in Abschnitt 7 erfüllt dieses Kriterium einschließlich
Phaseninvarianz und eindeutiger Ablehnung der Null- und allgemeinen
MIMO-Fälle.

  UNTERPHASE 3.3: SVD-PRECODING UND LEISTUNGSALLOKATION (IMPLEMENTIERT)

  - Pro Subcarrier $H[k] = U[k]\Sigma[k]V^H[k]$ zerlegen und die N_s stärksten
  rechten Singulärvektoren bei zunächst gleicher Streamleistung als $W[k]$
  verwenden.
  - N_s explizit wählen und gegen Antennenzahlen sowie einen dokumentierten
  numerischen Rangschwellwert validieren.
  - Sortierung, Vorzeichen-/Phasenkonvention und Verhalten bei gleichen
  Singularwerten deterministisch festlegen.
  - Diagonale, unitär gedrehte, frequenzselektive und rangdefiziente
  Referenzkanäle gegen analytische Eigenmode-SINRs testen.

Zwischenkriterium für SVD und Streamauswahl: Die Eigenmode-SINRs stimmen
innerhalb 1e-10 mit
den Singularwertreferenzen überein; Rangverlust wird durch eine explizite
Streamwahl behandelt und nicht durch versteckte Regularisierung kaschiert.

Optionale Leistungsallokation innerhalb derselben Unterphase:

  - Gleiche Leistung als reproduzierbaren Standard beibehalten und
  Wasserfüllung als explizit auswählbare Forschungsoption ergänzen.
  - Den Umfang zunächst auf räumliche SVD-Moden je Subcarrier begrenzen, damit
  spektrale Maske und Gesamtleistung pro Subcarrier unverändert bleiben.
  - Rauschleistung, Referenz-SNR, aktive Moden und die Summe der zugewiesenen
  Leistung vollständig ausgeben und validieren.
  - Analytische Zwei-Moden-Fälle unterhalb und oberhalb der Aktivierungsschwelle
  sowie Leistungserhaltung und KKT-Bedingungen testen.

Abnahmekriterium der optionalen Erweiterung: Die Leistungsanteile sind
endlich, nicht negativ und
summieren sich exakt zur vorgegebenen Gesamtleistung; Wasserfüllung erreicht
auf den Referenzkanälen mindestens die ideale Rate der geprüften
Gleichleistungsreferenz, ohne Sendeleistung zu erzeugen.

  UNTERPHASE 3.4: QUANTISIERTE RÜCKMELDUNG UND VERALTETE CSI (IMPLEMENTIERT)

  - Vor der Implementierung das konkret unterstützte HE-/EHT-Feedbackformat,
  Winkel-/Codebookdarstellung, Bitbreiten und Rekonstruktionsreihenfolge
  verbindlich dokumentieren; nicht unterstützte Standardvarianten klar
  abgrenzen.
  - Ideales SVD-Precoding, Quantisierung und Rekonstruktion als getrennte,

einzeln testbare Schritte modellieren.
- Quantisiertes $W[k]$ erneut leistungsnormalisieren und Feedbackumfang sowie Quantisierungsfehler als Diagnosegrößen bereitstellen.
- Bekannte Vektoren, Grenzwinkel, minimale/maximale Quantisierung und den unquantisierten Grenzfall testen.

Zwischenkriterium für die Quantisierung: Bitgenaue Referenzvektoren werden deterministisch
rekonstruiert, die Precoder bleiben leistungsnormalisiert und feinere Quantisierung verschlechtert die Referenzmetrik auf den Testkanälen nicht.

CSI-Alterung innerhalb derselben Unterphase:

- Die zur Precoderwahl verwendete Schätzung $\hat{H}[k, t_{\mathrm{csi}}]$ strikt von der bei der Paketübertragung wirksamen CFR $H[k, t_{\mathrm{tx}}]$ trennen.
- CSI-Alter, Aktualisierungsintervall und Feedbackverzögerung explizit und reproduzierbar konfigurieren; perfektes aktuelles CSI bleibt der Nullalter-Grenzfall.
- Richtung, Arraykonfiguration, Precodingmodus und CSI-Zeitstempel in Cache- beziehungsweise Zustandsidentität berücksichtigen.
- Statische Kanäle, kontrollierte Phasen-/Subraumrotationen und wiederholte Läufe mit identischen Seeds testen.

Abnahmekriterium: CSI-Alter null reproduziert das ideale Precoding exakt; kontrolliert veraltete Schätzungen werden nachweislich nur zur Wahl von W verwendet, während SINR und PER stets mit dem aktuellen H berechnet werden. Die bitgenauen Quantisierungs-, Leistungs-, Timing-, Subraum-, SINR- und PER-Regressionen in Abschnitt 7 erfüllen dieses Kriterium für den in Abschnitt 5.12 abgegrenzten SU-Feedbackumfang.

UNTERPHASE 3.5: BENUTZBARKEIT UND PHASENVALIDIERUNG (IMPLEMENTIERT)

- SionnaMimoHelper erst nach erfolgreicher Implementierung die jeweiligen Beamforming-/Precodingmodi als unterstützt melden lassen.
- Das Beispiel ns3_wifi_sumimo_precoding.cc anlegen und Open-Loop, festes Precoding, MRT sowie SVD bei gleicher Gesamtleistung vergleichen; Wasserfüllung, Quantisierung und CSI-Alter werden als explizite Optionen zugeschaltet.
- Durchsatz, PER, Stream-SINR, Kanalrang, Precodingnorm, Leistungsanteile und CSI-Alter maschinenlesbar ausgeben.
- Negative Konfigurationstests und vollständige CPU-End-to-End-Regressionen ergänzen sowie API-Migration und Modellgrenzen dokumentieren.

Abnahmekriterium: Die Beispiele benötigen keine manuelle Verdrahtung interner Objekte, analytisch überlegene Precoder zeigen den erwarteten Gewinn ohne zusätzliche Sendeleistung, ungültige Konfigurationen brechen früh ab und alle Phase-1-/2-Regressionen bleiben erfolgreich.

Dieses Kriterium ist erfüllt: Der Helper meldet SU_BEAMFORMING erst mit dieser Integration als unterstützt, das neue Vergleichsbeispiel verwendet nur die öffentliche High-Level-Konfiguration und SelectPrecoding, alle vier Modi halten dieselbe Gesamtleistungsdefinition ein, ungültige Kombinationen brechen vor Zustandsänderungen ab und die CPU-Regressionen in Abschnitt 7 decken die alten sowie die neuen Pfade ab.

PHASE 4: MU-MIMO

Sionna liefert auch in Phase 4 ausschließlich gerichtete komplexe CFRs \(H_u[k]\) für die einzelnen Verbindungen. Gemeinsame Nutzerwahl, Precoding, Leistungsaufteilung, Inter-User-Interferenz, Scheduling, Empfängermodell und

PER-Abbildung bleiben vollständig in ns-3. Eine MU-Übertragung darf nur Kanäle aus einem konsistenten gemeinsamen Zeitschnappschuss verwenden.

Phase 4 ist größer als die vorherigen Einzelphasen. Insbesondere sind Downlink und Uplink nicht symmetrisch: Im Downlink sendet ein AP mit einem gemeinsamen Precoder, während im Uplink mehrere unabhängige Sender am AP gemeinsam detektiert werden. Die Phase wird daher in acht vertikale Unterphasen geteilt. Die Unterphasen 4.1 bis 4.6 liefern zuerst einen vollständig nutzbaren Downlink-MU-MIMO-Pfad; 4.7 und 4.8 ergänzen danach den Uplink, ohne den Abschluss des Downlink-Pfads zu blockieren.

  UNTERPHASE 4.1: MEHRNUTZER-KANALSCHNAPPSCHUSS UND IDENTITÄT (IMPLEMENTIERT)

  - Einen dimensionssicheren Vertrag für gerichtete CFRs
  \(\{H_u[k]\}_{u=1}^{U}\) mit stabiler Nutzer-ID, Sender, Empfänger,
  Richtung und gemeinsamem CSI-Zeitstempel definieren.
  - Subcarrierzahl, Frequenzraster, Normalisierung und die Trennung zwischen
  CFR und skalarem Wideband-Pfadverlust über alle Nutzer konsistent halten.
  - Unterschiedliche Empfangsarraygeometrien und Antennenzahlen je Nutzer
  repräsentieren; die bisherige gemeinsame RX-Geometrie darf nicht
  stillschweigend auf alle Stationen übertragen werden.
  - Vorhandene gebündelte Sionna-Anfragen und Cachepfade soweit möglich
  wiederverwenden und gemischte Zeitstempel, doppelte Nutzer, inkonsistente
  Dimensionen oder Richtungen vor jeder Zustandsänderung ablehnen.
  - Python- und C++-Vertragstests für Nutzerreihenfolge, Serialisierung,
  Tensorindizes, heterogene RX-Dimensionen und Zeitidentität ergänzen.

Abnahmekriterium: Synthetische Mehrnutzer-CFRs kommen unabhängig von ihrer Eingabereihenfolge nutzer- und indexgenau in ns-3 an; jeder inkonsistente Schnappschuss wird früh abgelehnt und alle bisherigen SU-Verträge bleiben unverändert erfolgreich.

Dieses Kriterium ist erfüllt: Die vorhandene `ChannelState`-/`RxNodeInfo`-Bündelung trägt eine additive `UserChannelIdentity`; Python und C++ prüfen den gesamten Schnappschuss vor Verwendung, C++ sortiert nach stabiler Nutzer-ID und der Cache übernimmt erst danach alle Links gemeinsam. Die neuen synthetischen Verträge decken heterogene RX-Arrays ab. Sämtliche Phase-1- bis Phase-3-SU-Regressionen bleiben bestanden; Unterphase 4.2 wurde nicht vorweggenommen.

  UNTERPHASE 4.2: FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL (IMPLEMENTIERT)

  - Einen festen gemeinsamen Precoder

  \[
    W[k] = \left[W_1[k],\ldots,W_U[k]\right]
  \]

  mit expliziter Zuordnung seiner Spalten zu Nutzer und Stream definieren.
  - Die Phase-3-Leistungskonvention auf die Gesamtzahl
  \(N_{s,\mathrm{ges}}\) aller gleichzeitig gesendeten Streams erweitern:
  \(\Vert W[k]\Vert_F^2=N_{s,\mathrm{ges}}\), sodass \(P\) weiterhin die
  gesamte AP-Sendeleistung bezeichnet. Jede abweichende Nutzer- oder
  Streamgewichtung muss explizit sein und in Summe dieselbe Gesamtleistung
  erhalten.
  - Für jeden Nutzer gewünschten Anteil und Inter-User-Interferenz getrennt
  berechnen:

  \[
    y_u[k] = H_u[k]W_u[k]x_u[k]
    + \sum_{v\ne u} H_u[k]W_v[k]x_v[k] + n_u[k].

\]

- Den bestehenden linearen Empfänger mit der vollständigen Interferenzkovarianz auswerten und gewünschte Leistung, Interferenzleistung sowie Post-Processing-SINR je Nutzer, Subcarrier und Stream ausgeben.
- EESM und PER anschließend separat mit dem MCS und der Paketgröße des jeweiligen Nutzers bestimmen.

Abnahmekriterium: Ein einzelner Nutzer reproduziert Phase 3 innerhalb \(10^{-10}\); orthogonale Nutzer erzeugen keine Inter-User-Interferenz, korrelierte Referenzkanäle liefern die analytische Interferenz und die Gesamtsendeleistung bleibt invariant.

Dieses Kriterium ist erfüllt: `DlMuMimoPrecodingMatrix` ordnet jede Spalte des unveränderten Phase-3-Precoders explizit einem Nutzer und lokalen Stream zu. `DlMuMimoLinkModel` wertet die vollständige Empfangskovarianz aus und weist gewünschte Leistung, Inter-User-Interferenz und SINR getrennt aus. `HeDlMuMimoLinkAbstraction` wendet den bestehenden EESM- und ErrorRateModel-Vertrag anschließend nutzerweise an. Analytische native Tests erfüllen alle vier Invarianten; Unterphase 4.3 wurde nicht vorweggenommen.

UNTERPHASE 4.3: IDEALES DOWNLINK-ZF UND REGULARISIERTES ZF (IMPLEMENTIERT)

- Zunächst den eindeutigen Minimalfall mit einem Stream je Nutzer und perfekter gleichzeitiger CSI implementieren.
- ZF aus dem gestapelten Mehrnutzerkanal und RZF mit explizit konfigurierbarem Regularisierungsparameter berechnen; es gibt keinen versteckten Regularisierungsdefault.
- Rangschwelle, Nutzerreihenfolge, Pseudoinverse und deterministische Behandlung gleicher beziehungsweise nahezu gleicher Singulärwerte dokumentieren.
- Den gemeinsamen Precoder nach jeder Auswahl auf die Konvention aus Unterphase 4.2 normalisieren und überlastete oder rangdefiziente Nutzergruppen eindeutig ablehnen beziehungsweise als nicht planbar melden.
- Orthogonale, korrelierte, überlastete und rangdefiziente analytische Referenzkanäle testen.

Abnahmekriterium: ZF unterdrückt auf vollrangigen Referenzkanälen die Inter-User-Interferenz innerhalb der numerischen Toleranz; RZF bleibt auf schlecht konditionierten Kanälen endlich, deterministisch und leistungsnormalisiert. Mehrere Streams je Nutzer werden erst nach einem explizit dokumentierten Blockdiagonalisierungs- oder gemeinsamen Transceiververtrag freigeschaltet und bis dahin klar abgelehnt.

Dieses Kriterium ist erfüllt: `DlMuMimoPrecodingMatrix::ZeroForcing` bildet die Moore-Penrose-Pseudoinverse des nach stabiler Nutzer-ID gestapelten Einantennenkanals; `RegularizedZeroForcing` verwendet ausschließlich das explizit übergebene positive RZF-α. Beide Fabriken prüfen den vollständigen Phase-4.1-Snapshot und die Ein-Stream-Zuordnung vor Konstruktion, lehnen Überlast und Rangverlust ab und skalieren erst das fertige Ergebnis auf die Phase-4.2-Gesamtleistung. Die schnellen nativen Referenzfälle in Abschnitt 7.5 erfüllen das Abnahmekriterium.

UNTERPHASE 4.4: HE-DOWNLINK-MU-PHY-INTEGRATION (KORRIGIERT UND VALIDIERT)

- Den gemeinsamen MU-Zustand in den schmalsten geeigneten SpectrumWifiPhy-Pfad für `WIFI_PREAMBLE_HE_MU` einkoppeln.
- Die vorhandenen STA-ID-, RU-, MCS- und NSS-Angaben aus `WifiTxVector::HeMuUserInfoMap` verwenden, anstatt parallele Wi-Fi-Metadaten im Sionna-Modul einzuführen.

- Inter-User-Interferenz nur zwischen räumlich gemultiplexten Nutzern
  innerhalb derselben RU berücksichtigen; verschiedene RUs bleiben
  orthogonal.
- Für jeden Empfänger erst nach der matrixwertigen MU-Berechnung auf dessen
  skalare äquivalente Empfangs-PSD komprimieren. Pfadverlust, AP-Sendeleistung,
  Rauschen und Empfangsgewinn dürfen dabei jeweils genau einmal eingehen.
- Positive und negative Adaptertests für Nutzer-/STA-ID-Zuordnung, RU, NSS,
  Antennendimensionen und unbekannte Empfänger ergänzen.

Abnahmekriterium: Ein echter HE-DL-MU-PPDU liefert je adressiertem Empfänger
die analytisch erwartete SINR und PER; nicht adressierte oder inkonsistente
Empfänger verändern keinen MU-Zustand. Der bestehende SU-Pfad bleibt
bit- und funktionskompatibel. EHT bleibt bis zu einer expliziten
EHT-Link-/PER-Abstraktion als nicht unterstützt gekennzeichnet.

Die erste Implementierung vom 3. September prüfte die direkte
PPDU-/Adapterverdrahtung. Der Review vom 5. September zeigte Fehler in der
RU-Frequenzauswahl, RU-Leistungsnormierung, gemeinsamen Präambel und Behandlung
mithörender Stationen. Die Korrektur ergänzt explizite belegte Tonindizes,
gruppenweise Leistungsnormierung und getrennte Nicht-HE-/Nutzdatenpfade.
Die neuen numerischen und echten Kanal-/PHY-Empfangstests sowie der tatsächliche
Validierungsstand stehen in Abschnitt 7.9. Der genaue Arbeitsplan steht in
`doc/mu-mimo-4.4-improvement-plan-2026-09-07.md`.

  UNTERPHASE 4.5: LAUFZEIT-CSI UND SCHEDULER-SCHNITTSTELLE (IMPLEMENTIERT)

- Einen gerichteten, zeitgestempelten CSI- und Precoderzustand pro AP,
  Nutzergruppe, RU und Übertragungsrichtung verwalten; veraltete oder für eine
  andere Gruppe erzeugte Zustände dürfen nicht wiederverwendet werden.
- An die vorhandene ns-3-`MultiUserScheduler`-Schnittstelle anbinden und
  zunächst deterministische Nutzergruppen sowie RU-/NSS-Zuordnungen
  unterstützen.
- Danach eine explizit auswählbare korrelationsbasierte Gruppierung ergänzen,
  welche unbelegbare Gruppen verwirft und ihre Auswahlmetrik ausgibt.
- CSI-Aktualisierungsintervall, Feedbackverzögerung und den tatsächlichen
  Sendezeitpunkt aus Phase 3 übernehmen. Quantisierte MU-Rückmeldung erhält
  einen eigenen Vertrag; ein rekonstruierter SU-Precoder darf nicht
  stillschweigend als vollständige MU-Kanalschätzung behandelt werden.
- Wiederholte Läufe mit identischen Seeds sowie kontrollierte Änderungen von
  Nutzerreihenfolge, Korrelation und CSI-Alter testen.

Abnahmekriterium: Scheduler und PHY verwenden nachweislich denselben
gerichteten Kanalschnappschuss; unpassende oder veraltete Zustände werden
erkannt und identische Eingaben erzeugen exakt dieselbe Nutzergruppe,
RU-/NSS-Zuordnung und denselben Precoder.

Dieses Kriterium ist erfüllt: DlMuMimoRuntime übernimmt Kandidaten und
RU-/NSS-Metadaten direkt aus MultiUserScheduler::DlMuInfo, bindet STA-IDs
explizit an stabile Kanalnutzer und liefert eine deterministische feste oder
korrelationsbasierte Gruppenauswahl. Erst wenn der Scheduler genau diese
Auswahl für denselben Sendezeitpunkt ausgibt, installiert Install den
unveränderten Snapshot und ZF-/RZF-Precoder im Phase-4.4-Adapter. Falsche,
veraltete, anders gerichtete oder nicht unterstützte Zustände werden vor
jeder Änderung abgelehnt. Unterphase 4.6 verwendet den getrennten Übertragungsschnappschuss und
ersetzt den CSI-Zustand dabei nicht.

  UNTERPHASE 4.6: DOWNLINK-BENUTZBARKEIT UND PHASENVALIDIERUNG (IMPLEMENTIERT)

- `SionnaMimoHelper` den Modus `MU_MIMO` erst nach erfolgreicher
  End-to-End-Integration als unterstützt melden lassen.
- Das Beispiel `ns3_wifi_mumimo.cc` anlegen und SU-Round-Robin, festes

MU-Precoding, MU-ZF und MU-RZF bei gleicher Gesamtleistung und vergleichbarer Sendezeit gegenüberstellen.
- Orthogonale und stark korrelierte Nutzergruppen sowie aktuelles und veraltetes CSI vergleichen.
- Pro Nutzer Durchsatz, PER, Stream-SINR, gewünschte Leistung, Inter-User-Interferenz, RU, NSS und CSI-Alter sowie für die Gruppe Rang, Precodingnorm, Gesamtleistung und Fairness maschinenlesbar ausgeben.
- Vollständige CPU-End-to-End- und Negativtests sowie alle Regressionen der Phasen 1 bis 3 ausführen.

Abnahmekriterium: MU-ZF erzielt bei ausreichend unkorrelierten Nutzern den erwarteten gleichzeitigen Übertragungsgewinn gegenüber SU-Round-Robin, ohne zusätzliche AP-Sendeleistung zu erzeugen; bei hoher Korrelation verschlechtert sich das Ergebnis nachvollziehbar oder die Gruppe wird abgelehnt. Beispiele benötigen keine manuelle Verdrahtung interner Objekte.

Dieses Kriterium ist erfüllt: `InstallDownlink` übernimmt einen AP, stabile Nutzerbindungen, Spectrumadapter und einen zeitabhängigen Snapshotanbieter in ein AP-eigenes `MultiUserScheduler`-Objekt. Die Installation verlangt zwei bis acht AP-Sendeantennen, eine Empfangsantenne und NSS eins je Station, eine explizite MU-Laufzeitkonfiguration sowie ein EESM-Beta für den verwendeten MCS. Am tatsächlichen `PhyTxPsduBegin`-Zeitpunkt wählt sie CSI erneut, behält die ausgegebene Gruppe bei und installiert den physischen Downlink-Snapshot getrennt vom CSI-Snapshot. Der Adapter verwendet damit auch für SU-Empfang und Mithörer die gerichtete Übertragungs-CFR.

Das CPU-Beispiel verwendet einen Zweiantennen-AP und zwei Einantennen-HE-Stationen auf Kanal 36 mit 20 MHz, 800 ns GI, 7 dB Rauschzahl, 0 dBm AP-Leistung, MCS 5, statischem ARP sowie einem MPDU je Nutzer und PPDU. Sein frequenzflacher Kanal lautet

$$
 h_1(t)=[1,e^{j\theta(t)}],\qquad
 h_2(t)=[1,e^{j(\theta(t)+\phi)}],\qquad
 \theta(t)=2\pi f t .
$$

Dabei sind $\phi=\pi$ und $\phi=0{,}2$ die orthogonale und korrelierte Variante; $f=25\,\mathrm{Hz}$. Aktuelles CSI verwendet Nullintervall und Nullverzögerung, altes CSI 5 ms und 2 ms. Der Verifikationslauf mit `--eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1` bestätigt die konstante AP-Leistung von 0,001 W, zwei adressierte Nutzer pro HE-MU-PPDU, endliches normalisiertes RZF, nichtverschwindende Alterungsinterferenz und den erwarteten ZF-Unterschied: orthogonales aktuelles ZF erreicht 75,226685740 Mbit/s je Airtime-Sekunde gegenüber 49,586776860 für SU-MRT. Die Ausgabe trennt tatsächliche PHY-Drops und empirische PER von der numerischen EESM/PER-Prognose.

UNTERPHASE 4.7: UPLINK-MU-NUMERIKKERN (IMPLEMENTIERT)

- Das Uplink-Signalmodell

$$
  y[k] = \sum_{u=1}^{U} H_u[k]x_u[k] + n[k]
$$

mit getrennten Nutzer-Sendeleistungen, Pfadverlusten und Streamzuordnungen definieren.
- Zunächst ideale Zeit- und Frequenzsynchronisation sowie einen Stream je Nutzer annehmen und diese Grenze ausdrücklich dokumentieren.
- Gemeinsame ZF- und lineare MMSE-Detektion am AP implementieren und Post-Processing-SINR, Inter-User-Interferenz und PER je Nutzer bestimmen.

  - Nah-Fern-Fälle, orthogonale Nutzer, Rangverlust, unterschiedliche
  Sendeleistungen und den Einzelnutzer-Grenzfall analytisch testen.

Abnahmekriterium: Der Einzelnutzerfall reproduziert die vorhandene
SU-Empfangsabstraktion; analytische Mehrnutzerkanäle und Nah-Fern-Fälle
liefern die erwarteten ZF-/MMSE-SINRs, ohne Nutzerleistungen zu vermischen.

Dieses Kriterium ist erfüllt: `UlMuMimoLinkModel` stapelt die gerichteten
Ein-Stream-Uplinkkanäle am gemeinsamen AP, hält jede Nutzer-SNR getrennt und
liefert für ZF beziehungsweise lineares MMSE gewünschte Leistung,
Inter-User-Interferenz, verstärkte Rauschleistung und SINR je Subcarrier.
`HeUlMuMimoLinkAbstraction` verwendet danach nutzerweise den vorhandenen
EESM-/ErrorRateModel-Vertrag. Der Einzelnutzer-, Orthogonal-, Rangverlust- und
Nah-Fern-Fall stimmen mit den analytischen Referenzen überein. HE-TB-/Trigger-
und Gleichzeitigkeitseinbindung bleiben ausschließlich Unterphase 4.8.

  UNTERPHASE 4.8: UPLINK-PHY-INTEGRATION UND GESAMTABSCHLUSS (IMPLEMENTIERT)

  - Den Uplink-Kern an koordinierte HE-TB-/UL-MU-Übertragungen, Triggerrahmen,
  STA-ID-, RU- und MCS-Zuordnungen anbinden.
  - Sicherstellen, dass nur tatsächlich gleichzeitig empfangene Nutzer in
  denselben Detektionszustand eingehen; fehlende, verspätete oder
  inkonsistent konfigurierte Teilnehmer werden explizit behandelt.
  - Das Phasenbeispiel um UL-MU gegenüber sequenziellen SU-Übertragungen
  erweitern und die ideale Synchronisationsannahme in der Ausgabe sichtbar
  machen.
  - Alle API-, Numerik-, PHY-, Scheduler-, CPU-End-to-End- und Legacytests
  ausführen sowie Dokumentation und PDF abschließend aktualisieren.

Abnahmekriterium: Eine koordinierte UL-MU-Gruppe liefert pro Nutzer
reproduzierbare SINR-, PER- und Durchsatzwerte; der erwartete
Gleichzeitigkeitsgewinn sowie Nah-Fern- und Rangverlusteffekte sind sichtbar,
und alle Phase-1- bis Phase-3-Regressionen bleiben erfolgreich.

Dieses Kriterium ist erfüllt: `HeUlMuMimoPhyAdapter` validiert einen
HE-Basic-Trigger gemeinsam mit den tatsächlich am AP beobachteten
`HePpdu`-Objekten. Stabile Nutzer-, STA- und Senderbindungen sowie Trigger-UID,
HE-TB-Präambel, PPDU-Dauer, NSS, MCS-Beta und AP-Antennendimension werden vor
der Rechnung geprüft. Nur PPDUs mit exakt dem expliziten gemeinsamen
Empfangsintervall werden nach RU gruppiert; verschiedene RUs bleiben
orthogonal. Fehlende und nicht gleichzeitige STA-IDs werden getrennt im
Ergebnis ausgewiesen. Das erweiterte CPU-Beispiel vergleicht dieselben
orthogonalen Nutzer sequenziell und gleichzeitig, kennzeichnet die ideale
Synchronisation maschinenlesbar und zeigt außerdem Nah-Fern- sowie
Rangverlustverhalten.

  PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

Der ursprüngliche Umfang bleibt: PER gegen unabhängige Link-Level-Daten
kalibrieren; Abstand, Orientierung, CSI-Alter und RT-Samplebudget untersuchen;
Cache-/IPC-Speicherbedarf messen; Seeds, Ergebnisformate und CPU/GPU-CI absichern.
Methoden stehen in 5.19, Ergebnisse und Grenzen in 7.13, Befehle in Abschnitt 9.

5.1 liefert einen validierten HE-MCS-0/BCC/32-Byte/1x1-Piloten.
5.2.1–5.2.5 liefern CPU-Studien für 2x2-Kanäle, Samplekonvergenz, lokale
Cache-/IPC-Skalierung, getrennte Arrayabstände und analytische CSI-Alterung.
Das sind begrenzte Abnahmen; Phase 5 insgesamt ist weiterhin offen.

Die bisherigen R1–R6 werden als jeweils **eine** Unterphase mit festen
Abnahmekriterien behandelt. Ihr verbleibender Stand ist:

Paket	Vorhanden	Noch unbelegt/offen
R1: Reproduzierbarkeit	native Seed/Run/Streams, portable Bundles, Lock-/Systemchecks und Reproduktion abgenommen	Container/Docker außerhalb des Umfangs
R2: Kalibrierung	Pilotprofil mit Laufzeitvertrag, tatsächlicher HE-SU-/PSDU-Anwendung und Negativtests	weitere Profile und Aggregation außerhalb dieser Abnahme
R3: Speicherung	Cachebudgets, Kopiereigentum, Zeit-/Link-/Tiefensweeps, begrenzte Historien und Stagingpool abgenommen	RT-/CIR-/Protobuf-Pools und RSS außerhalb des Umfangs
R4: Live-Skalierung	reale CPU-Server-/HE-SU-PHY-Messung, 42 Prozesspaare und Speicherbeobachtungen abgenommen	MU-/MAC-, GPU- und universelle Leistungsgrenzen außerhalb des Umfangs
R5: Physikalische Alterung	getrennte Bewegungs-/Doppler-Sounding-/Sendekanäle und CPU-Abnahme abgeschlossen	automatische Abläufe, zusätzliche Szenen und breitere Statistik außerhalb des Umfangs
R6: Backendabschluss	explizite Backendwahl, CPU-Abnahme, CUDA-CI-Workflow, gepaarter Vergleich und vollständiges Quell-/Ergebnisbundle implementiert	tatsächliche RTX-3060-Messung und CUDA-CI-Ausführung ausstehend

R1 wird als eine vollständige Unterphase im vorhandenen nativen Conda-CPU-Environment umgesetzt; Container/Docker gehören nicht zum Umfang. Feste Abnahme: versionierte Seed-/Run-/Streamzuordnung mit Reproduzierbarkeits- und Isolationstests; verschobene Bundles mit Konfigurationen, benötigten Eingängen, Ergebnissen und prüfbaren Hashes; Reproduktion eines kleinen RT-/Native-Beispiels; Prüfung des bestehenden Python-Locks und dokumentierter Systemabhängigkeiten; etablierte CPU-Regressionen sequenziell. Historische Messhashes und R2 bleiben unverändert. Resume, neue wissenschaftliche Studien und GPU-Abnahme gehören nicht zu R1.

Die verbleibende R3-Arbeit wurde als eine vollständige Unterphase abgeschlossen: begrenzte knotenbezogene
Mobilitätshistorien mit Wiederverwendung ihrer Arrays, historiesparende Look-ahead-Checkpoints mit Wiederherstellung bei Fehlern und ein begrenzter nativer Pool für leere Batch-Staging-Slots. Feste Abnahme: unveränderte deterministische Trajektorien und Look-ahead-Werte; endliche Retention bei Langläufen; nachgewiesene Allokations-wiederverwendung, unabhängige Getterwerte und atomare Datenvalidierung bei Fehlern; sequenzielle CPU-Regressionen. R1/R2 bleiben geschlossen. Live-RT/PHY-Skalierung, allgemeine CFR-/Protobuf-Allokatorersetzung und GPU-Nachweise gehören nicht dazu.

R4 wurde als eine vollständige Unterphase für den echten lokalen CPU-Server-/HE-SU-PHY-Pfad umgesetzt. Vorab festgelegter Umfang: getrennte frische Server-/ns-3-Prozesse, feste Szene `simple_room`, 1/4 Empfänger, 1x1/2x2, 20/40 MHz, Cache an/aus und Look-ahead-Tiefe 1/4 in sechs repräsentativen Kombinationen; je drei Prozesspaare bei 10000 RT-Samples, zusätzlich die Cachepaarung des 4-Empfänger-/2x2-/40-MHz-/Tiefe-1-Falls bei 100000 Samples. Vier Epochen mit je zwei echten HE-SU-PPDUs bei 20 dBm bilden den Arbeitsumfang. Abnahme: produktive `SionnaEnv.compute_cfr`-Antworten mit geprüfter Identität, Zeit, Dimension und endlichen Koeffizienten; echte PHY-Empfangsereignisse; numerisch konsistente statische Cachepaarungen; getrennte Kalt-/Folgelatenzen und Prozess-RSS/HWM; Cache-/Pool-/Mobilitätsnutzbytes separat; erhaltene Fehlerartefakte und sequenzielle CPU-Regressionen. Zeit-/RSS-Werte sind deskriptive Beobachtungen mit festem Samplebudget, keine Leistungsgrenze oder Konvergenzfreigabe. MU-/MAC-Scheduling bleibt spätere Arbeit; R5 schließt die begrenzte physikalische CPU-Alterung ab, GPU-Abnahme bleibt R6. Weitere Optimierungen gehören nicht zu diesem R4-Abschluss.

Weitere Profile, Szenen und Optimierungen sind Erweiterungen; sie dürfen bestehende Abnahmen nicht nachträglich verschärfen oder neue Inkrementketten begründen. Automatisches Sounding und Synchronisationsfehler bleiben spätere Arbeit.

4. DATENMODELL UND IPC-VERTRAG DER UNTERPHASEN 1.1/1.2

4.1 TENSOR UND FLACHE REIHENFOLGE

Für jeden Subcarrier k ist

$$
  \mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container und die beiden Protocol-Buffer-Felder csi_real und csi_imag
speichern die Werte flach. Der Index für den Eintrag h[k,r,t] lautet

$$
  i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Es gelten die
verbindlichen Längen

$$
  |\mathrm{frequencies}| = N_\mathrm{sc},
  \qquad
  |\mathrm{csi_real}| = |\mathrm{csi_imag}|
  = N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}.
$$

Die neuen skalaren Felder num_rx_antennas und num_tx_antennas wurden mit den
Feldnummern 9 und 10 an RxNodeInfo angehängt. Der Proto3-Standardwert null wird
beim Empfänger als eins interpretiert. Für eine neue 1x1-Antwort lässt der
Python-Server beide Felder bewusst ungesetzt; ihre serialisierte Darstellung
ist dadurch bytegleich zur alten Nachricht. Alte Sender werden als 1x1 gelesen,
alte Empfänger ignorieren die unbekannten neuen Felder einer MIMO-Antwort.

4.2 TRANSFORMATION AUS SIONNA

Sionna RT 2.0.1 liefert die CFR weiterhin in der Form

$$
  H_\mathrm{raw}
  [\ell,r,q,t,n,k]
$$

mit Linkindex ℓ, RX-Antenne r, TX-Index q, TX-Antenne t, Zeitindex n und
Subcarrier k. Eine ns3sionna-Anfrage enthält genau einen Sender und einen
Zeitpunkt. Deshalb wird ohne squeeze-Operation explizit ausgewählt:

$$
  H_\ell[r,t,k] =
  H_\mathrm{raw}[\ell,r,0,t,0,k].
$$

Vor der Serialisierung wird nach [k,r,t] transponiert und anschließend in
C-Reihenfolge abgeflacht. Dadurch bleiben auch Singleton-Dimensionen sichtbar
und 2x2-Werte landen indexgenau im C++-Container.

4.3 WIDEBAND-VERLUST UND MIMO-NORMALISIERUNG

Für die rohe Link-CFR wird die mittlere Leistung gemeinsam über Subcarrier und
Antennenpaare gebildet:

$$
  \bar P =
  \frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
$$

$$
  \sum_{k=0}^{N_\mathrm{sc}-1}
  \sum_{r=0}^{N_\mathrm{rx}-1}
  \sum_{t=0}^{N_\mathrm{tx}-1}
  |H_\ell[r,t,k]|^2.
$$

Der übertragene skalare Wideband-Verlust und die normalisierte Matrix lauten

$$
  L_\mathrm{wb} = -10\log_{10}(\bar P)\ \mathrm{dB},
  \qquad
  H_\mathrm{norm}[r,t,k] =
  \frac{H_\ell[r,t,k]}{\sqrt{\bar P}}.
$$

Damit gilt

$$
  \frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
  \sum_{k,r,t}|H_\mathrm{norm}[r,t,k]|^2 = 1.
$$

Für $N_{\mathrm{rx}} = N_{\mathrm{tx}} = 1$ ist dies exakt die bisherige SISO-Normalisierung. Die Aufteilung
der gesamten Sendeleistung auf N_s Streams bleibt davon getrennt und erfolgt weiterhin im numerischen Modell mit dem Faktor $1/N_s$.

4.4 ARRAYKONFIGURATION UND NS-3-CACHE

SimInitMessage besitzt optionale tx_array- und rx_array-Nachrichten mit Zeilen, Spalten, vertikalem und horizontalem Abstand in Wellenlängen, Elementmuster und Polarisation. Ohne Nachricht werden exakt die alten Arrays verwendet: 1x1/tr38901/V am Sender und 1x1/dipole/V am Empfänger. SionnaHelper bietet dafür die kleinen Methoden ConfigureTransmitArray und ConfigureReceiveArray; die umfassende knotenbezogene High-Level-API bleibt Phase 2 vorbehalten.

Die Antwortdimensionen werden aus der tatsächlich von Sionna gelieferten Form abgeleitet und berücksichtigen damit auch die Polarisation. CacheEntry hält die CFR nun als MimoChannelMatrix. GetPropagationChannelMatrix erhält die Dimensionen; GetPropagationCSI bleibt als flache, für 1x1 unveränderte Kompatibilitätsansicht bestehen.

4.5 MEHRNUTZER-SNAPSHOT UND GERICHTETE IDENTITÄT DER UNTERPHASE 4.1

Die bestehende gebündelte Antwort bleibt der Transportcontainer. Eine `ChannelState`-Nachricht bezeichnet genau einen CSI-Zeitpunkt und enthält weiterhin einen gemeinsamen Sender sowie mehrere `RxNodeInfo`-Nachrichten. Feld 11 von `RxNodeInfo` ergänzt additiv die Nachricht `UserChannelIdentity` mit

$$
  I_u=(u, n_{\mathrm{tx}}, n_{\mathrm{rx}}, d, t_{\mathrm{csi}}, A_{\mathrm{rx},u}).
$$

Hier sind u die stabile Nutzer-ID, n_{tx} und n_{rx} die gerichteten Endpunkte, d die explizite Richtung und $A_{\mathrm{rx},u}$ die vollständige Empfangsarraygeometrie. Unterphase 4.1 erzeugt ausschließlich `CHANNEL_DIRECTION_DOWNLINK`; Uplink wird damit nicht vorweggenommen. Fehlt `UserChannelIdentity`, bleibt die vorhandene SU-/Legacy-Deserialisierung unverändert.

Für einen gültigen Snapshot gelten gleichzeitig

$$
  t_{\mathrm{csi},u}=t_{\mathrm{start}},\qquad
  f_{u,k}=f_k,\qquad
  N_{\mathrm{sc},u}=N_{\mathrm{sc}},\qquad
  N_{\mathrm{tx},u}=N_{\mathrm{tx}}.
$$

Die Empfangsdimension darf dagegen nutzerabhängig sein. Für ein planares Array mit R_u Zeilen, C_u Spalten und $P_u \in \{1,2\}$ Polarisationsports gilt verbindlich

$$
  N_{\mathrm{rx},u}=R_u\, C_u\, P_u.
$$

Jeder Nutzer trägt sein eigenes $A_{\mathrm{rx},u}$; die bisherige globale RX-Konfiguration wird nicht mehr stillschweigend aus einem anderen Link abgeleitet. Der aktuelle Server schreibt für jeden von ihm erzeugten Mehrnutzer-Link die tatsächlich konfigurierte globale Geometrie explizit aus; Einzelnutzerantworten bleiben identitätsfrei. Der IPC- und C++-Vertrag kann bereits unterschiedliche Geometrien und $N_{\mathrm{rx},u}$ innerhalb desselben synthetischen Snapshots abbilden; eine knotenbezogene High-Level-Konfiguration des Sionna-Raytracings gehört nicht zu dieser Unterphase.

Die CFR jedes Nutzers verwendet unverändert die Reihenfolge $i_u(k,r,t) = (kN_{\mathrm{rx},u} + r)N_{\mathrm{tx}} + t$, einen eigenen skalaren Wideband-Pfadverlust und die Einheitsleistungsnormalisierung

$$
  \frac{1}{N_{\mathrm{sc}}N_{\mathrm{rx},u}N_{\mathrm{tx}}}
  \sum_{k,r,t}|H_u[k,r,t]|^2=1.
$$

`validate_multi_user_snapshot` prüft denselben Vertrag auf der Python-Seite. `DeserializeMultiUserChannelSnapshot` prüft in C++ zunächst sämtliche Nutzer, ohne Cachezustand zu verändern, und baut erst danach `MultiUserChannelSnapshot` auf. Dessen Nutzervektor ist unabhängig von der Eingabereihenfolge streng aufsteigend nach u sortiert. Gemischte oder fehlende Identitäten, doppelte Nutzer, falsche Endpunkte/Richtungen, Zeitstempel, Frequenzraster, TX-Dimensionen, nicht passende RX-Geometrien, nicht endliche Werte und abweichende Normalisierung werden abgelehnt.

Auch Antworten mit mehreren Look-ahead-Zeitpunkten werden vollständig in temporäre Einträge dekodiert, bevor der bestehende Cache einen davon übernimmt. Explizit identifizierte Einträge dürfen nur in ihrer gespeicherten TX-RX-Richtung getroffen werden. Identitätsfreie alte SU-Einträge behalten dagegen die bisherige reziproke Schlüsselverwendung. Damit bleibt der SU-Datenpfad erhalten, während Mehrnutzerzustände weder partiell noch in umgekehrter Richtung sichtbar werden.

5. NUMERISCHES SU- UND DOWNLINK-MU-MIMO-MODELL

5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
  \mathbf{y}[k] =
$$

$$
  \sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

mit $N_s = N_{\mathrm{tx}}$, $E[xx^H] = I$ und räumlich weißem Rauschen
$E[nn^H] = NI$. Die Eingangsgröße

$$
  \rho = \frac{P}{N}
$$

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell
verlangt in dieser Unterphase $N_{\mathrm{rx}} \geq N_{\mathrm{tx}}$. Overloaded MIMO wird erst mit
einer
expliziten Streamauswahl zugelassen.

5.2 LINEARER MMSE-EMPFÄNGER

Mit $\alpha = \rho / N_s$ und $G[k] = H[k]^H H[k]$ folgt die
Fehlerkovarianz des linearen MMSE-Detektors:

$$
  \mathbf{E}[k] =
  (\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
$$

Die Post-Processing-SINR des Streams s ist

$$
  \gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
$$

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und
Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch
definiert, weil I + alpha G positiv definit ist.

5.3 SISO-GRENZFALL

Für $N_{\mathrm{tx}} = N_{\mathrm{rx}} = 1$ ergibt sich

$$
  E = \frac{1}{1 + \rho |h|^2},
  \qquad
  \gamma = \rho |h|^2.
$$

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem
üblichen SISO-SNR-Modell.

5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine
Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

$$
  \gamma_{\mathrm{eff},s} =
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
$$

Die mittlere ideale spektrale Effizienz über alle Streams ist

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese MI-Größe bleibt eine Validierungsmetrik und ist keine unmittelbare 802.11-Datenrate oder PER. Die HE-PER-Abbildung verwendet stattdessen die MCS-spezifische EESM-Kompression aus Abschnitt 5.6.

5.5 REFERENZ-SNR AUS DEM LINK BUDGET

Die skalare Wideband-Dämpfung L_wb bleibt von der normalisierten Small-Scale-CFR getrennt. Aus der gesamten Sendeleistung $P_\mathrm{tx,dBm}$ folgen Empfangsleistung und lineare Empfangsleistung

$$
  P_\mathrm{rx,dBm} = P_\mathrm{tx,dBm} - L_\mathrm{wb},
  \qquad
  P_\mathrm{rx} =
  10^{(P_\mathrm{rx,dBm}-30)/10}\ \mathrm{W}.
$$

Mit Rauschbandbreite B in Hz, Temperatur T in Kelvin und Empfängerrauschzahl NF in dB gelten

$$
  N_\mathrm{th} = kTB,
  \qquad
  F = 10^{NF/10},
  \qquad
  N_\mathrm{rx} = FkTB,
$$

und damit

$$
  \rho =
  \frac{P_\mathrm{rx}}{N_\mathrm{rx}}
  =
  \frac{10^{(P_\mathrm{tx,dBm}-L_\mathrm{wb}-30)/10}}
    {kTB\,10^{NF/10}}.
$$

WifiLinkBudget gibt Empfangsleistung, thermisches Rauschen, Empfängerrauschen und ρ separat aus. dB-/Linear- und dBm-/Watt-Umrechnungen delegieren an DbToRatio, RatioToDb und DbmToW aus ns-3.40 wifi-utils. Für $k=1{,}3803\cdot10^{-23}\ \mathrm{J/K}$ wird bewusst derselbe Zahlenwert wie in ns-3.40 InterferenceHelper verwendet. Das ist die Provenienz dieser physikalischen Konstante; es ist keine Fit-Konstante. Anders als InterferenceHelper ist T hier ein expliziter positiver Eingabeparameter und nicht fest auf 290 K gesetzt.

P_tx ist die Gesamtleistung über alle Spatial Streams; die Aufteilung durch N_s erfolgt erst im MMSE-Modell. B ist die Empfänger-Rauschbandbreite. Link Budget und MMSE addieren selbst keine externe Interferenz; der SpectrumWifiPhy-Pfad bewertet sie anschließend skalar gemäß Abschnitt 5.8. Ein zusätzlicher pauschaler RX-Diversity-Gewinn wird nicht aufgeschlagen: Array- und Detektorgewinn sind bereits in $\mathbf H[k]$ und den Post-MMSE-SINRs enthalten.

5.6 MCS-ABHÄNGIGE EESM-KOMPRESSION

Für HE MCS m komprimiert EESM alle $N_\mathrm{sc}N_s$
Post-MMSE-SINRs mit gleicher Gewichtung:

$$
\gamma_{\mathrm{EESM},m}
=
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k=0}^{N_\mathrm{sc}-1}
  \sum_{s=0}^{N_s-1}
  \exp\left(-\frac{\gamma_s[k]}{\beta_m}\right)
\right).
$$

$\beta_m>0$ hat wie γ lineare SINR-Einheiten und ist
MCS-abhängig. SetEesmBeta erwartet den linearen Wert; SetEesmBetaDb ist die
dB-Komfortfunktion. Zur Vermeidung numerischen Unterlaufs wird mit
$\gamma_\min=\min_{k,s}\gamma_s[k]$ äquivalent gerechnet:

$$
\gamma_{\mathrm{EESM},m}
=
\gamma_\min
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k,s}
  \exp\left(-\frac{\gamma_s[k]-\gamma_\min}{\beta_m}\right)
\right).
$$

Für einen flachen Kanal mit identischen SINRs ist das Ergebnis unabhängig von
β_m exakt diese SINR. Das ermöglicht die SISO-AWGN-Paritätsprüfung.

Es werden bewusst keine Default-Betas ausgeliefert. Jeder HE MCS von 0 bis 11
muss vor der Nutzung explizit konfiguriert werden. Damit enthält der Code keine
unbelegten Kalibrierkonstanten. Die in analytischen Tests verwendeten positiven
Werte sind ausschließlich Formel-Sentinels und ausdrücklich keine
802.11ax-Kalibrierung. Unterphase 5.1 versioniert erstmals einen
Link-Level-Fit für HE MCS 0/BCC/32 Byte/1x1 einschließlich einer getrennten
AWGN-SNR-Korrektur. Andere MCS und PHY-Konfigurationen benötigen eigene
Referenzen. Die bisherige MI-Kompression je
Stream bleibt als modellunabhängige Diagnosemetrik erhalten und wird nicht als
kalibrierte PER-Abbildung ausgegeben.

5.7 ABBILDUNG AUF NS-3-PER

HeSuMimoLinkAbstraction übernimmt WifiTxVector einschließlich HE-MCS, NSS und
Codeart. NSS muss der Zahl der übertragenen Streams entsprechen. Nach EESM wird
die vorhandene Schnittstelle verwendet:

$$
\gamma_{\mathrm{model}}=
\gamma_{\mathrm{EESM},m}10^{\delta_c/10},\qquad
P_\mathrm{succ} =
\mathrm{ErrorRateModel::GetChunkSuccessRate}
(m,\mathrm{TxVector},\gamma_{\mathrm{model}},8L,N_\mathrm{rx}),
\qquad
PER = 1-P_\mathrm{succ},
$$

\]

wobei \(L\) die Paketlänge in Byte ist. Ohne eine exakt passende
Phase-5.1-Kalibrierung gilt \(\delta_c=0\). ErrorRateModel ist als ns-3-Attribut
und über SetErrorRateModel konfigurierbar. Standard ist
TableBasedErrorRateModel, also das ns-3.40-Defaultmodell für 802.11n/ac/ax.

Die Kalibrierprovenienz bleibt vollständig bei ns-3: Laut
src/wifi/doc/source/wifi-design.rst stammen die AWGN-PER-Tabellen aus
End-to-End-Linksimulationen mit MATLAB WLAN Toolbox, wurden gegen IEEE-TGn-
Ergebnisse validiert und verwenden die Paketlängenextrapolation aus
IEEE P802.11 TGax 11-14/0803r1. BCC besitzt Referenztabellen für 32 und
1458 Byte mit einer 400-Byte-Schwelle und MCS 0 bis 9; MCS 10/11 fällt auf
YansErrorRateModel zurück. LDPC besitzt eine 1458-Byte-Referenz für MCS 0 bis
11. Die Abstraktion kopiert oder verändert keine dieser Kurven. Die
Phase-5.1-Korrektur verschiebt ausschließlich die ErrorRateModel-Eingabe
für eine exakt passende PHY-Konfiguration. Da sie Erfolgswahrscheinlichkeiten und keine
Zufallsentscheidung ausgibt, ist die Berechnung bei identischen Eingaben
deterministisch.

5.8 EINKOPPLUNG IN SPECTRUMWIFIPHY

Die Integration bleibt auf
SionnaSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity beschränkt.
Für einen HE-SU-PPDU liest sie \(N_s\) ausschließlich mit
WifiTxVector::GetNss. Vor der Link-Abstraktion müssen
die physikalischen TX-/RX-Antennenzahlen gleich den CFR-Dimensionen, die
maximal unterstützten Spatial Streams mindestens \(N_s\) und
\(N_\mathrm{ant,rx}\geq N_s\) sein. Ohne konfiguriertes Precoding bleibt
zusätzlich unverändert \(N_s=N_\mathrm{tx}\). Mit festem Precoding muss
\(N_s\) dagegen exakt der Spaltenzahl von W[k] entsprechen. Inkonsistenzen
brechen damit vor MMSE, EESM und Paketempfang ab. Der neue kontrollierte Fall
\(N_s<N_\mathrm{tx}\) ist ausschließlich mit explizitem festem W[k] zulässig.

Der SpectrumChannel hat den skalaren Wideband-Pfadverlust bereits auf
params->psd angewendet. Die MIMO-Integration verwendet daher

\[
  P_\mathrm{in} =
  \int S_\mathrm{params}(f)\,\mathrm df \; G_\mathrm{rx}
\]

als bereits pfadverlustbehaftete Empfangsleistung. Der Wideband-Verlust aus
dem Cache wird an dieser Stelle nur noch für den bestehenden Diagnose-Tag
gelesen und nicht erneut multipliziert. Empfangsverstärkung,
Rauschbandbreite, 290 K und Empfängerrauschzahl gehen genau einmal in das
Link Budget ein. Weil ns-3.40 die Rauschzahl nicht lesbar als WifiPhy-Attribut
bereitstellt, verlangt SetReceiverNoiseFigureDb einen expliziten Wert; er muss
mit SpectrumWifiPhy::RxNoiseFigure identisch konfiguriert werden. Ein
fehlender Wert bricht den MIMO-Pfad früh ab.

MMSE und EESM liefern \(\gamma_\mathrm{eff}\). SpectrumWifiPhy erwartet
dagegen weiterhin eine skalare PSD und wendet seine InterferenceHelper-
Berechnung später selbst an. Für das AWGN-Fehlermodell ist dessen interner
Diversity-Faktor

\[
  G_\mathrm{AWGN} =
  \begin{cases}
    N_\mathrm{ant,rx}/N_s, & N_\mathrm{ant,rx}>N_s,\\
    1, & \text{sonst}.

```
  \end{cases}
\]

Da der MMSE-Schritt den Arraygewinn bereits enthält, wird die äquivalente
skalare Empfangsleistung auf

\[
  P_\mathrm{eq} =
  \frac{\gamma_\mathrm{eff} P_N}{G_\mathrm{AWGN}},
  \qquad P_N = kTB\,10^{NF/10}
\]

gesetzt. InterferenceHelper erhält damit bei zusätzlicher skalarer
Interferenz \(I\)

\[
  \gamma_\mathrm{phy}
  = \frac{P_\mathrm{eq}}{P_N+I}G_\mathrm{AWGN}
  = \gamma_\mathrm{eff}\frac{P_N}{P_N+I}.
\]

Ohne Interferenz gilt exakt
\(\gamma_\mathrm{phy}=\gamma_\mathrm{eff}\); Pfadverlust, thermisches
Rauschen, Rauschzahl und Arraygewinn werden nicht doppelt angewendet.
Zeitvariable Interferenz wirkt weiterhin über SpectrumWifiPhy, jedoch als
skalare Nachbewertung und nicht als subcarrierabhängige
Interferenzkovarianz im MMSE-Detektor.

Der bisherige 1x1-Pfad bleibt unverändert skalar und benötigt kein EESM-Beta.
Nicht-HE-Kontrollrahmen einer MIMO-PHY verwenden während Phase 1 das erste
Antennenpaar, damit Beacon, Association und ACK ohne vorweggenommene
Streamauswahl funktionieren. Der letzte Guard-Bin und das bestehende
CFR-Tag-Format bleiben im 1x1-Fall kompatibel. Für automatisierte Beispiele
kann eine normalisierte feste CFR den Sionna-Cache ersetzen; dieser
deterministische Test-Hook ändert den Produktionspfad nicht. Der MimoRx-Trace
meldet PPDU-UID, Knoten, Effective-SINR, PER und den kleinsten Kanalrang über
alle Subcarrier.

Das Beispiel ns3_wifi_sumimo_minimal.cc erzeugt drei flache, normierte Kanäle
bei HE MCS 5 und 20 dB Referenz-SNR: 1x1 mit \(h=1\), gut konditioniertes 2x2
mit \(\mathrm{diag}(\sqrt{2},\sqrt{2})\) und Rang-1-2x2 mit ausschließlich
Einsen. Zusätzlich führt es den gut konditionierten Kanal mit explizitem
Identitäts-Precoding und mit einem 2x1-Precoder zur Auswahl der ersten
TX-Antenne aus. Es misst realen UDP-Durchsatz über SpectrumWifiPhy und gibt
zusätzlich PER, Stream-SINR, EESM-SINR und Mindestkanalrang aus. Der Aufruf
muss ein positives --eesmBetaLinear liefern. Der im Regressionstest verwendete
Wert 2 ist nur ein expliziter Formel-Sentinel; bei diesen flachen Kanälen ist
das Ergebnis betaunabhängig und stellt keine Produktionskalibrierung dar.

Die deterministische Referenzausgabe lautet:

  - 1x1: Rang 1, Stream-/Effective-SINR 20,000 dB, PER 0,000016 und
  68,1408 Mbit/s;
  - gut konditioniertes 2x2: Rang 2, beide Stream-SINRs und Effective-SINR
  20,000 dB, PER 0,000016 und 134,3904 Mbit/s;
  - Rang-1-2x2: Rang 1, beide Stream-SINRs und Effective-SINR -0,043214 dB,
  PER 1 und 0 Mbit/s;
  - Identitäts-Precoding 2x2: dieselben analytischen SINR-, EESM- und
  PER-Ergebnisse wie der offene Vollrangpfad und 133,4784 Mbit/s im
  sequenziellen Netzwerkregressionslauf;
  - 2x1-Streamauswahl auf dem 2x2-Kanal: Rang 1, Stream-/Effective-SINR
```

  23,010300 dB, PER 0 und 68,1504 Mbit/s. Der 3,0103-dB-Unterschied zum
  offenen Einzelstreamvergleich folgt aus der vollen Gesamtleistung für
  \(N_s=1\), nicht aus zusätzlicher Precoderleistung.

  5.9 FESTES PRECODING DER UNTERPHASE 3.1

MimoPrecodingMatrix ist ein unveränderlicher, dimensionssicherer Werttyp für

\[
  \mathbf W[k]\in\mathbb C^{N_\mathrm{tx}\times N_s},
  \qquad
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx}).
\]

Seine flache Reihenfolge ist verbindlich subcarrier/tx/stream:

\[
  j(k,t,s)=(kN_\mathrm{tx}+t)N_s+s.
\]

Der frequenzselektive Konstruktor erwartet deshalb genau
\(N_\mathrm{sc}N_\mathrm{tx}N_s\) komplexe Koeffizienten. FrequencyFlat
erwartet genau \(N_\mathrm{tx}N_s\) Werte in tx/stream-Reihenfolge und
expandiert sie beim Erzeugen des Werttyps explizit auf alle \(N_\mathrm{sc}\)
Subcarrier. Intern existiert damit kein implizites Broadcasting und jeder
Zugriff besitzt dieselben drei Dimensionen. Alle Real- und Imaginärteile
müssen endlich sein.

Für jeden Subcarrier gilt verpflichtend

\[
  \Vert\mathbf W[k]\Vert_F^2
  =\sum_{t=0}^{N_\mathrm{tx}-1}\sum_{s=0}^{N_s-1}|w_{t,s}[k]|^2
  =N_s.
\]

Die absolute Prüftoleranz beträgt \(10^{-10}\). Eine verletzte Norm wird
abgelehnt und nicht stillschweigend korrigiert. Mit
\(\mathrm E[\mathbf x\mathbf x^H]=\mathbf I\) lautet das erweiterte
Signalmodell

\[
  \mathbf y[k]=
  \sqrt{\frac{P}{N_s}}\,\mathbf H[k]\mathbf W[k]\mathbf x[k]
  +\mathbf n[k].
\]

Damit bleibt \(P\) die gesamte Sendeleistung und

\[
  \mathbf H_\mathrm{eff}[k]=\mathbf H[k]\mathbf W[k]
  \in\mathbb C^{N_\mathrm{rx}\times N_s}.
\]

Der bestehende MMSE-Kern wird ohne neuen Detektortyp auf
\(\mathbf H_\mathrm{eff}\) angewendet. Mit
\(\alpha=\rho/N_s\) gilt weiterhin

\[
  \mathbf E[k]=
  \left(\mathbf I_{N_s}+\alpha

$$
  \mathbf H_\mathrm{eff}^H[k]\mathbf H_\mathrm{eff}[k]\right)^{-1},
  \qquad
  \gamma_s[k]=\frac{1}{E_{s,s}[k]}-1.
$$

SuMimoLinkModel, HeSuMimoLinkAbstraction und der Spectrumadapter besitzen dafür explizite Precoding-Pfade. Ihre bisherigen Überladungen ohne W[k] bleiben bestehen und rechnen unverändert direkt mit H[k]. Insbesondere nimmt der bytegleiche 1x1-IPC-Vertrag kein neues Feld auf: Sionna liefert weiterhin ausschließlich H[k], W[k] existiert nur in ns-3.

SionnaMimoConfiguration enthält optional precodingMatrix. Validate prüft vor jeder Helper-Zustandsänderung die TX-Dimension, die Gleichheit von maxNss und W-Spalten sowie $N_s\leq N_\mathrm{rx}$. Apply ersetzt den Adapterzustand entweder durch den vollständig validierten Precoder oder löscht ihn explizit, sodass erneutes Anwenden einer Phase-2-Konfiguration kein altes W[k] zurücklässt. Am Laufzeitpfad werden zusätzlich CFR-Subcarrierzahl und beide CFR-Antennendimensionen geprüft, bevor ErrorRateModel- oder Empfangszustand verändert wird. WifiTxVector::GetNss muss bei vorhandenem W[k] exakt dessen Spaltenzahl, andernfalls weiterhin exakt N_tx sein.

Unterphase 3.1 implementiert benutzergegebenes festes Precoding, Unterphase 3.2 ergänzt die CFR-abhängige MISO-MRT-Fabrik und Unterphase 3.3 die deterministische allgemeine SVD-Fabrik mit gleicher oder wassergefüllter räumlicher Leistung. Alle erzeugen unveränderliche W[k]-Schnappschüsse, die anschließend denselben H[k]W[k]-Pfad durchlaufen. CSI-Rückmeldung, CSI-Alterung, automatische Laufzeit-Beamformingmodi und MU-MIMO bleiben außerhalb dieses Vertrags.

5.10 EIN-STREAM-MRT/MRC DER UNTERPHASE 3.2

Für den eindeutig definierten MISO-Fall $N_\mathrm{rx}=1$ sei die CFR je Subcarrier der Zeilenvektor

$$
  \mathbf h[k]\in\mathbb C^{1\times N_\mathrm{tx}}.
$$

Bei $\Vert\mathbf h[k]\Vert_2>0$ erzeugt MimoPrecodingMatrix::MaximumRatioTransmission aus perfekter CFR genau einen Precodervektor

$$
  \mathbf w_\mathrm{MRT}[k]
  =\frac{\mathbf h^H[k]}{\Vert\mathbf h[k]\Vert_2},
  \qquad
  \Vert\mathbf w_\mathrm{MRT}[k]\Vert_2^2=1=N_s.
$$

Damit erfüllt das Ergebnis direkt den Leistungsvertrag aus Abschnitt 5.9 und der effektive Kanal ist

$$
  \mathbf h[k]\mathbf w_\mathrm{MRT}[k]
  =\Vert\mathbf h[k]\Vert_2.
$$

Der unveränderte Ein-Stream-MMSE-Kern liefert folglich

$$
  E[k]=\frac{1}{1+\rho\Vert\mathbf h[k]\Vert_2^2},
$$

$$
  \qquad
  \gamma[k]=\rho\Vert\mathbf h[k]\Vert_2^2.
$$

Die Normierung erfolgt unabhängig für jeden Subcarrier; das zurückgegebene W[k] besitzt deshalb dieselbe sc/tx/stream-Reihenfolge wie jeder feste Precoder. Eine gemeinsame Phasendrehung $\mathbf h'[k]=e^{j\phi}\mathbf h[k]$ führt zu $\mathbf w'[k]=e^{-j\phi}\mathbf w[k]$ und lässt den effektiven Kanal sowie die SINR invariant. Besitzt auch nur ein Subcarrier Norm null, wird der gesamte Erzeugungsaufruf eindeutig mit std::invalid_argument abgelehnt. Es wird weder eine beliebige Strahlrichtung gewählt noch Energie erzeugt. Nicht endliche CFR-Koeffizienten werden ebenfalls abgelehnt.

Für $N_\mathrm{tx}=1$ ist $\mathbf H[k]=\mathbf h[k]\in\mathbb C^{N_\mathrm{rx}\times1}$. Dann ist die Richtung des linearen MMSE-Kombinierers kollinear zu $\mathbf h[k]$ und die bestehende Lösung reduziert sich ohne neuen Empfängertyp auf MRC:

$$
  \gamma[k]=\rho\,\mathbf h^H[k]\mathbf h[k]
  =\rho\Vert\mathbf h[k]\Vert_2^2.
$$

Im 1x1-Fall entfernt MRT nur die Kanalphase; der Betrag und damit das bisherige skalare Ergebnis bleiben exakt erhalten. Für $N_\mathrm{tx}>1$ und $N_\mathrm{rx}>1$ weist die MRT-Funktion den Aufruf bewusst ab. Eine optimale Ein-Stream-Richtung in diesem allgemeinen Fall ist der stärkste rechte Singulärvektor und gehört ausschließlich in Unterphase 3.3.

Der ideale numerische Aufruf lautet:

```
    const auto mrt =
    MimoPrecodingMatrix::MaximumRatioTransmission(channelAtCsiTime);
    const auto result = SuMimoLinkModel::Calculate(channelAtCsiTime, mrt, rho);
```

Das Ergebnis ist ein unveränderlicher W[k]-Schnappschuss für die übergebene perfekte CFR. Unterphase 3.2 führt absichtlich keinen automatischen Helper-/SpectrumWifiPhy-Beamformingmodus, keine CSI-Zeitachse und keine periodische Aktualisierung ein. Der erzeugte Precoder kann wie jeder feste 2x1-Precoder durch die bereits validierten Pfade aus Unterphase 3.1 geführt werden; die darauf aufbauende end-to-end Modusauswahl ist nun in Abschnitt 6.7 als Teil der abgeschlossenen Unterphase 3.5 beschrieben.

5.11 DETERMINISTISCHES SVD-PRECODING UND LEISTUNGSALLOKATION DER UNTERPHASE 3.3

Für jeden Subcarrier berechnet MimoPrecodingMatrix::SingularValueDecomposition aus der explizit übergebenen perfekten CFR eine dünne Jacobi-SVD

$$
  \mathbf H[k]=\mathbf U[k]\boldsymbol\Sigma[k]\mathbf V^H[k],
  \qquad
  \sigma_0[k]\geq\sigma_1[k]\geq\ldots\geq0.
$$

Die Streamzahl wird nicht aus dem Kanal erraten, sondern als N_s explizit übergeben und muss

$$
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})
$$

erfüllen. Der numerische Rang je Subcarrier ist

$$
  r[k]=\#\left\{i:\sigma_i[k]>
  \tau_\mathrm{rank}\sigma_0[k]\right\}.
$$

Der dokumentierte Standard ist
$\tau_\mathrm{rank}=10^{-12}$; ein benutzergegebener Wert muss endlich
und strikt in (0,1) liegen. Gilt auf auch nur einem Subcarrier
$r[k]<N_s$, wird der gesamte Fabrikaufruf mit Subcarrierindex, geforderter
Streamzahl und gefundenem Rang abgelehnt. Es gibt weder eine versteckte
Regularisierung noch eine stille Verringerung von N_s. Nicht endliche
CFR-Koeffizienten werden vor der Zerlegung zurückgewiesen.

Die von Eigen absteigend gelieferten Singularwerte erhalten zusätzlich eine
explizite deterministische Vektorkonvention. Zwei Werte derselben Zerlegung
gelten bei

$$
  |\sigma_i-\sigma_j|\leq10^{-12}\sigma_0
$$

als numerisch gleich. Für jede solche entartete Gruppe wird nicht die vom
SVD-Algorithmus beliebig gewählte Basis übernommen. Stattdessen bildet ns-3
den eindeutigen Unterraumprojektor
$\mathbf P=\mathbf V_g\mathbf V_g^H$, projiziert die kanonischen
TX-Koordinatenachsen und orthonormalisiert sie deterministisch. Je Schritt
wird die Achse mit der größten Residualnorm gewählt; numerisch gleiche
Kandidaten verwenden den kleinsten TX-Index. Abschließend wird jeder
Singulärvektor so phasengedreht, dass sein betragsgrößter Eintrag reell und
nicht negativ ist; bei gleichem Betrag gewinnt ebenfalls der kleinste
TX-Index. Damit sind Reihenfolge, Vorzeichen beziehungsweise komplexe Phase
und die Basis bei gleichen Singularwerten vollständig festgelegt.

  GLEICHE LEISTUNG ALS STANDARD

Im Standardmodus MimoPowerAllocationMode::EQUAL_POWER gilt für alle gewählten
Moden

$$
  f_s[k]=\frac{1}{N_s},
  \qquad
  \mathbf W[k]=[\mathbf v_0[k],\ldots,\mathbf v_{N_s-1}[k]].
$$

Damit besitzt jede Spalte Norm eins und der Vertrag aus Abschnitt 5.9 bleibt
unverändert:

$$
  \Vert\mathbf W[k]\Vert_F^2=N_s.
$$

Die SVD-Moden sind orthogonal. Der unveränderte MMSE-Kern liefert deshalb
ohne Inter-Stream-Interferenz

$$

$$
  \gamma_s[k]=\frac{\rho}{N_s}\sigma_s^2[k].
$$

Der 1x1-Grenzfall verwendet den einzigen rechten Singulärvektor mit Einheitsnorm und reproduziert das bisherige skalare Ergebnis exakt. Die Optionen verwenden für die normierten Leistungsdiagnosen standardmäßig $\rho=1$ und $N=1$; die gleichmäßige Richtungsauswahl ist davon unabhängig.

  OPTIONALE RÄUMLICHE WASSERFÜLLUNG

Wasserfüllung wird nur durch die explizite Auswahl MimoPowerAllocationMode::WATER_FILLING aktiviert. Sie verteilt die Leistung ausschließlich über die N_s bereits gewählten räumlichen SVD-Moden eines Subcarriers. Zwischen Subcarriern wird keine Leistung verschoben; spektrale Maske und Gesamtleistung je Subcarrier bleiben daher unverändert. Mit Rauschleistung $N>0$, Referenz-SNR $\rho>0$, Gesamtleistung $P=\rho N$ und Modengewinn $g_s[k]=\sigma_s^2[k]$ lauten die Leistungsanteile

$$
  f_s[k]=\left(
    \nu[k]-\frac{1}{\rho g_s[k]}
  \right)^+,
  \qquad
  \sum_{s=0}^{N_s-1}f_s[k]=1.
$$

Der aktive Modensatz und der normierte Wasserstand $\nu[k]$ werden deterministisch aus den absteigend sortierten Moden bestimmt. Eine Mode am exakten Aktivierungsschwellwert bleibt inaktiv. Der Precoder skaliert danach

$$
  \mathbf w_s[k]=\sqrt{N_s f_s[k]}\,\mathbf v_s[k].
$$

Somit gilt auch bei inaktiven Nullspalten exakt

$$
  \sum_s\Vert\mathbf w_s[k]\Vert_2^2
  =N_s\sum_s f_s[k]=N_s.
$$

Die physikalisch ausgewiesene Modenleistung und der Wasserstand sind

$$
  p_s[k]=P f_s[k],
  \qquad
  \mu[k]=P\nu[k].
$$

Für jede aktive Mode erfüllen sie die KKT-Bedingung

$$
  p_s[k]+\frac{N}{g_s[k]}=\mu[k],
$$

während inaktive Moden keinen negativen Leistungsanteil erhalten. Eine abschließende Rundungskorrektur wird ausschließlich auf den letzten aktiven Anteil angewendet, sodass die Anteile wieder exakt zur normierten Gesamtleistung summieren. Alle Gewinne, Anteile und Wasserstände werden auf

Endlichkeit und Nichtnegativität geprüft.

MimoSvdPrecodingResult hält den festen Precoder sowie pro Subcarrier
numerischen Rang, gewählte Singularwerte, aktive Modenzahl, Leistungsanteile,
absolute Modenleistungen, deren Summe und den Wasserstand. Zusätzlich werden
Rauschleistung, Referenz-SNR, Gesamtleistung und Rangtoleranz unverändert
ausgewiesen. Ein idealer Aufruf mit Wasserfüllung lautet:

```
    MimoSvdPrecodingOptions options;
    options.powerAllocation = MimoPowerAllocationMode::WATER_FILLING;
    options.referenceSnrLinear = rho;
    options.noisePower = noisePower;

    const auto svd = MimoPrecodingMatrix::SingularValueDecomposition(
    channelAtCsiTime, numStreams, options);
    const auto result = SuMimoLinkModel::Calculate(
    channelAtCsiTime, svd.GetPrecodingMatrix(), options.referenceSnrLinear);
```

Wie MRT erzeugt auch SVD ausschließlich einen unveränderlichen perfekten-CSI-
W[k]-Schnappschuss. Unterphase 3.3 aktiviert weder SU_BEAMFORMING im Helper
noch automatische Aktualisierung, Feedback, CSI-Zeitbezug oder einen neuen
SpectrumWifiPhy-Pfad. Der vorhandene feste-W[k]-Vertrag ist die einzige
Integration; Unterphase 3.5 verwendet genau diesen Vertrag nun für die
benutzerseitige end-to-end Modusauswahl, ohne die SVD- oder
Leistungsallokationssemantik zu verändern.

5.12 QUANTISIERTE HE-/EHT-SU-RÜCKMELDUNG UND CSI-ALTERUNG DER UNTERPHASE 3.4

UNTERSTÜTZTER FEEDBACKUMFANG UND ABGRENZUNG

MimoCompressedBeamformingFeedback bildet die semantische komprimierte
Beamformingmatrix $\mathbf V[k]$ aus der HE-/EHT-SU-Rückmeldung ab. IEEE
802.11ax beschreibt diese Matrix als geordnete Folge von
ϕ-Phasenwinkeln und reellen ψ-Givens-Winkeln. EHT übernimmt
dieselbe komprimierte Darstellung mit erweiterten Grenzen für N_c und
N_r; dies ist auch in der öffentlichen
[IEEE-802.11be-Beamforming-Korrespondenz](https://www.ieee802.org/11/email/stds-802-11-tgbe/msg01987.html)
festgehalten. Die öffentlich gelisteten
[HE-Comment-Resolution-Dokumente](https://www.ieee802.org/11/Reports/tgax_update.htm)
verweisen auf die HE Compressed Beamforming Report- und
SU-MIMO-Beamforming-Klauseln. Die Implementierung unterstützt daraus bewusst
folgenden schmalen Vertrag:

- ausschließlich Single-User-Feedback für eine semi-unitäre
  Gleichleistungs-SVD-Matrix mit $1\leq N_s\leq N_\mathrm{tx}$;
- HE mit höchstens acht und EHT mit höchstens 16 TX-Antennen;
- das grobe SU-Codebook mit $b_\phi=4,\ b_\psi=2$ sowie das feine
  SU-Codebook mit $b_\phi=6,\ b_\psi=4$;
- eine Matrix je modelliertem Subcarrier und einen unquantisierten
  Forschungsgrenzfall.

Der Typ ist absichtlich kein serialisierter IEEE-802.11-Action-Frame.
MIMO-Control-, Average-SNR-/CQI-Felder, MAC-Header, Segmentierung,
Subcarriergruppierung N_g, Interpolation, RU-/Partial-Bandwidth-Auswahl,
MU-Exclusive-Feedback und die feineren MU-Codebooks sind nicht implementiert.
Wassergefülltes W[k] wird ebenfalls nicht als $\mathbf V[k]$ ausgegeben,
weil dessen Spalten nicht semi-unitär sind; die Richtungsrückmeldung und eine
separate Leistungsallokation werden nicht fälschlich vermischt. Damit wird
weder MU-MIMO noch die Laufzeitintegration aus Unterphase 3.5 vorweggenommen.

GIVENS-DARSTELLUNG UND VERBINDLICHE WINKELREIHENFOLGE

Sei $m=\min(N_s,N_\mathrm{tx}-1)$. Für jede Stufe
$i=0,\ldots,m-1$ enthält $\mathbf D_i$ auf den Diagonaleinträgen
$i,\ldots,N_\mathrm{tx}-2$ die Phasen
$\exp(j\phi_{r,i})$; der letzte Eintrag und die Einträge vor i sind
eins. $\mathbf G_{l,i}^{T}(\psi_{l,i})$ ist die reelle Rotation, deren
(i,l)-Block

$$
  \begin{bmatrix}
    \cos\psi_{l,i} & -\sin\psi_{l,i}\\
    \sin\psi_{l,i} & \cos\psi_{l,i}
  \end{bmatrix}
$$

lautet. Die Rekonstruktion verwendet exakt

$$
  \widehat{\mathbf V}[k]
  =
  \left[
  \prod_{i=0}^{m-1}
    \mathbf D_i
    \left(
    \mathbf G_{N_\mathrm{tx}-1,i}^{T}
    \cdots
    \mathbf G_{i+1,i}^{T}
    \right)
  \right]
  \mathbf I_{N_\mathrm{tx}\times N_s}.
$$

Die flache Reihenfolge je Subcarrier ist für jede Stufe zuerst

$$
  \phi_{i,i},\ldots,\phi_{N_\mathrm{tx}-2,i}
$$

und danach

$$
  \psi_{i+1,i},\ldots,\psi_{N_\mathrm{tx}-1,i}.
$$

Die getrennten ganzzahligen ϕ- und ψ-Arrays sind innerhalb
ihres Typs subcarrier-major. Die Zahl der Winkel jedes Typs lautet

$$
  A =
  \sum_{i=0}^{m-1}(N_\mathrm{tx}-i-1).
$$

Encode entfernt die für die Linkmetrik irrelevante Phase jeder SVD-Spalte
deterministisch, extrahiert danach die idealen Winkel stufenweise und
quantisiert nur diese Winkel. FromQuantizedAngles überspringt die Extraktion
und erzeugt aus bekannten Integerindizes eine bitgenaue Referenzdarstellung.
Reconstruct ist davon getrennt und verwendet ausschließlich die gespeicherten
Indizes. Die idealen SVD-, Feedback- und Rekonstruktionsobjekte bleiben damit
einzeln prüfbar.

  DETERMINISTISCHE QUANTISIERUNG UND GRENZWERTE

Für $\phi\in[0,2\pi)$ und 2^{b_ϕ} Zellen gelten

$$
  q_\phi =
  \left\lfloor
    \frac{2^{b_\phi}\phi}{2\pi}
  \right\rfloor,
  \qquad
  \widehat\phi =
  \frac{2\pi(q_\phi+\tfrac12)}{2^{b_\phi}}.
$$

Vor der Quantisierung wird ϕ modulo 2π normalisiert; der obere Rand 2π fällt damit eindeutig auf Index null. Für $\psi\in[0,\pi/2]$ gilt

$$
  q_\psi =
  \min\left(
    2^{b_\psi}-1,
    \left\lfloor
    \frac{2^{b_\psi+1}\psi}{\pi}
    \right\rfloor
  \right),
  \qquad
  \widehat\psi =
  \frac{\pi(q_\psi+\tfrac12)}{2^{b_\psi+1}}.
$$

Damit rekonstruieren auch die minimalen und maximalen Indizes stets die Zellmitte und nie einen mehrdeutigen periodischen Rand. Nicht passende Indexanzahlen und Werte außerhalb der gewählten Bitbreite werden vor der Rekonstruktion mit std::invalid_argument abgelehnt. Der unquantisierte Modus speichert das bereits validierte feste W[k] direkt, rekonstruiert es koeffizientenexakt und weist null Feedbackbits aus; null bedeutet hier „kein endlicher Drahtumfang modelliert“ und nicht ein kostenloses Standardfeedback.

Der reine Winkelumfang ohne PHY-/MAC-Header und ohne CQI ist

$$
  B_\mathrm{fb}
  =
  N_\mathrm{sc} A (b_\phi+b_\psi).
$$

Für eine 2x1-Matrix sind dies je Subcarrier sechs Bit im groben und zehn Bit im feinen SU-Codebook. Eine 3x2-Matrix besitzt je Typ drei Winkel und benötigt im feinen Profil 30 Bit je Subcarrier.

LEISTUNGSRENORMIERUNG UND DIAGNOSEGRÖSSEN

Obwohl das Produkt idealer Rotationen semi-unitär ist, wird jede quantisiert rekonstruierte Matrix vor dem bestehenden festen-W[k]-Konstruktor explizit erneut normalisiert. Mit

$$
  P_\mathrm{vor}[k]
  =
  \Vert\widehat{\mathbf V}[k]\Vert_F^2,
$$

```
  \qquad
  a[k]=\sqrt{\frac{N_s}{P_\mathrm{vor}[k]}},
  \qquad
  \mathbf W_q[k]=a[k]\widehat{\mathbf V}[k]
\]

gilt anschließend wieder

\[
  \Vert\mathbf W_q[k]\Vert_F^2=N_s.
\]

MimoFeedbackReconstruction weist \(P_\mathrm{vor}[k]\), \(a[k]\) und die
endgültige Leistung getrennt aus. Nullleistung, nicht endliche Werte oder ein
Ergebnis außerhalb des festen-W[k]-Vertrags werden abgelehnt.

MimoFeedbackDiagnostics meldet neben \(B_\mathrm{fb}\) zwei
phaseninvariante Fehler je Subcarrier. Für normierte ideale und
rekonstruierte Spalten sind dies der mittlere Spalten-Chordalfehler

\[
  e_\mathrm{ch}[k]
  =
  \sqrt{
    \frac1{N_s}\sum_s
    \left(
    1-
    \frac{
    |\mathbf w_s^H[k]\mathbf w_{q,s}[k]|^2
    }{
    \Vert\mathbf w_s[k]\Vert_2^2
    \Vert\mathbf w_{q,s}[k]\Vert_2^2
    }
    \right)
  }
\]

und der RMS-Koeffizientenfehler nach optimaler gemeinsamer Phasenausrichtung
jeder Spalte. Zusätzlich stehen die Mittelwerte über alle Subcarrier bereit.
Damit kann der Fehler ohne willkürliche SVD-Spaltenphase verglichen werden.
Auf den Referenzvektoren ist das feine Codebook nicht schlechter als das
grobe.

  EXPLIZITE CSI-ZEITACHSE

MimoCsiAgingModel::Select erhält ausschließlich die zur Auswahl verwendete
Schätzung

\[
  \widehat{\mathbf H}[k,t_\mathrm{csi}]
\]

sowie \(t_\mathrm{csi}\), \(t_\mathrm{tx}\) und eine
MimoCsiSelectionConfiguration. Mit Aktualisierungsintervall \(T_u\geq0\) und
Feedbackverzögerung \(d_\mathrm{fb}\geq0\) ist der neueste zum Sendezeitpunkt
verfügbare Schätzzeitpunkt verbindlich

\[
  t_\mathrm{csi}
  =
  \begin{cases}
```

```
    t_\mathrm{tx}-d_\mathrm{fb}, & T_u=0,\\
    T_u\left\lfloor
    \dfrac{t_\mathrm{tx}-d_\mathrm{fb}}{T_u}
    \right\rfloor, & T_u>0.
  \end{cases}
\]

Ein Sendezeitpunkt vor \(d_\mathrm{fb}\) besitzt noch kein verfügbares
Feedback. Ein explizit übergebener Zeitstempel, der nicht der Gleichung
entspricht, wird abgelehnt. Das ausgewiesene CSI-Alter ist

\[
  \Delta t_\mathrm{CSI}=t_\mathrm{tx}-t_\mathrm{csi}.
\]

\(T_u=0\) und \(d_\mathrm{fb}=0\) bilden den perfekten Nullalter-Grenzfall.
Select berechnet aus \(\widehat{\mathbf H}[k,t_\mathrm{csi}]\) zuerst das
deterministische Gleichleistungs-SVD-W[k], danach Feedback und Rekonstruktion.
CalculateLink verlangt dagegen die davon getrennt übergebene aktuelle CFR

\[
  \mathbf H[k,t_\mathrm{tx}]
\]

und berechnet ausschließlich
\(\mathbf H[k,t_\mathrm{tx}]\mathbf W_q[k]\). Der CSI-Schnappschuss wird in
diesem Schritt nicht gespeichert oder wiederverwendet. Derselbe feste
Precoder kann an HeSuMimoLinkAbstraction übergeben werden; dadurch verwenden
auch EESM und PER explizit die aktuelle CFR. Eine kontrollierte orthogonale
Subraumrotation liefert im Test mit altem W null SINR und eine entsprechend
schlechtere PER, während ein statischer Kanal trotz positiven Alters
unverändert bleibt.

MimoCsiStateIdentity ist der vollständige wertbasierte Schlüssel des
Auswahlzustands. Er enthält gerichtete TX- und RX-Node-IDs, beide
Arraygeometrien einschließlich Abstand, Muster, Polarisation und Orientierung,
den Precodingmodus, HE-/EHT-Feedbackprofil und \(t_\mathrm{csi}\).
Vertauschte Richtung, eine geänderte Arraykonfiguration, ein anderes Profil
oder ein anderer CSI-Zeitstempel sind damit verschiedene Zustände. Diese
Identität ist von der bestehenden rohen Sionna-PropagationCache-Ablage
getrennt und verhindert, dass ein späterer Laufzeitcache versehentlich
Precoderzustände verschiedener Richtungen oder Konfigurationen teilt.

Der ideale Aufruf ohne Quantisierung lautet:

```
    MimoCsiSelectionConfiguration configuration;
    configuration.numStreams = numStreams;
    configuration.feedback.codebook =
    MimoFeedbackCodebook::UNQUANTIZED;
    configuration.timing.updateInterval = MilliSeconds(5);
    configuration.timing.feedbackDelay = MilliSeconds(2);

    const auto selection = MimoCsiAgingModel::Select(
    channelEstimateAtCsiTime,
    csiTimestamp,
    transmissionTimestamp,
    configuration);
    const auto currentLink = selection.CalculateLink(
    channelAtTransmissionTime,
    referenceSnrLinear);
```
```

Der Vertrag bleibt bewusst keine automatische Simulationstaktung. Unterphase 3.5 bindet ihn nun durch einen expliziten SelectPrecoding-Aufruf an SionnaMimoHelper und SpectrumWifiPhy: Der Anwender übergibt CSI- und Sendezeitpunkt, der Helper hält das unveränderliche Ergebnis und Apply nutzt anschließend den vorhandenen festen-W[k]-Pfad. Periodische Ereignisse oder MAC-Feedbackframes werden dadurch nicht vorgetäuscht.

5.13 FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL DER UNTERPHASE 4.2

`DlMuMimoPrecodingMatrix` bettet den unveränderten, leistungsvalidierten `MimoPrecodingMatrix`-Tensor ein. Der Index eines Eintrags im Zuordnungsvektor ist die globale Precoder-Spalte c; der Eintrag

$$
  a_c=(u,s_u)
$$

nennt die stabile Nutzer-ID u und den nutzerlokalen Stream s_u. Die lokalen Streamindizes jedes Nutzers beginnen bei null und sind lückenlos. Jeder Nutzer des Phase-4.1-Snapshots erhält mindestens einen Stream, und kein Stream darf auf einen nicht vorhandenen Nutzer zeigen.

Die Phase-3-Normierung gilt jetzt über alle gleichzeitig gesendeten Spalten:

$$
  \Vert W[k]\Vert_F^2=N_{s,\mathrm{ges}}.
$$

Mit der expliziten Spaltenleistung

$$
  q_c[k]=\sum_t |W_{t,c}[k]|^2,\qquad
  \sum_c q_c[k]=N_{s,\mathrm{ges}}
$$

beträgt der Anteil der gesamten AP-Leistung für Spalte c genau $q_c[k]/N_{s,\mathrm{ges}}$. Gleiche Spaltennormen ergeben gleiche Leistung; abweichende Nutzer- oder Streamgewichte stehen sichtbar in $W[k]$, ohne die Gesamtleistung zu ändern.

Für Nutzer u wird zunächst der vollständige effektive Kanal gebildet:

$$
  G_u[k]=H_u[k]W[k]
    =\left[g_{u,0}[k],\ldots,g_{u,N_{s,\mathrm{ges}}-1}[k]\right].
$$

Die nutzerspezifische Referenz-SNR ρ_u enthält den getrennten Wideband-Pfadverlust und die Empfängerrauschleistung. Für jede globale Spalte gilt derselbe Symbolleistungsfaktor

$$
  \alpha_u=\frac{\rho_u}{N_{s,\mathrm{ges}}}.
$$

Der lineare Empfänger verwendet die vollständige Kovarianz aller gewünschten und fremden Spalten,

$$
  R_u[k]=I+\alpha_u G_u[k]G_u^H[k],
$$

\qquad
f_{u,c}[k]=R_u^{-1}[k]g_{u,c}[k].
\]

Die Skalierung von $f_{u,c}$ ist für die SINR ohne Bedeutung. Der ausgegebene gewünschte Anteil und die Inter-User-Interferenz nach Kombination lauten

\[
D_{u,c}[k]=\alpha_u|f_{u,c}^H[k]g_{u,c}[k]|^2,
\]

\[
I_{u,c}^{\mathrm{MU}}[k]
=\alpha_u\sum_{d:a_d=(v,\cdot),\,v\ne u}
|f_{u,c}^H[k]g_{u,d}[k]|^2.
\]

Mit dem analogen Intra-User-Anteil über die anderen Streams desselben Nutzers und $N_{u,c}[k]=\|f_{u,c}[k]\|_2^2$ folgt

\[
\gamma_{u,c}[k]
=\frac{D_{u,c}[k]}
{N_{u,c}[k]+I_{u,c}^{\mathrm{intra}}[k]
+I_{u,c}^{\mathrm{MU}}[k]}.
\]

Für genau einen Nutzer ist dies algebraisch derselbe lineare MMSE-Vertrag wie in Phase 3. Das gilt auch für den 1x1-Grenzfall und für null SNR beziehungsweise einen verschwindenden effektiven Kanal.

`HeDlMuMimoLinkAbstraction` komprimiert anschließend ausschließlich die SINR-Werte des jeweiligen Nutzers:

\[
\gamma_{\mathrm{eff},u}
=-\beta_{m_u}\ln\left(
\frac{1}{N_{\mathrm{sc}}N_{s,u}}
\sum_{k,s_u}\exp\left(-\frac{\gamma_{u,s_u}[k]}{\beta_{m_u}}\right)
\right).
\]

MCS m_u, EESM-β_{m_u}, `WifiTxVector` und Paketgröße gehören explizit zum Nutzer. Die anschließende ErrorRateModel-Abfrage ist daher getrennt und kann für zwei Nutzer unterschiedliche PER liefern.

Vor der Berechnung werden Snapshotrichtung und -ordnung, CSI-Zeitidentität, Frequenzraster, CFR-/Precoderdimensionen, endliche Koeffizienten, die vollständige Spaltenzuordnung, nutzerweise Stream-/RX-Grenzen sowie genau eine SNR- und HE-Eingabe je Nutzer geprüft. Konstruktion und Berechnung sind wertbasiert; eine Ablehnung verändert weder Snapshot noch Precoder oder EESM-Konfiguration. Der Kern konsumiert den Phase-4.1-Snapshot nur lesend und ändert weder Protocol Buffers, Batching noch Cache.

5.14 IDEALES DOWNLINK-ZF UND REGULARISIERTES ZF DER UNTERPHASE 4.3

Die wertbasierten Fabriken `DlMuMimoPrecodingMatrix::ZeroForcing` und `RegularizedZeroForcing` konsumieren denselben vollständig validierten Phase-4.1-Snapshot wie der feste Phase-4.2-Kern. Der absichtlich schmale Vertrag umfasst gleichzeitige perfekte Downlink-CSI, genau eine Empfangsantenne und genau den lokalen Stream null je Nutzer. Für U Nutzer

und N_{tx} AP-Antennen gilt $1 \le U \le N_{\mathrm{tx}}$.
Mehrere RX-Antennen oder Streams je Nutzer benötigen eine explizite
Empfängerauswahl, Blockdiagonalisierung oder einen gemeinsamen
Transceiververtrag und werden in dieser Unterphase nicht stillschweigend
angenommen.

Die Nutzer des Snapshots liegen bereits streng aufsteigend nach stabiler
Nutzer-ID vor. Die als Menge behandelten Streamanforderungen dürfen in
beliebiger Reihenfolge eingegeben werden, müssen aber exakt $(u, 0)$ einmal
für jeden Snapshotnutzer enthalten. Die ausgegebenen Spalten und
`DlMuMimoStreamMapping`-Einträge folgen immer der stabilen Snapshotreihenfolge.
Pro Subcarrier entsteht damit eindeutig

$$
  \bar H[k]=\begin{bmatrix}H_{u_0}[k]\\ \vdots\\ H_{u_{U-1}}[k]\end{bmatrix}
  \in\mathbb C^{U\times N_\mathrm{tx}}.
$$

ns-3 berechnet eine dünne Jacobi-SVD

$$
  \bar H[k]=U_H[k]\Sigma[k]V_H^H[k],
  \qquad \sigma_0[k]\geq\ldots\geq\sigma_{U-1}[k]\geq0.
$$

Wie in Phase 3 ist die dokumentierte Standardrangschwelle
$\tau_{\mathrm{rank}} = 10^{-12}$; ein expliziter Wert muss endlich und strikt
in $(0, 1)$ liegen. Der numerische Rang zählt genau die Werte
$\sigma_i[k] > \tau_{\mathrm{rank}}\sigma_0[k]$. Auf jedem Subcarrier muss er
U betragen. Es gibt weder abgeschnittene schwache Nutzermoden noch eine
stille Nutzerreduktion oder versteckte Regularisierung.

Für ZF wird auf dem vollständig akzeptierten Zeilenraum die ungekürzte
Moore-Penrose-Pseudoinverse verwendet:

$$
  \widetilde W_\mathrm{ZF}[k]
    =V_H[k]\,\mathrm{diag}(\sigma_i^{-1}[k])\,U_H^H[k].
$$

RZF besitzt keinen Defaultparameter. Der Aufrufer muss ein endliches, strikt
positives dimensionsloses α angeben; für die gemeinsam normierte CFR
gilt

$$
  \widetilde W_\mathrm{RZF}[k]
    =V_H[k]\,\mathrm{diag}\!\left(
    \frac{\sigma_i[k]}{\sigma_i^2[k]+\alpha}\right)U_H^H[k]
    =\bar H^H[k]\left(\bar H[k]\bar H^H[k]+\alpha I\right)^{-1}.
$$

Singulärwerte und zugehörige linke/rechte Vektoren bleiben als gekoppelte,
absteigend gelieferte Tripel zusammen. Die vollständige Pseudoinversen-
beziehungsweise Filtermatrix ist bei exakt gleichen Singularwerten invariant
gegen Basisrotationen und gemeinsame Spaltenphasen; nahezu gleiche, aber
verschiedene Werte behalten ihren eigenen Filterfaktor. Da keine einzelnen
Moden als Nutzerstrahlen exportiert werden, gelangt keine beliebige
SVD-Basiswahl in die Nutzerordnung. Wiederholte Auswahl und umgekehrte
Anforderungsreihenfolge liefern im nativen Test koeffizientengleiche
Ergebnisse.

Erst nach vollständiger Auswahl wird pro Subcarrier ein einziger reeller Skalierungsfaktor angewendet:

$$
  W[k]=\sqrt{\frac{U}{\Vert\widetilde W[k]\Vert_F^2}}\,
    \widetilde W[k],
  \qquad \Vert W[k]\Vert_F^2=U.
$$

Damit bleibt P exakt die gesamte AP-Sendeleistung aus Unterphase 4.2. Die globale Skalierung erhält die ZF-Nullstellen. RZF bleibt auf vollrangigen, schlecht konditionierten Kanälen endlich; auch RZF akzeptiert absichtlich keinen numerischen Rangverlust, weil Regularisierung in Unterphase 4.3 keine nicht planbare Nutzergruppe kaschiert.

Vor SVD oder Ergebniskonstruktion werden Richtung, Zeit, Nutzerordnung, Frequenzraster, Dimensionen, Endlichkeit, Last, Streamanforderungen, RX-Antennenzahl, Rangtoleranz und gegebenenfalls α geprüft. Jede Ablehnung ist `std::invalid_argument` und verändert keinen Zustand. Das fertige Objekt läuft unverändert durch `DlMuMimoLinkModel` sowie die bestehenden nutzerweisen MMSE-, EESM- und PER-Pfade. Protocol Buffers, Serialisierung, Batching, Cache, ZMQ, Sionna RT und SpectrumWifiPhy werden durch Unterphase 4.3 nicht geändert.

5.15 HE-DOWNLINK-MU-SPECTRUMINTEGRATION DER UNTERPHASE 4.4

`SetDlMuMimoState` übernimmt einen wertbasierten Downlink-Snapshot und dessen gemeinsamen Precoder. Vor der atomaren Installation werden numerische Dimensionen, Zeit-/Nutzerordnung und Precoder geprüft. Zusätzlich müssen ein gemeinsamer Sender, eindeutige Empfänger ohne Selbstverbindung und endliche Widebandverluste vorliegen. Diese Installationsprüfung ersetzt nicht die Array-/Normalisierungsprüfung des Protobuf-Deserialisierers; die direkte numerische API bleibt für synthetische Referenzkanäle nutzbar.

Die installierte Gruppe gilt nur für ihre gerichteten AP-/Empfängerlinks und tatsächlich adressierte PSDUs. Die MAC-Zieladresse wird eindeutig auf die STA-ID abgebildet; RU, MCS und NSS stammen aus `WifiTxVector::HeMuUserInfoMap`. Mithörer und andere Sender verwenden ihren physikalischen Cache-/Festkanalpfad und ändern den installierten MU-Zustand nicht. Auch ein Teilnehmer einer zuvor installierten Gruppe darf ein aktuell nicht an ihn adressiertes PPDU mithören. Für diese Verbindungen muss deshalb ein physikalischer Kanal konfiguriert sein.

Belegte Frequenzen. Die Laufzeit-CFR verwendet das Sender-PSD-Gitter mit 78,125 kHz Abstand, abzüglich des abschließenden zusätzlichen Guardbins. `HeRu::GetSubcarrierGroup` liefert die einzelnen belegten Teilspannen; `HePhy::ConvertHeRuSubcarriers` bildet sie auf dieses Sendergitter ab. RX-Bandindizes dürfen bei unterschiedlicher Betriebsbandbreite nicht zur Indizierung des Sendespektrums verwendet werden.
Datentöne und Piloten gehen gleich gewichtet in die Abstraktion ein; DC, Nulltöne, Guardtöne und andere RUs werden ausgeschlossen. Dies modelliert keine gesonderten Pilotschätzfehler. Für die belegte Tonmenge K_u gilt

$$
 \gamma_{\mathrm{eff},u}=-\beta_{m_u}\log\left(
 \frac{1}{|K_u|N_{s,u}}\sum_{k\in K_u}\sum_s
 e^{-\gamma_{u,s}[k]/\beta_{m_u}}\right).
$$

Auch eine Vollband-RU benötigt eine explizite Tonauswahl. Die optionale Liste `subcarrierIndices` darf nur in der direkten numerischen API leer bleiben und bezeichnet dort wie bisher das gesamte übergebene Gitter. Im Laufzeitpfad ist

sie stets nicht leer. Doppelte oder außerhalb der CFR liegende Indizes werden abgelehnt. Matrixdiagnostik bleibt auf dem vollständigen Gitter verfügbar; nur die ausgewählten Töne gehen in die EESM-/PER-Reduktion ein.

Leistung je RU. Gleiche `HeRu::RuSpec`-Werte bilden eine Gruppe g. Auf jedem Ton wird die Energie aller in dieser Gruppe aktiven Spalten berechnet:

$$
E_g[k]=\sum_{j\in g}\sum_t |W_{t,j}[k]|^2,\qquad
\rho_u=\frac{P_{\mathrm{RU}(u)}L_uG_{\mathrm{rx},u}}{N_u},\qquad
\rho_{u,s}[k]=\frac{\rho_u}{E_{g(u)}[k]}.
$$

Dieser letzte Faktor wird konsistent in gewünschter Leistung, MMSE-Kovarianz und Interferenz verwendet. Damit gilt $\sum_{j\in g} P_g\|w_j[k]\|^2/E_g[k]=P_g$, auch für ungleiche Spaltennormen und ungleiche RU-Leistungen. Relative Gewichte innerhalb einer RU bleiben erhalten. Beim bisherigen Ein-Gruppen-Vertrag gilt $E_g[k]=N_{s,\mathrm{ges}}$, also die bisherige Skalierung. Gruppenenergie muss auf jedem numerisch ausgewerteten Ton endlich und positiv sein; vollständig abgeschaltete Gruppenspalten werden explizit abgelehnt. Andere RUs gehen nicht in die MMSE-Kovarianz oder IUI ein.

Der skalare SpectrumChannel-Pfadverlust liegt beim Eintritt bereits in der PSD. RU-Leistung wird über die vom PHY verwendete umschließende RU-Bandspanne auf dem Sendergitter integriert. Der Adapter rekonstruiert die AP-RU-Leistung und wendet pro Snapshotnutzer dessen Widebandverlust und Empfangsgewinn an. Rauschen verwendet die nominelle ns-3-RU-Bandbreite, 290 K und die konsistente PHY-Rauschzahl. Die skalare Nutzdatenleistung ist

$$
P_{\mathrm{eq},u}=\gamma_{\mathrm{eff},u}N_u/G_{\mathrm{AWGN},u}.
$$

Nur die RU-Bandspanne des aktuellen Nutzers wird auf diese Leistung skaliert; der nachgelagerte PHY-Empfangsgewinn wird vorher herausgerechnet. Außerhalb der RU wird auf anderen belegten RUs die tatsächliche Energie $\sum_{j\in g}|H[k,0,:]w_j[k]|^2/E_g[k]$ verwendet. Unbelegte Spektralmaskenbins verwenden die Gleichleistungs-Antennenannahme. Eine Nutzdaten-SINR darf nicht alle fremden Interferenzbänder skalieren.

Gemeinsame Präambel und Mithörer. Nur die adressierte `HePpdu::PSD_HE_PORTION` verwendet die MU-Nutzdatenabstraktion. Die gemeinsame `PSD_NON_HE_PORTION` enthält keine IUI- oder MCS-Beta-Kompression. Als explizites physikalisches Modell gelten gleiche Leistung je TX-Antenne und die erste RX-Verzweigung:

$$
G_{\mathrm{common}}[k]=\frac{1}{N_{\mathrm{tx}}}\sum_t|H[k,0,t]|^2.
$$

Die vorhandene PHY-Verarbeitung behandelt anschließend Empfangsgewinn und skalare Diversity. ns-3.40 sendet die Nicht-HE-Portion mit 312,5 kHz Abstand; die installierte HE-CFR besitzt 78,125 kHz Abstand. Bei gleichem Zentrum und Bandumfang wird jeder vierte CFR-Eintrag am entsprechenden Frequenzzentrum verwendet. Gleich aufgelöste Cache-CFR bleiben ebenfalls zulässig.
Der physikalische HE-MU-Pfad für Mithörer verwendet dieselbe Gleichleistungsannahme. Dies modelliert kein separates Präambel-Beamforming und keine räumliche Kovarianz fremder MU-Interferenz. Gemeinsame Portionen und Mithörer erhalten kein MU-Payload-CFR-Tag. Die HE-Portion umfasst im ns-3-Spektrumpfad auch HE-Training; eine weitere feldweise Trennung wird hier

nicht eingeführt.

Inkonsistente Metadaten adressierter installierter Teilnehmer werden weiterhin abgelehnt. HE-SU und die bisherigen numerischen 1x1-/Precoderverträge bleiben kompatibel. EHT-MU und EHT-TB werden weiterhin abgelehnt. Die Gruppenrechnung wird noch pro Empfänger ausgeführt; diese quadratische Adapterarbeit bleibt eine bekannte Leistungsgrenze. Automatischer Scheduler, Helperfreigabe für MU, Uplink und wissenschaftliche EESM-Kalibrierung sind nicht Teil dieser Korrektur.

5.16 LAUFZEIT-CSI UND SCHEDULER-SCHNITTSTELLE DER UNTERPHASE 4.5

DlMuMimoRuntime bildet eine schmale, wertbasierte Brücke zwischen dem vorhandenen ns-3-MultiUserScheduler und dem Phase-4.4-Spectrumadapter. Ein Auswahlaufruf übernimmt dessen DlMuInfo einschließlich WifiTxParameters::m_txVector, HeMuUserInfoMap und WifiPsduMap. Eine explizite Menge von DlMuMimoStaUserBinding trennt weiterhin STA-ID und stabile Kanalnutzer-ID. PSDU-, HE-MU-TxVector- und Binding-Menge müssen identisch sein; jede PSDU muss existieren.

Der Laufzeitschlüssel lautet

$$
K=\left(n_{\mathrm{AP}},d,
  \{(u,\mathrm{STA},\mathrm{RU},N_{\mathrm{SS}})\}\right).
$$

Die Nutzerfolge in K ist unabhängig von der Eingabereihenfolge streng nach stabiler Nutzer-ID sortiert. Unterphase 4.5 akzeptiert nur WIFI_PREAMBLE_HE_MU, Downlink, einen Stream und eine Empfangsantenne je Nutzer sowie mindestens zwei Nutzer pro Gruppe. Die ausgewählten Nutzer müssen dieselbe RU belegen. Der Scheduler erhält diese geordnete Auswahl über DlMuMimoRuntimeSelection::GetUsers; nur ein anschließend ausgegebenes DlMuInfo mit exakt denselben STA-IDs, RUs und NSS kann den Zustand durch Lookup oder Install treffen. So bleiben interne WifiTxParameters-Informationen im Scheduler und werden nicht nachträglich teilweise gefiltert.

Die feste Gruppierung übernimmt eine explizite Nutzermenge, ignoriert deren Eingabereihenfolge und verwendet danach die unveränderte ZF- oder explizite RZF-Fabrik aus Unterphase 4.3. Die alternativ explizit wählbare MINIMUM_CORRELATION-Gruppierung untersucht alle Teilmengen der angeforderten Größe in lexikographischer Reihenfolge. Für eine Gruppe G ist die ausgegebene Metrik

$$
\mu(G)=\max_{u\ne v\in G}
\frac{\left|\sum_{k,t}H_u[k,0,t]H_v^*[k,0,t]\right|}
  {\sqrt{\sum_{k,t}|H_u[k,0,t]|^2}
   \sqrt{\sum_{k,t}|H_v[k,0,t]|^2}}.
$$

Nur Nutzer mit identischer RU kommen gemeinsam in Frage. Überlastete, rangdefiziente oder anderweitig von der Phase-4.3-Fabrik abgelehnte Gruppen werden verworfen. Gewählt wird die verbleibende Gruppe mit kleinstem μ; innerhalb 10^{-15} gleicher Metriken entscheidet die lexikographische Nutzer-ID-Folge. Gibt es keine belegbare Gruppe, wird kein Zustand erzeugt. Der fertige ZF-/RZF-Precoder behält $\lVert W[k]\rVert_F^2=|G|$ und damit die gesamte AP-Leistung.

Für den tatsächlichen Sendezeitpunkt t_{tx} wird unverändert der Phase-3-Vertrag MimoCsiTimingConfiguration::ResolveCsiTimestamp benutzt:

\[
 t_{\mathrm{csi}}=
 \begin{cases}
   t_{\mathrm{tx}}-t_{\mathrm{fb}}, & T_{\mathrm{upd}}=0,\\
   T_{\mathrm{upd}}\left\lfloor
    \dfrac{t_{\mathrm{tx}}-t_{\mathrm{fb}}}{T_{\mathrm{upd}}}
   \right\rfloor, & T_{\mathrm{upd}}>0.
 \end{cases}
\]

Snapshot und alle Nutzer müssen exakt diesen gerichteten CSI-Zeitpunkt
tragen. Auswahl und Diagnose speichern \(t_{\mathrm{csi}}\),
\(t_{\mathrm{tx}}\), CSI-Alter, Aktualisierungsintervall und Rückmeldedauer.
Ein Lookup für einen anderen Sendezeitpunkt, eine andere Gruppe, AP, RU, NSS
oder Richtung schlägt fehl. Pro Schlüssel bleibt der neueste Zustand
erhalten; ein älterer Ersatz sowie verschiedene CFRs unter derselben
CSI-Identität werden atomar abgelehnt.

DlMuMimoFeedbackRepresentation unterscheidet vollständige perfekte
MU-Kanalschätzung, quantisierte MU-Kanalschätzung und einen rekonstruierten
SU-Precoder. In dieser Unterphase ist ausschließlich PERFECT_CHANNEL
unterstützt. Die beiden anderen Varianten werden eindeutig abgelehnt;
insbesondere wird ein SU-Precoder nicht als vollständige Mehrnutzer-CFR
interpretiert. Install prüft zuerst den vollständigen Laufzeitschlüssel und
Zeitpunkt und übergibt erst danach genau den gespeicherten Snapshot und
Precoder an SionnaSpectrumPropagationLossModel::SetDlMuMimoState.

Alle Fehler der neuen wertbasierten API sind std::invalid_argument mit dem
Präfix DlMuMimoRuntime. Konstruktion, Auswahl und Lookup prüfen Konfiguration,
Richtung, Zeit, Endpunkte, Nutzer-, STA-ID-, PSDU-, RU- und NSS-Mengen,
Frequenz-/Antennendimensionen, Endlichkeit, Last und Rang vor einer
Zustandsänderung. Protocol Buffers, Serialisierung, Batching, Rohkanalcache,
ZMQ, Server, Sionna RT, SU-Helper, EESM/PER und der Spectrum-Rechenpfad
bleiben unverändert. Unterphase 4.6 ergänzt den konkreten End-to-End-Scheduler und die
Helperfreigabe. Automatische Sounding-/Feedbackereignisse, EHT und Uplink
bleiben ausdrücklich offen.

5.17 GEMEINSAME UPLINK-ZF-/MMSE-DETEKTION DER UNTERPHASE 4.7

Der gerichtete Uplink-Snapshot enthält je Nutzer genau einen Sendekanal

\[
  h_u[k]\in\mathbb C^{N_{\mathrm{rx,AP}}\times1}.
\]

Alle Nutzer enden am selben AP, besitzen dasselbe Frequenzraster und denselben
CSI-Zeitstempel. Unterphase 4.7 nimmt ausdrücklich ideale Zeit- und
Frequenzsynchronisation, eine Sendeantenne und den lokalen Stream null je
Nutzer an. Mehrere Nutzerstreams, Synchronisationsfehler sowie die Bindung an
HE-TB-Trigger und tatsächliche gleichzeitige PHY-Empfänge bleiben Unterphase
4.8.

Die nutzerspezifische Referenz-SNR

\[
  \rho_u=\frac{P_{\mathrm{tx},u}10^{-L_u/10}}{N_{\mathrm{AP}}}
\]

enthält Sendeleistung, Wideband-Pfadverlust und AP-Rauschleistung genau
einmal. `UlMuMimoUserSnr` übergibt diese Werte getrennt; es gibt keine

Gruppenmittelung oder gemeinsame Leistungsnormierung. Die HE-Abstraktion trägt dafür je Nutzer einen eigenen `WifiLinkBudgetResult`. Der im Snapshot separat sichtbare Pfadverlust wird im Numerikkern nicht erneut angewendet. Mit

$$
  H_{\mathrm{UL}}[k]=[h_0[k],\ldots,h_{U-1}[k]]
$$

lautet das rausch-normalisierte Signalmodell

$$
  y[k]=\sum_u\sqrt{\rho_u}\,h_u[k]x_u[k]+n[k],
  \qquad E[nn^H]=I.
$$

Der ZF-Detektor verlangt $U\leq N_{\mathrm{rx,AP}}$ und vollen numerischen Rang auf jedem Subcarrier. Mit derselben relativen Standardrangschwelle 10^{-12} wie die bisherigen SVD-Kerne werden seine Kombinierer als Spalten von

$$
  F_{\mathrm{ZF}}[k]
    =H_{\mathrm{UL}}[k]
    \bigl(H_{\mathrm{UL}}^H[k]H_{\mathrm{UL}}[k]\bigr)^{-1}
$$

mittels dünner Jacobi-SVD berechnet. Überlast oder Rangverlust werden vor einem Ergebnis als `std::invalid_argument` abgelehnt. Das lineare MMSE bleibt auch für rangdefiziente Gruppen definiert:

$$
  R[k]=I+\sum_v\rho_v h_v[k]h_v^H[k],
  \qquad f_u[k]=R^{-1}[k]h_u[k].
$$

Für beide Detektoren werden die nicht beliebig normierten Kombinierer vollständig ausgewertet:

$$
  D_u[k]=\rho_u|f_u^H[k]h_u[k]|^2,
$$

$$
  I_u[k]=\sum_{v\ne u}\rho_v|f_u^H[k]h_v[k]|^2,
  \qquad N_u[k]=\lVert f_u[k]\rVert_2^2,
$$

$$
  \gamma_u[k]=\frac{D_u[k]}{I_u[k]+N_u[k]}.
$$

Damit bleiben insbesondere Nah-Fern-Leistungen dem verursachenden Nutzer zugeordnet. Für einen Nutzer reduzieren ZF und MMSE auf $\gamma=\rho\lVert h\rVert_2^2$ und reproduzieren den vorhandenen SU-SIMO-/MRC-Grenzfall.

`HeUlMuMimoLinkAbstraction` komprimiert ausschließlich die SINRs des jeweiligen Nutzers mit dessen explizitem MCS-Beta und fragt anschließend das vorhandene `ErrorRateModel` mit dessen `WifiTxVector` und Paketgröße ab. STA-ID, MCS, Paketlänge, Link Budget, effektive SINR und PER bleiben damit

nutzerweise getrennt. Diese wertbasierte API installiert keinen PHY-Zustand.

Vor jeder Rechnung werden Uplinkrichtung, stabile Nutzerordnung, gemeinsamer AP und Empfangsarray, Zeitstempel, Frequenzraster, CFR-Endlichkeit, Ein-Stream-Grenze, vollständige Zuordnungen, getrennte endliche SNRs und gegebenenfalls ZF-Rang geprüft. Snapshot und Eingaben werden nur gelesen.

5.18 HE-TB-/TRIGGERINTEGRATION DER UNTERPHASE 4.8

`HeUlMuMimoPhyAdapter` bildet die schmale Grenze zwischen ns-3-Wi-Fi-PHY-Objekten und dem wertbasierten Uplinkkern. Der Aufrufer übergibt den dekodierten `CtrlTriggerHeader`, dessen PPDU-UID, stabile Bindungen $(u,\mathrm{STA-ID},n_\mathrm{tx})$, den gerichteten Uplink-Snapshot sowie die am AP beobachteten `HePpdu`-Empfangsintervalle. Zulässig sind ausschließlich HE-Basic-Trigger und davon ausgelöste `WIFI_PREAMBLE_HE_TB`-PPDUs mit derselben UID. RU, MCS und NSS stammen ausschließlich aus den User-Info-Feldern des Triggers; das entspricht der ns-3-HE-TB-Semantik, bei der der AP diese nicht selbsttragenden PHY-Angaben aus seinem TRIGVECTOR ergänzt.

Für das explizite ideale Empfangsintervall $[t_0,t_1)$ und eine RU r lautet die tatsächlich gemeinsam detektierte Nutzermenge

$$
\mathcal A_r =
\left\{u\;\middle|\;
  \mathrm{uid}_u=\mathrm{uid}_\mathrm{TF},
  \mathrm{RU}_u=r,
  t_{\mathrm{start},u}=t_0,
  t_{\mathrm{end},u}=t_1
\right\}.
$$

Der Adapter konstruiert für jede nichtleere Menge $\mathcal A_r$ einen unveränderten, nach Nutzer-ID geordneten Teilsnapshot und ruft genau einmal `HeUlMuMimoLinkAbstraction` auf. Nutzer verschiedener RUs gehen nie in dieselbe Interferenzkovarianz ein. Ein im Trigger angekündigter Nutzer ohne PPDU erscheint in `GetMissingStaIds()`; ein gültiges HE-TB-PPDU mit abweichendem Start oder Ende erscheint in `GetNonSimultaneousStaIds()`. Beide Kategorien bleiben aus jedem Detektionszustand ausgeschlossen. Unbekannte oder doppelte STA-IDs, falsche Sender-/AP-Knoten, andere Trigger-UIDs, falsche Präambeln, ungültige Leistung oder Dauer, Antennendimensionsfehler und fehlende MCS-Betas brechen atomar als `std::invalid_argument` ab.

Je tatsächlich teilnehmendem Nutzer berechnet der Adapter das Link Budget mit der Bandbreite seiner RU und reicht Paketgröße, belegte CFR-Töne und Trigger-TxVector an die bestehende nutzerweise EESM-/PER-Abbildung weiter. Für eine nachfolgende skalare SpectrumWifiPhy-Einkopplung steht analog zum Downlink die äquivalente Empfangsleistung

$$
P_{\mathrm{eq},u} =
  \frac{\gamma_{\mathrm{eff},u}N_{\mathrm{AP},u}}
  {G_{\mathrm{div},u}}
$$

bereit. $N_{\mathrm{AP},u}$ enthält thermisches Rauschen, RU-Bandbreite und Rauschzahl genau einmal; $G_{\mathrm{div},u}=N_{\mathrm{rx,AP}}$ korrigiert nur den zusätzlichen AWGN-Diversityfaktor des ns-3-ErrorRateModel. Die

matrixwertige gewünschte Leistung, Inter-User-Interferenz, Rauschverstärkung,
SINR und PER bleiben zusätzlich vollständig im Gruppenergebnis erhalten.

5.19 PHASE-5-MESSVERTRAG

Kalibrierung. `tools/calibrate_eesm.py` verwendet die vom nativen
`ns3_wifi_mimo_awgn_curve` exportierte `TableBasedErrorRateModel`-Kurve und
unabhängige Paketfehlerzählungen mit linearen Post-Processing-SINRs.
`tools/generate_he_link_level_reference.m` erzeugt die MATLAB-R2026a-Referenz
mit idealem Timing und perfekter RX-CSI. Zuerst wird der AWGN-Offset
δ_c, danach bei festem Offset das EESM-Beta $\beta_m>0$ durch
binomiale Maximum-Likelihood auf getrennten Trainingsdaten gefittet.
Validierung verwendet zurückgehaltene AWGN-Punkte und andere Kanalrealisationen;
synthetische Quellen, Randoptima und inkompatible PHY-Metadaten werden abgelehnt.
EESM und PER folgen unverändert den Abschnitten 5.6/5.7.

Der Offset gilt ausschließlich bei passendem `HeAwgnCalibrationKey`
(MCS, BCC/LDPC, Bytezahl, NSS, RX-Antennen, Kanalbreite, GI). Sonst bleibt der
ErrorRateModel-Pfad unverändert. Es gibt keine allgemeinen Default-Betas.
R2 installiert den bestehenden Piloten explizit:

```
    helper.LoadHeCalibrationProfile("contrib/sionna/data/he_mcs0_bcc_32_calibration.json");
    helper.Apply(sionna, txPhyHelper, rxPhyHelper, spectrumModel);
    helper.InstallHeCalibration(rxDevice); // nach Geräteanlage, vor Simulator::Run()
```

Der Loader prüft Status, Quellenart, PHY-Kontext und den Pilotkurvenhash;
Quellversion und Referenzhash werden unverändert übernommen. Die Installation
prüft den tatsächlichen `TableBasedErrorRateModel` einschließlich Defaultattributen.
32 Byte bedeuten hier eine nicht aggregierte PSDU einschließlich MAC-Header/FCS,
keine Anwendungsnutzlast, S-/A-MPDU- oder zeitbasierte Chunklänge. Inkompatible
Präambel, MCS, Codeart, NSS, RX-Antennen, Breite, GI oder Länge bleiben unverändert.
Der skalare 1x1-Empfang behält seine physikalische PSD; nur HE-SU-Daten werden
aus den 234 Datentönen durch EESM und Offset abgebildet. Empfängereigene
PPDU-Snapshots verfallen nach Empfangsende. Bei skalarer Interferenz gilt
$\gamma_k^{(j)}=\gamma_k^{(0)}\gamma_{\rm scalar}^{(j)}/\gamma_{\rm scalar}^{(0)}$.
Für Chunkdauer Δt_j und gesamte MPDU-Dauer T gilt
$P_{s,j}=P_s(8L,\gamma_{\rm EESM}^{(j)}10^{\delta/10})^{\Delta t_j/T}$.
Damit verändern Symbolpadding und eine Aufteilung bei konstantem SINR die
kalibrierte PSDU-Wahrscheinlichkeit nicht. Header/CCA bleiben unverändert;
`CalibrationApplied` meldet die tatsächlich verwendete Abbildung.

Kanalstudien. `tools/benchmark_sumimo.py` und
`tools/benchmark_rt_convergence.py` verwenden feste 2x2-Links in
`simple_room`, 256 FFT-Bins einschließlich DC/Guards, gleiche Gesamtleistung,
MMSE/SVD und MI-Diagnostik. Seeds 42/43/44 bei ns-3 Run 1 werden für
10000/100000/1000000 RT-Samples gepaart; deterministische Wiederholungen
sind keine unabhängigen statistischen Stichproben. Die vorab festgelegten
Vergleichsgrenzen sind: 0,5 dB Verlust, identischer Tonrang, Singularwerte
höchstens 5 % relativ plus 1e-7 absolut, 1 dB Ton-SINR, 0,5 dB MI-SINR
und 0,2 bit/s/Hz MI-Effizienz. Ein Konvergenzbefund verlangt alle ausgewählten
Fall-/Seedpaare. Die genauen Geometrien und Solveroptionen stehen im jeweiligen
`plan.json`; unabhängige NumPy-Referenzen prüfen die native Rechnung.

Der Alterungstest vergleicht frisches/verzögertes gleichleistungs-SVD gegen
denselben Sendekanal, der durch eine unitäre TX-Basisrotation entwickelt wird.
Nullalter und ein statischer Kanal sind Kontrollen. Die Rotation erhält Energie
und Singularwerte; sie ist keine Doppler-/Mobilitätsreferenz. MI-Ergebnisse
belegen keine 2x2-PER- oder Wi-Fi-Durchsatzkalibrierung.

Cache und IPC. `tools/benchmark_cache_ipc.py` misst den Produktionscache
und getrennte lokale REQ/REP-Prozesse mit eingefrorenen Protobufs. Synthetisch

vergrößerte Tensoren sind Transportfixtures. Für einen Snapshot gelten
\(B_H=16N_{\mathrm{sc}}N_{\mathrm{rx}}N_{\mathrm{tx}}\) und
\(B_F=4N_{\mathrm{sc}}\) auf der verwendeten Plattform; logische Nutzbytes,
Kapazitäten, glibc-Heapproxy und RSS werden getrennt ausgewiesen.
`SionnaPropagationCache::ResourceLimits` konfiguriert die Grenzen;
`GetStorageStatistics()` liefert aktuelle Belegung und Spitzen.
Gültigkeit ist inklusiv. Abgelaufene inaktive Links werden bereinigt;
bei deaktiviertem Cache wird jede neue vollständig validierte Antwort verwendet,
Wiederaktivierung beginnt kalt. Dimensions-, Batch-, Antwort-, Snapshot- und
Nutzbytegrenzen werden vor CFR-/Cacheallokation geprüft; zukünftige Einträge
werden nicht zur Budgetanpassung verdrängt. Look-ahead-Tiefe ist
`floor(sub_mode/receiver_count)`; Nulltiefe wird vor RT abgelehnt.

Öffentliche CFR-/Frequenzergebnisse bleiben unabhängige Werte. Interne Getter
leihen Einträge nur während des Aufrufs aus; MU-CFRs werden einmal validiert,
nach expliziter Nutzer-ID übernommen und verschoben. Atomare Datenvalidierung
besteht weiterhin; Allokationsfehler bei der Übernahme besitzen keine volle
Rollbackgarantie. Antwortbudgets begrenzen weder Transportpuffer noch RT-/CIR-
Zwischenspeicher oder RSS. Heap-/RSS-Beobachtungen sind keine harte Speicherfreigabe.

`--copy-profile` misst fünf Getter separat. `--workload-sweep` kombiniert
Cache an/aus, 1/16/64-Link-Arbeitsmengen und Tiefen 1/4/16, je drei Prozesspaare.
Bei 64 Tönen und 2x2 sind \(B=4352\) Byte je Snapshot. Ein zyklisches Fenster
\(A=\max(1,L/8)\) ist für D Epochen aktiv; ein Batch trägt D Zukunftssnapshots.
Bei T Epochen werden \(4AT\) Getter aufgerufen, gecacht \(T/D\) RPCs und
ungecacht \(4AT\). Retention muss pro Epoche exakt \(A(D-e\bmod D)\) Snapshots
betragen, ungecacht null; nach finalem Ablauf auch Kapazität null.
Ganze Epochen einschließlich Koeffizientenprüfung werden separat von Getter-/RTT-
Timern gemessen. Transcripte prüfen jeden Request nach Link und Zeit.

R3: begrenzte Mobilitäts- und Pool-Speicherung (abgeschlossen).
`--mobility_history_limit H` begrenzt die gemeinsam gespeicherten Positions-/
Geschwindigkeitspaare pro Knoten (Default 1024; 0 oder `collect_history=False`
deaktiviert die Historie). Bei gleicher Zeit werden bestehende Arrays überschrieben;
nach Füllung werden die ältesten zwei Arrays mit `np.copyto` wiederverwendet.
Getter und Historienexport liefern unabhängige Werte; nicht mehr gespeicherte
RandomWalk-Zeitpunkte erzeugen `KeyError`. Direkte Historienarrays sind geliehene,
wiederverwendete Slots. Für N Knoten gilt für die NumPy-Nutzdaten
\(B_{hist} \le 48NH\): zwei float64-Vektoren mit je drei Komponenten.

Look-ahead schreibt keine Zukunftshistorie: `clone_for_prediction()` kopiert nur
veränderlichen Bewegungszustand und lokale RNGs. Pro Empfänger und Epoche werden
Position/Geschwindigkeit explizit gespeichert, unabhängig von H. Mit R Empfängern
und D budgetgeprüften Epochen gilt \(B_{forecast}=48RD\), zusätzlich 48N Byte
Positions-/Geschwindigkeitszustand der Klone. Nach jeder Vorhersage, auch nach
Platzierungs-/CFR-Fehlern, werden die ursprünglichen Knoten, ihre Historien und RNGs
wieder verwendet; das globale NumPy-RNG wird ebenfalls zurückgesetzt.
Bewegung bis zur tatsächlichen Anfragezeit wird vorher wie bisher ausgeführt;
diese Aktualisierung ist keine Transaktion. Python-/RNG-/RT-/CIR-Overhead und RSS
sind nicht in diesen Arraybytes enthalten. `mobility_storage_statistics()` trennt
Historien, vorübergehende Epochen-/Klonzustände und Spitzen.

`SetBatchPoolLimit(bytes)` begrenzt unabhängig von Cachebudgets einen leeren
nativen Stagingvektor (Default 1 MiB; 0 deaktiviert). Ganze Batches durchlaufen
zuerst die bestehenden Budgetprüfungen, dann Datenvalidierung vor Übernahme.
Ein RAII-Lease löscht alle Einträge auch bei Fehlern; nur leere strukturelle
Kapazität wird wiederverwendet. Übergrößen umgehen den Pool, kleinere Grenzen
geben übergroße freie Kapazität sofort frei. `GetPoolStatistics()` trennt Hits,
Misses, echte Reserve-Vergrößerungen, Bypässe und freie Kapazität vom CSI-Cache.
Es werden keine CFR-, Frequenz-, Protobuf- oder Transportpuffer gepoolt;
ungecachte CSI-Belegung bleibt null. Dies ist kein RSS- oder Latenzversprechen.

R4: instrumentierter Live-Server-/HE-SU-PHY-Pfad (abgeschlossen).
`tools/benchmark_live_scaling.py` startet je Wiederholung einen frischen
lokalen REQ/REP-Prozessverbund; der Server ruft die produktive
`SionnaEnv.init_simulation_env`-/`compute_cfr`-Implementierung auf.
`examples/ns3_sionna_live_benchmark.cc` sendet echte HE-SU-PPDUs über
SpectrumWifiPhy, Cache und Sionna-Adapter. Alle Rohrequests/-antworten werden
mit Anfrageindex, TX/RX und simulierter Zeit erhalten. Die Abstraktion und
SINR/PER-Rechnung bleiben ausschließlich in ns-3.

Die sechs repräsentativen Formen `(Empfänger,Antennen je Ende,MHz,Tiefe)` sind
`(1,1,20,1)`, `(4,1,20,1)`, `(1,2,40,1)`, `(4,2,40,1)`, `(1,1,20,4)` und
`(4,2,40,4)`: jeweils Cache an/aus und drei frische Prozesse bei 10000 Samples.
Die vierte Form wird zusätzlich bei 100000 Samples gepaart. Seed/Run/Streams
sind 42/1/0–1; LLVM verwendet einen Thread. Vier Epochen mit je zwei PPDUs
beginnen bei 1 ms, Epochenabstand 6 ms und Paketabstand 1 ms. Kohärenzzeit 5 ms,
MCS 0/BCC/800 ns, 256 Byte Nutzdaten beziehungsweise 284 Byte PSDU, offene
räumliche Multiplexierung, isotrope V-Arrays mit 0,5 Lambda und 20 dBm bleiben
fest. Beta 2 ist eine explizite Diagnosegröße, keine zusätzliche Kalibrierung.
Ein vorheriger 0-dBm-Diagnoselauf zeigte frühe Präambelabbrüche in einzelnen
2x2-Fällen; die akzeptierte Messung wird deshalb vorab bei 20 dBm festgelegt
und verlangt erfolgreiche vollständige PHY-Rezeption in jedem Prozess.

Der produktive Helper erweitert den CFR-Bereich um die SpectrumWifiPhy-Guards:
20/40 MHz PHY entsprechen 60/120 MHz und 768/1536 CFR-Bins. Für L Empfänger,
D Zukunftsintervalle und A Antennen je Ende gilt

$$
 B_{\rm CSI}=LDN_{\rm sc}(16A^2+4)\ {\rm Byte}.
$$

Der größte geplante Batch enthält 1671168 logische Byte. Kapazität, struktureller
Stagingpool, Mobilitätsarrays, serialisierte Wirebytes sowie nativer/Server-RSS
bleiben getrennte Größen. `peak_rss_bytes` ist ein Stichprobenmaximum;
`VmHWM` ist das Linux-Prozessmaximum einschließlich Initialisierung und JIT.
Die beiden Prozessmaxima werden nicht als gleichzeitiger Gesamtspeicher addiert.

Native Timer trennen `WifiPhy::Send` (skalare Loss-/Delay-RPCs), den später
geplanten Adapteraufruf (CFR-RPC, MMSE/EESM und MimoRx-Traceausgabe) und
`Simulator::Run`. Servertimer trennen Parsing, produktive Berechnung,
Serialisierung und Send. Servervalidierung/Rohdateien liegen außerhalb dieser
Servertimer; Simulation und Adapter enthalten die beschriebenen Beobachterkosten.
Für n Beobachtungen werden n, Median und empirisches p95 ausgewiesen, jeweils
erste Kanalrechnung und Folgerechnungen getrennt. Ohne Folgerechnung existiert kein
warmer Messwert (JSON `null`); drei gleiche Seeds sind Zeitwiederholungen, keine unabhängigen
Kanäle. Für simulierte Dauer T_sim und gemessene Laufzeit T_wall wird zusätzlich
T_sim/T_wall angegeben. Unsynchronisierte Prozessuhren werden nicht subtrahiert.
Alle Latenz-/RSS-Werte bleiben deskriptive instrumentierte CPU-Beobachtungen.

Vier Produktionsgetter je Empfänger und PPDU erzeugen ungecacht vier RPCs;
ein gecachter P2MP-Batch bedient dagegen die ganze Gruppe, Look-ahead auch
künftige Intervalle. Abnahme prüft jeden RPC nach Identität/Zeit sowie jede
Antwort nach Gültigkeit, Shape, Frequenzgitter, Normierung und Endlichkeit.
Statische Cachepaare vergleichen tatsächliche native PHY-Metriken und initiale
Roh-CFRs mit rtol 1e-4/atol 1e-7. Ungecachte CSI-Belegung muss überall null sein,
Poolkapazität höchstens 1 MiB. Das experimentlokale REP-Frontend verändert den
Produktionsserver nicht; automatische MAC-/MU-Steuerung, Doppler und GPU
bleiben außerhalb dieser Abnahme. Details stehen im
[R4-Vertrag](doc/r4-live-scaling.md).

Provenienz. Neue Läufe schreiben `run.json` (`ns3sionna.run`, Version 1)

mit Konfiguration, Quellen-/Eingangs-/Artefakthashes, tatsächlichem Backend,
Hardware/Build/Paketen, Befehlen und atomarem Status `incomplete`, `failed`
oder `success`; wissenschaftliche Abnahme steht separat unter `validation`.
Nichtleere Ausgabeordner bleiben geschützt; Resume wird nicht angeboten.
UTF-8-Text besitzt zusätzlich einen versionierten CRLF→LF-Digest, Binärdaten
bleiben rohbytegeprüft. Historische Exporte und Messzeitpunkt-Quellhashes werden
nicht umgeschrieben; ohne Sidecar ist ihr Ausführungsstatus unbekannt.
Fehlerlogs und verfügbare Prozessausgaben bleiben erhalten. Details des Formats
stehen in doc/provenance-cpu-increment.md.

R1: native Reproduzierbarkeit (abgeschlossen). Der feste Umfang umfasst
Seed/Run/Random-Streams, vollständige portable Studienbundles und leichte
Lock-/Systemprüfungen im bestehenden Conda-CPU-Environment. Abnahme: ein
verschobenes Bundle verifiziert und reproduziert einen kleinen RT-/Native-Fall;
Seedwiederholung und die etablierten CPU-Regressionen bestehen. Container/Docker,
Resume, neue Studien und GPU-Abnahme gehören nicht dazu; R2 bleibt geschlossen.

`SionnaHelper::AssignStreams(s)` reserviert RT-ID s und Mobilitäts-ID $s+1$
(Default 0/1, Rückgabewert 2). Andere ns-3-Zufallsobjekte benötigen disjunkte IDs.
`SimInitMessage.randomness` überträgt Mappingversion 1, tatsächlichen ns-3-Seed,
Run und beide IDs. Für Domäne q (`rt` oder `mobility`) gilt

$$
\begin{aligned}
 D_q&=\operatorname{SHA256}(\operatorname{ASCII}(\\
 &\texttt{ns3sionna-rng-v1:seed:run:stream:q})),\qquad\\
 s_q&=\operatorname{uint32be}(D_q[0:4]).
\end{aligned}
$$

Alle Zahlen sind dezimal, ohne Leerzeichen/Endzeile. RT bei Run 1/Stream 0
verwendet direkt den ns-3-Seed: 42/43/44 bleiben erhalten. Beide PathSolver-Pfade
verwenden s_{rt}; jeder Mobilitätsknoten verwendet
`SeedSequence([s_mobility,node_id,0x51A0])`. Ziehungen anderer Knoten/RT verändern
diesen Generator nicht. Das emuliert keine ns-3-Zufallsfolge und beweist keine
statistische Unabhängigkeit. Wiederholte gleiche Tupel sind gepaarte Wiederholungen.
Ohne neue Felder bleibt der Legacyvertrag (Version 0) erhalten; unbekannte
Versionen/ungültige Tupel scheitern. Sidecars erfassen Tupel und effektive Seeds.

`experiment_bundle.py` kopiert Konfigurationen, notwendige Szenen-/Mesh-/Material-
und Laufinputs sowie Ergebnisse/Logs unverändert. Der externe
[Artefaktkatalog](experiments/r1/catalog.json) hält den vertrauenswürdigen Indexhash
und relative lokale Zugriffspfade; ein gehosteter Download wird nicht behauptet.
Verifikation prüft das vollständige Inventar, relative Pfade, Referenzabschluss
und Rohbinär-/LF-Texthashes. Historische Messzeitpunkt-Quellhashes bleiben erhalten;
fehlende Ausführungsprovenienz bleibt unbekannt. Reproduktion schreibt neue
Ausführungs-/Validierungsnachweise außerhalb des unveränderten Bundles.
Native Metriken werden mit rtol $1\mathrm{O}^{-4}$ und atol $1\mathrm{O}^{-7}$ verglichen;
Rang, Tonindex und Frequenz sind exakt. Laufzeit und maschinenübergreifende
Bytegleichheit gehören nicht zur numerischen Abnahme. `--mode rt` rechnet frisch,
`--mode frozen` analysiert den gespeicherten Kanal. 10000 Samples sind ein
Reproduktionsprüfsatz, kein Konvergenznachweis.

`check_cpu_environment.py --strict` prüft Python 3.12.12, pip 26.0.1, alle
55 bestehenden Paketpins, `pip check`, Compiler/CMake/Protobuf/ZMQ/LLVM, aktuelle
Bindings und tatsächliche LLVM-Ausführung. Systemversionen sind dokumentiert,
nicht hermetisch gesperrt. Befehle und Format stehen im
[nativen Vertrag](doc/provenance-cpu-increment.md).

6. HIGH-LEVEL-API-VERTRAG BIS UNTERPHASE 4.8

6.1 TYPEN, BESITZ UND DEFAULTS

SionnaMimoHelper ist ein wertbasierter, simulationsunabhängiger Konfigurationshalter. Sein Konstruktor erzeugt einen gültigen SISO-Vertrag; der Konstruktor mit SionnaMimoConfiguration und SetConfiguration prüfen eine vollständige Kopie, bevor sie den gespeicherten Zustand ersetzen. Eine fehlgeschlagene Änderung lässt den bisherigen Zustand damit unverändert. Alle Validierungsfehler sind std::invalid_argument mit einem SionnaMimoHelper-Präfix und benötigen weder Simulator, Node noch eine ZMQ-Verbindung.

SionnaMimoArrayConfiguration enthält Zeilen, Spalten, vertikalen und horizontalen Abstand in Wellenlängen, Elementmuster und Polarisation. Die repräsentierten Sionna-Muster sind isotrop, dipole, half-wave dipole und TR 38.901. Vertikale und horizontale Polarisation erzeugen je Element einen, duale Polarisation zwei Antennenports. Für Zeilenzahl R, Spaltenzahl C und Polarisationsfaktor $p\in\{1,2\}$ gilt deshalb

$$
  N_\mathrm{array} = R C p.
$$

SionnaMimoOrientation speichert Azimut, Elevation und Rollwinkel in Grad. Die SionnaMimoConfiguration führt beide Arrays, den Modus, TX-/RX-Antennenzahl, maximale NSS, Empfänger, Normalisierung, Empfängerrauschzahl, Effective-SINR-Modell, explizite EESM-Betas und eine Map von ns-3-Node-ID auf Orientierung sowie optional MimoPrecodingMatrix oder eine explizite SionnaMimoBeamformingConfiguration zusammen. Die festgeschriebenen Defaults sind:

  - Modus SISO, eine TX-Antenne, eine RX-Antenne und maximale NSS eins;
  - TX-Array 1 x 1, Abstände 0,5 Wellenlängen, TR-38.901-Muster, vertikale Polarisation;
  - RX-Array 1 x 1, Abstände 0,5 Wellenlängen, Dipolmuster, vertikale Polarisation;
  - keine explizite knotenspezifische Orientierung; der API-Lesezugriff liefert dafür neutrale Nullwinkel, während Apply kein IPC-Feld erzeugt und der Server deshalb für alte und neue 1x1-Aufrufe exakt seine bisherige interne Orientierung [0, -180, 0] beibehält;
  - linearer MMSE-Empfänger und gemeinsame Unit-Average-Power-Normalisierung über alle Subcarrier und Antennenpaare gemäß Abschnitt 4.3;
  - 7 dB Empfängerrauschzahl entsprechend dem ns-3.40-WifiPhy-Default;
  - EESM als HE-Effective-SINR-Modell, aber weiterhin keine impliziten MCS-Betas;
  - weder ein fester Precoder noch Beamformingoptionen oder eine ausgewählte Precodingmomentaufnahme. Dadurch nehmen alle bisherigen 1x1- und offenen Mehrstromkonfigurationen weiterhin exakt den Phase-2-Pfad.

SetNodeOrientation, ClearNodeOrientation und GetNodeOrientation verwalten die knotenbezogenen Winkel. SetEesmBeta, HasEesmBeta und GetEesmBeta verwalten explizite lineare Betas für HE MCS 0 bis 11. SetPrecodingMatrix, ClearPrecodingMatrix, HasPrecodingMatrix und GetPrecodingMatrix verwalten den optionalen festen Precoder atomar. Die gespeicherte Konfiguration bleibt ein wertbasierter Vertrag und verändert Simulationsobjekte erst mit Apply.

6.2 UNTERSTÜTZTE BETRIEBSARTEN

GetSupportedModes und IsModeSupported melden vier akzeptierte Modi:

1. SISO mit genau einer TX- und einer RX-Antenne sowie NSS eins;
2. SU_MIMO_SPATIAL_MULTIPLEXING mit linearem MMSE-Empfänger. Ohne W[k] bleibt genau ein Stream je TX-Antenne aktiv; mit explizitem festem W[k] bestimmt dessen Spaltenzahl N_s;
3. SU_BEAMFORMING mit explizit gewähltem MRT oder SVD. SelectPrecoding erzeugt

   aus der übergebenen CSI-Matrix genau eine unveränderliche Auswahl, die
   Apply anschließend über den vorhandenen festen-W[k]-Pfad installiert.
4. MU_MIMO für den Helper-installierten HE-Downlink mit einem Stream und einer
   Empfangsantenne je Station, einer Vollband-242-Ton-RU und expliziter
   Laufzeitkonfiguration.

Festes benutzergegebenes Precoding bleibt bewusst
SU_MIMO_SPATIAL_MULTIPLEXING zugeordnet, weil es keine aus H[k] abgeleitete
Strahlformung implementiert. MRT ist weiterhin auf perfektes aktuelles
2x1-MISO-CSI und einen Stream begrenzt. SVD unterstützt Gleichleistung,
quantisiertes HE-/EHT-SU-Feedback und explizites CSI-Alter. Räumliche
Wasserfüllung ist mit unquantisiertem aktuellem CSI verfügbar; ihre
Kombination mit Feedbackquantisierung oder Alterung bleibt aufgrund des
semi-unitären Feedbackvertrags unzulässig. MU_MIMO wird nur ohne explizite `dlMu`-Konfiguration
oder außerhalb von
`InstallDownlink` eindeutig abgelehnt. Ebenso werden der
repräsentierte Zero-Forcing-Empfänger, eine unnormalisierte CFR und Mutual
Information als PER-wirksames Effective-SINR-Modell abgelehnt. Die
MI-Kompression aus Abschnitt 5.4 bleibt eine Diagnosemetrik.

6.3 ZENTRALE KONSISTENZPRÜFUNG

Validate erzwingt gemeinsam die Grenzen von Sionna, ns-3-Wi-Fi und dem
Phase-1-Abstraktionsmodell:

- Arraydimensionen und Abstände sind endlich und positiv; die abgeleitete
  Portzahl stimmt exakt mit numTxAntennas beziehungsweise numRxAntennas
  überein;
- TX-/RX-Antennenzahlen und maxNss liegen im ns-3.40-Bereich 1 bis 8;
- SISO besitzt exakt die Dimensionen 1 x 1 und maxNss eins;
- SU-MIMO ohne W[k] erfüllt weiterhin
  $2\leq N_\mathrm{tx}\leq N_\mathrm{rx}\leq8$ und
  $N_\mathrm{ss,max}=N_\mathrm{tx}$;
- SU-MIMO mit W[k] erfüllt
  $1\leq N_s=N_\mathrm{ss,max}\leq
  \min(N_\mathrm{tx},N_\mathrm{rx})$. TX- und Streamdimension stimmen
  exakt mit der Matrix überein; Subcarrierzahl, Koeffizienten und
  Leistungsnorm sind bereits Bestandteil des unveränderlichen
  MimoPrecodingMatrix-Vertrags;
- SU_BEAMFORMING besitzt mindestens zwei TX-Antennen, kein gleichzeitig
  benutzergegebenes W[k] und eine explizite MRT- oder SVD-Konfiguration;
- MRT verlangt genau eine RX-Antenne, einen Stream, unquantisiertes aktuelles
  CSI und unveränderte SVD-Standardoptionen;
- SVD prüft endliche positive Referenz-SNR und Rauschleistung, einen relativen
  Rangschwellwert in (0,1), HE/EHT und unquantisiertes, grobes oder feines
  SU-Codebook sowie nichtnegative CSI-Zeiten. Wasserfüllung schließt
  Quantisierung und positives CSI-Alter aus;
- jede Änderung der Konfiguration oder einer Knotenorientierung verwirft eine
  zuvor ausgewählte Momentaufnahme, damit Apply kein veraltetes W[k] nutzt;
- Azimut und Rollwinkel liegen endlich in [-180, 180] Grad, Elevation endlich
  in [-90, 90] Grad;
- die Rauschzahl ist endlich und nicht negativ;
- jedes vorhandene EESM-Beta gehört zu HE MCS 0 bis 11 und ist endlich und
  positiv.

ValidateForHeMcs ergänzt die nutzungsbezogene Vollständigkeitsprüfung: Für
Open-Loop-SU-MIMO und SU_BEAMFORMING muss das konkret verwendete MCS ein
explizites Beta besitzen. SISO verlangt wegen des unveränderten skalaren
Phase-1-Pfads weiterhin kein Beta. Damit werden keine unbelegten
Kalibrierwerte eingeführt.

6.4 GEMEINSAMER INTEGRATIONSAUFRUF

Apply erhält einen vorhandenen SionnaHelper, getrennte SpectrumWifiPhyHelper-Instanzen für Sender und Empfänger sowie den SionnaSpectrumPropagationLossModel-Adapter. Die Trennung der beiden PHY-Helper hält auch $N_\mathrm{tx}\ne N_\mathrm{rx}$ darstellbar. Apply validiert zuerst erneut den vollständigen gespeicherten Vertrag und verändert danach ausschließlich die explizit übergebenen Objekte:

```
    SionnaMimoHelper mimo(configuration);
    mimo.Apply(sionna, txPhy, rxPhy, spectrumModel);
```

Die Zuordnung ist eindeutig:

- txArray und rxArray werden auf die bestehenden ConfigureTransmitArray-/ConfigureReceiveArray-Aufrufe abgebildet;
- numTxAntennas beziehungsweise numRxAntennas setzen das Antennenzahlattribut des jeweiligen SpectrumWifiPhyHelper;
- maxNss setzt auf beiden Endpunkten MaxSupportedTxSpatialStreams und MaxSupportedRxSpatialStreams;
- receiverNoiseFigureDb setzt sowohl WifiPhy::RxNoiseFigure als auch den expliziten Rauschzahlwert des MIMO-Spectrumadapters;
- die vollständige EESM-Map ersetzt atomar die bisherige Adapterkonfiguration, sodass beim erneuten Anwenden keine veralteten MCS-Betas zurückbleiben;
- eine durch SelectPrecoding gespeicherte MRT-/SVD-Auswahl ersetzt den festen Adapter-Precoder vollständig und hat ausschließlich im Modus SU_BEAMFORMING Vorrang;
- precodingMatrix ersetzt den festen Adapter-Precoder vollständig. Fehlt der optionale Wert in einem Nicht-Beamformingmodus, wird ein zuvor gesetztes W[k] gelöscht und der Phase-2-Pfad wiederhergestellt.

Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung und EESM bleiben unverändert. Festes oder ausgewähltes W[k] wählt ausschließlich die bereits vorhandene $H[k]W[k]$-Überladung. Eine abweichende Empfänger-, Normalisierungs- oder Effective-SINR-Auswahl wird weiterhin vor Apply abgelehnt.

6.5 KNOTENORIENTIERUNG IM IPC- UND SERVERPFAD

NodeInfo besitzt das neue optionale Feld orientation mit Azimut, Elevation und Rollwinkel in Grad. SionnaHelper trägt nur explizit konfigurierte Knotenorientierungen ein. Der Python-Server prüft endliche Werte und dieselben Winkelgrenzen wie die High-Level-API, konvertiert Grad mit

$$
\boldsymbol\theta_\mathrm{rad}
= \frac{\pi}{180}\boldsymbol\theta_\mathrm{deg}
$$

und verwendet die drei Eulerwinkel beim Anlegen jedes Transmitter- und Receiver-Objekts im klassischen sowie im Look-ahead-Pfad. Das ausgelagerte PopulateMimoConfiguration schreibt Array- und Orientierungsfelder sowohl für Start als auch für den bytegenauen Integrationstest.

Fehlt orientation, verwendet der Server weiterhin exakt die bisherige Konstante [0, -180, 0]. Der Wert wird bewusst nicht als neue API-Orientierung interpretiert, sondern nur zur vollständigen Verhaltenskompatibilität alter Nachrichten beibehalten. Ein alter SionnaHelper-Aufruf konfiguriert weder Array- noch Orientierungsfelder und erzeugt deshalb weiterhin die bisherigen 1x1-Arrays und die bisherige Serverausrichtung.

Für eine 1x1-Verbindung reicht der Defaultvertrag. Er behält SISO, je eine TX-/RX-Antenne, NSS eins, 7 dB Rauschzahl, die bisherigen Arraymuster und den skalaren Legacy-Empfangspfad bei. Deshalb ist für diesen Pfad kein EESM-Beta erforderlich:

```
    SionnaMimoConfiguration configuration;
    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Das kleinste Vollrang-2x2-Beispiel verwendet zwei vertikale Elemente je Array, zwei offene Spatial Streams und ein vom Anwender kalibriertes Beta für den gewählten HE-MCS:

```
    SionnaMimoConfiguration configuration;
    configuration.mode = SionnaMimoMode::SU_MIMO_SPATIAL_MULTIPLEXING;
    configuration.txArray.numColumns = 2;
    configuration.rxArray.numColumns = 2;
    configuration.numTxAntennas = 2;
    configuration.numRxAntennas = 2;
    configuration.maxNss = 2;
    configuration.eesmBetaLinear.emplace(mcs, betaFromCalibration);

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Eine feste 2x1-Streamauswahl auf einem 2x2-Kanal ergänzt ausschließlich maxNss und den normalisierten Precoder:

```
    configuration.maxNss = 1;
    configuration.precodingMatrix = MimoPrecodingMatrix::FrequencyFlat(
    numSubcarriers, 2, 1, {{1.0, 0.0}, {0.0, 0.0}});

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Für frequenzselektives Precoding wird stattdessen der MimoPrecodingMatrix-Konstruktor mit $N_\mathrm{sc}N_\mathrm{tx}N_s$ Koeffizienten in sc/tx/stream-Reihenfolge verwendet. Beide Varianten durchlaufen dieselbe Validierung und denselben Adapterpfad.

ApplyForHeMcs führt zuerst ValidateForHeMcs aus. Ein ungültiger MCS oder ein fehlendes SU-MIMO-Beta bricht daher mit std::invalid_argument und einem SionnaMimoHelper-Präfix ab, bevor SionnaHelper, die beiden PHY-Helper oder der Spectrumadapter verändert werden. Apply bleibt für vorhandenen Code erhalten, wenn der verwendete MCS an dieser Stelle noch nicht bekannt ist.

Das Programm ns3_wifi_sumimo_minimal.cc enthält die vollständigen helperkonfigurierten 1x1-, offenen Vollrang-2x2-, Identitäts-Precoding- und 2x1-Streamauswahlszenarien. Es verbindet sie über eine feste normalisierte CFR mit dem echten UDP-/SpectrumWifiPhy-Pfad und behält zusätzlich den Phase-1-Rang-1-Vergleich:

```
    ./ns3 run "ns3_wifi_sumimo_minimal \
    --mcs=5 --referenceSnrDb=20 --packetSize=1200 \
    --eesmBetaLinear=<kalibrierter-linearer-Wert>"
```

Der feste Kanal ersetzt in diesem reproduzierbaren Beispiel nur die Kanalquelle. Array, Antennenzahlen, NSS, Rauschzahl und EESM werden nicht am

Adapter oder PHY gesetzt, sondern ausschließlich mit SionnaMimoHelper.

Für die Migration eines bestehenden Programms werden die manuellen Aufrufe ConfigureTransmitArray, ConfigureReceiveArray, die PHY-Attribute Antennas, MaxSupportedTxSpatialStreams, MaxSupportedRxSpatialStreams und RxNoiseFigure sowie SetReceiverNoiseFigureDb/SetEesmBeta am Spectrumadapter durch genau eine SionnaMimoConfiguration und ApplyForHeMcs ersetzt. SionnaHelper::Configure, SetMode, Start und Destroy, der PropagationCache sowie die normale SpectrumChannel-Verdrahtung bleiben unverändert. Ein Programm, das weiterhin nur SionnaHelper verwendet, nimmt damit unverändert den bisherigen 1x1-Defaultpfad.

Funktional unterstützt der Stand bis Unterphase 3.5 SISO, offenen SU-MIMO, festes benutzergegebenes Precoding und explizites SU-Beamforming mit MRT oder SVD. Für festes W[k] gilt weiterhin $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$. Die MRT-Fabrik erzeugt für eine explizit übergebene perfekte MISO-CFR ein normiertes Ein-Stream-W[k]. Die SVD-Fabrik erzeugt aus einer allgemeinen perfekten CFR ein festes W[k] für explizit gewählte, numerisch tragfähige Eigenmoden; gleiche Leistung bleibt Standard und räumliche Wasserfüllung ist eine explizite Option. Beide Fabriken verwenden anschließend unverändert den festen Precodingpfad. Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung der CFR und EESM bleiben unverändert. MimoCompressedBeamformingFeedback, MimoFeedbackReconstruction und MimoFeedbackDiagnostics bilden nun den getrennten Feedbackpfad. MimoCsiAgingModel liefert einen festen Precoder-Schnappschuss mit explizitem Zeitbezug und zwingt für die Linkberechnung eine separate Übertragungskanal-Matrix. Der Helper kapselt diese wertbasierten Ergebnisse unverändert; sie bleiben auch ohne Simulator testbar. Typische eindeutige Fehler sind:

- SionnaMimoHelper: open-loop SU-MIMO uses one stream per TX antenna bei widersprüchlichem maxNss;
- SionnaMimoHelper: SU beamforming requires an explicit MRT or SVD configuration bei fehlender Auswahlmethode;
- SionnaMimoHelper: water-filling cannot be combined with quantized feedback or CSI aging für nicht darstellbare Vertragskombinationen;
- SionnaMimoHelper: SU beamforming must select a channel-dependent precoder before Apply bei fehlendem CSI-Auswahlschritt;
- SionnaMimoHelper: ... requires an explicit EESM beta for HE MCS ... bei fehlender Kalibrierung für Open Loop oder Beamforming;
- SionnaMimoHelper: maximum NSS must equal the fixed-precoding stream dimension bei widersprüchlicher Helper-Konfiguration;
- WifiTxVector NSS must equal the transmitted stream dimension, falls WifiTxVector und W-Spalten beziehungsweise der alte offene TX-Pfad widersprüchlich sind;
- MimoPrecodingMatrix: MRT requires exactly one RX antenna beziehungsweise ... a non-zero channel on every subcarrier für mehrdeutige oder nicht normierbare MRT-Eingaben;
- MimoPrecodingMatrix: SVD stream count must be in ... beziehungsweise ... requested ... streams but subcarrier ... has numerical rank ... für ungültige Streamwahl oder Rangverlust sowie eindeutige Meldungen für nicht endliche CFR-, SNR-, Rausch- und Allokationseingaben.
- MimoCsiFeedback: SU compressed feedback requires a semi-unitary equal-power precoder beziehungsweise ... angle index exceeds the selected codebook width für nicht darstellbare Leistung oder ungültige Indizes;
- MimoCsiFeedback: CSI timestamp does not match update interval and feedback delay beziehungsweise ... no CSI feedback is available ... für einen off-grid oder noch nicht verfügbaren Schätzzeitpunkt.

6.7 KANALABHÄNGIGE AUSWAHL, DIAGNOSE UND PHASENBEISPIEL

SionnaMimoBeamformingConfiguration bündelt die explizite Methode MRT oder SVD, die unveränderten MimoSvdPrecodingOptions, MimoFeedbackConfiguration und MimoCsiTimingConfiguration. SionnaMimoPrecodingSelection besitzt den ausgewählten festen Precoder, CSI- und Sendezeitpunkt, CSI-Alter und, sofern vorhanden, die ursprünglichen SVD- und Feedback-/Alterungsergebnisse. Der Helper berechnet oder interpretiert diese Diagnosen nicht neu.

```
    configuration.mode = SionnaMimoMode::SU_BEAMFORMING;
    configuration.maxNss = 1;
    configuration.beamforming = SionnaMimoBeamformingConfiguration{};
    configuration.beamforming->method = SionnaMimoBeamformingMethod::SVD;
    configuration.beamforming->feedback.codebook = MimoFeedbackCodebook::SU_FINE;
    configuration.beamforming->csiTiming.updateInterval = MilliSeconds(5);
    configuration.beamforming->csiTiming.feedbackDelay = MilliSeconds(2);

    SionnaMimoHelper mimo(configuration);
    const auto& selection = mimo.SelectPrecoding(
    channelEstimateAtCsiTime,
    txNodeId, rxNodeId,
    csiTimestamp, transmissionTimestamp);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

SelectPrecoding validiert Konfiguration, CSI-Dimensionen und Zeitraster, erstellt das Ergebnis zunächst als lokalen Kandidaten und ersetzt den gespeicherten Zustand erst nach vollständigem Erfolg. SetConfiguration sowie Änderungen einer Knotenorientierung verwerfen die Auswahl. Apply lehnt SU_BEAMFORMING ohne vorherige Auswahl ab und installiert sonst ausschließlich selection.GetPrecodingMatrix() durch SetPrecodingMatrix. Der Adapter, das Signalmodell, die Leistungsnorm $\Vert W[k]\Vert_F^2=N_s$ und die 1x1-/Open-Loop-/Festprecodingpfade bleiben unverändert.

Das neue Programm ns3_wifi_sumimo_precoding.cc führt vier getrennte UDP-/SpectrumWifiPhy-CPU-Läufe aus. Open Loop und SVD verwenden denselben normalisierten 2x2-Kanal; fester Precoder und MRT verwenden denselben 2x1-MISO-Kanal, sodass der MRT-Gewinn direkt gegen die erste feste Antenne vergleichbar ist. Alle Fälle verwenden dieselbe Gesamtleistung P; die unterschiedlichen Frobeniusnormen $\sqrt{N_s}$ kompensiert das unveränderte Signalmodell durch P/N_s.

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --referenceSnrDb=16 --packetSize=1200 \
    --eesmBetaLinear=2"
```

Der Referenzlauf liefert für festes W[k] 16,000000 dB und 24,931200 Mbit/s, für MRT den analytischen Gewinn auf 19,010300 dB und 68,582400 Mbit/s und für Ein-Stream-SVD 21,563025 dB und 68,697600 Mbit/s. Die Precodingnorm bleibt in allen Ein-Stream-Fällen eins; Open Loop verwendet bei zwei Streams $\sqrt 2$. Das Beispiel bricht ab, falls die Gesamtleistungsidentität, die analytische MRT-SINR oder der ideale SVD-Gewinn verletzt wird.

Quantisierung und Alterung sind keine versteckten Defaults, sondern ausdrückliche Optionen:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --quantization=coarse \
    --csiUpdateIntervalMs=5 --feedbackDelayMs=2 \
    --transmissionTimeMs=13 --agingRotationDegrees=25"
```

quantization akzeptiert none, coarse und fine.
csiUpdateIntervalMs, feedbackDelayMs und transmissionTimeMs bestimmen

den nach Abschnitt 5.12 zulässigen CSI-Zeitstempel;
agingRotationDegrees verändert ausschließlich den aktuellen Sendekanal.
Wasserfüllung wird separat und weiterhin nur mit unquantisiertem aktuellem
CSI aktiviert:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --svdStreams=2 --waterFilling=1"
```

Jede Ausgabezeile besteht aus whitespace-getrennten schlüssel=wert-Feldern.
Die configuration-Zeile enthält Referenz-SNR, konstante Gesamtleistung,
Quantisierung, Wasserfüllschalter, Aktualisierungsintervall,
Feedbackverzögerung, CSI-Alter, kontrollierte Kanalrotation und das explizite
EESM-Beta. Jede scenario-Zeile enthält mindestens:

- throughput_mbps, expected_throughput_mbps und per;
- stream_sinr_db als kommaseparierte Streamliste sowie
effective_sinr_db;
- channel_rank, precoding_norm und power_fractions;
- csi_age_ns, wobei -1 einen nicht-CSI-basierten Modus bezeichnet;
- feedback_bits, chordal_error und rms_coefficient_error.

Eine abschließende validation=PASS-Zeile macht den Prozessstatus zusätzlich
maschinenlesbar. Die ExampleAsTest-Referenzdateien fixieren alle Felder für
den idealen, quantisiert-gealterten und Wasserfüllungsfall.

Die Migration ist additiv: Bestehende SISO-, Open-Loop- und
Festprecodingprogramme ändern weder Modus noch Aufrufreihenfolge. Nur
kanalabhängige Programme wählen SU_BEAMFORMING, füllen beamforming, rufen
nach Vorliegen einer expliziten CSI-Matrix einmal SelectPrecoding auf und
verwenden danach das bestehende ApplyForHeMcs. ClearSelectedPrecoding
ermöglicht eine bewusste Neuwahl. Es gibt weiterhin keinen automatischen
Simulator-Timer, keine implizite CSI-Abfrage und keinen MAC-Feedbacktransport;
diese Grenzen verhindern, dass das explizite Forschungsmodell eine
vollständige 802.11-Soundingprozedur vortäuscht.

7. VALIDIERUNG

Die native Suite sionna enthält die schnellen API-, Numerik- und Adapterfälle:

1. API-Defaulttest für den vollständigen legacy-kompatiblen SISO-Vertrag,
   leere EESM-Kalibrierung, fehlendes W[k] und alle vier gemeldeten Betriebsarten.
2. Gültiger 2x2-Vertrag mit dual polarisiertem TX-Element, knotenspezifischer
   Orientierung, explizitem Beta, Identitäts-Precoding, frequenzselektiver
   2x1-Streamauswahl sowie zulässigem 2-TX-/1-RX-/1-Stream-Grenzfall.
3. Zentrale Negativmatrix für nicht endliche, unvollständige,
   widersprüchliche oder nicht unterstützte API-Konfigurationen einschließlich
   Precoding-TX-/NSS-Dimension, unzulässiger Beamforming-/Feedback-/Alterungs-
   Kombinationen, MU-MIMO und fehlendem MCS-Beta.
4. Manuell und über SionnaMimoHelper konfigurierte 1x1-Integration mit
   bytegleicher Array-/Orientierungs-IPC-Nachricht, identischer installierter
   SpectrumWifiPhy-Antennen-/NSS-Konfiguration, gleicher Rauschzahl und
   EESM-Kalibrierung sowie exakt gleichen MMSE-, EESM- und PER-Ergebnissen.
5. Derselbe Integrationsvergleich für eine vollständige offene
   2x2-Konfiguration ohne W[k].
6. Manuell gesetztes und durch SionnaMimoHelper angewendetes
   frequenzselektives komplexes 2x1-W[k] liefern dieselben Koeffizienten,
   installierten NSS, Post-MMSE-SINRs, EESM-SINR und PER.
7. Layouttest eines 2 x 2 x 2-CFR-Tensors mit eindeutig nummerierten
   Koeffizienten.

8. Precoding-Vertragstest für sc/tx/stream-Reihenfolge, explizite Frequenzflach-Expansion, Frobenius-Leistung sowie Dimensions-, Endlichkeits- und Normierungsfehler.
9. Reversible dB-/Linear-Umrechnung einschließlich 10 dB gleich Faktor zehn.
10. Link-Budget-Test für Sendeleistung, Pfadverlust, \(B\), \(T\) und \(NF\) sowie die erwartete Halbierung der SNR bei Verdopplung von \(B\) oder \(T\).
11. Flacher orthogonaler 2x2-Kanal: EESM erhält identische Tone-/Stream-SINRs unabhängig vom positiven Test-Beta.
12. Frequenzselektiver SISO-Kanal: EESM stimmt mit der analytischen Zweitonformel für das konfigurierte Beta überein.
13. SISO-Parität zur direkten ns-3.40-ErrorRateModel-Abfrage für HE MCS 0, 3, 7 und 11, neun SNR-Punkte von -5 bis 35 dB sowie 32, 400 und 1458 Byte. Das sind 108 Kombinationen; MCS 11 prüft auch den TableBased-Fallback auf Yans.
14. Monotone PER auf einem normierten frequenzselektiven Vier-Ton-Kanal für dieselben vier MCS und drei Paketgrößen bei 26 SNR-Punkten von -10 bis 40 dB. Die 312 geprüften Werte bilden je Konfiguration eine nicht steigende Folge mit einem echten PER-Abfall.
15. Exakte Reproduzierbarkeit von Post-MMSE-SINR, EESM-SINR und PER über 20 Wiederholungen eines komplexen frequenzselektiven 2x2-Kanals.
16. SISO-Test mit \(h=2+j\) und \(\rho=10\); erwartet wird \(\gamma=50\).
17. Orthogonaler 2x2-Kanal \(\mathbf H=\mathbf I\) und \(\rho=20\); jeder Stream erhält \(\gamma=10\) und \(\eta=2\log_2(11)\).
18. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei \(\rho=8\); erwartet werden (4,16), (16,4) und die analytische MI-Kompression.
19. Fester-Precoding-Referenztest für komplexes SISO, Identitätsparität, indexgenaues komplexes \(H[k]W[k]\), identische MMSE-Ausgabe zum vorberechneten effektiven Kanal und 2x2 mit einem ausgewählten Stream.
20. Ein-Stream-MRT/MRC-Test für frequenzselektives komplexes 2x1-MISO, Einheitsleistung von W[k], analytischen Arraygewinn \(\rho\Vert h\Vert_2^2\), gemeinsame Phasendrehung, äquivalentes 1x2-SIMO-MRC, exakte 1x1-Parität sowie die eindeutige Ablehnung von Nullkanal und allgemeinem 2x2-MIMO.
21. Deterministischer SVD-/Leistungsallokationstest für diagonale, unitär gedrehte, frequenzselektive, rangdefiziente und gleichsinguläre Kanäle. Er prüft explizite Stream-/Ranggrenzen, den relativen Rangschwellwert, Phasen-/Entartungskonvention, 1x1-Parität und analytische Eigenmode-SINRs. Zwei Zweimodenfälle unterhalb und oberhalb der Wasserfüllschwelle prüfen aktive Moden, endliche nichtnegative Anteile, exakte Leistungserhaltung, Wasserstand, KKT-Bedingungen und mindestens die Gleichleistungsrate.
22. Mindestkanalrang eins beziehungsweise zwei für Rang-1- und orthogonale 2x2-Kanäle über alle Subcarrier.
23. Spectrumadapter-Test für exaktes Rauschleistungs- und AWGN-Diversity-Compensation-Mapping sowie SISO-, Identitäts- und 2x1-Precoding ohne doppelte Anwendung von Rauschen, Leistung oder Arraygewinn.
24. Deserialisierung einer alten Nachricht ohne Dimensionsfelder als \(N_\mathrm{sc}\times1\times1\).
25. Protocol-Buffer-Rundlauf einer synthetischen 2 x 2 x 2-CFR mit den komplexen Werten 1-j bis 8-8j und indexgenauer Prüfung.

26. Helperauswahl für exaktes MRT, quantisierte gleichleistungs-SVD mit vollständiger Zustandsidentität und drei Millisekunden CSI-Alter sowie räumliche Wasserfüllung. Der Fall vergleicht jedes W[k] mit der bereits abgeschlossenen Fabrik, prüft Feedbackbits, Leistungsanteile und Gesamtleistung, weist nach, dass Apply exakt dieselben Koeffizienten am Adapter installiert, und verwirft die Auswahl nach einer Orientierungsänderung.
27. Protocol-Buffer-Rundlauf eines absichtlich umgekehrt eingegebenen Zweinutzer-Snapshots. Er prüft die sortierte Nutzerfolge 3, 9, gerichtete TX-/RX-Identität, gemeinsamen Zeitstempel und Frequenzraster,

    sc/rx/tx-Indizes sowie heterogene 1x1- und 1x2-RX-Arrays bei gemeinsamem
    Zweiantennensender.
28. Atomare Negativmatrix für doppelte Nutzer, gemischte Zeitstempel und
    Richtungen, abweichende Frequenzraster sowie eine RX-Geometrie, die nicht
    zur CFR-Dimension passt. Jeder Fall wird vor Cacheverwendung als
    `invalid_argument` abgelehnt.
29. Fester-Downlink-MU-Referenzfall mit expliziter Nutzer-/Streamzuordnung,
    exakter Gesamtleistung bei gleichen und ungleichen Spaltengewichten,
    Einzelnutzerparität zu Phase 3 einschließlich 1x1, null Interferenz für
    orthogonale Nutzer und analytischer gewünschter Leistung,
    Inter-User-Interferenz sowie SINR für korrelierte Kanäle.
30. Nutzerspezifischer HE-Vertrag mit zwei MCS, EESM-Beta, Link Budgets und
    Paketgrößen. Beide effektiven SINRs und Paketwahrscheinlichkeiten werden
    gegen direkte Aufrufe des bestehenden EESM- und ErrorRateModel-Vertrags
    geprüft.
31. Atomare Phase-4.2-Negativmatrix für doppelte oder lückenhafte
    Nutzer-/Streamzuordnung, fehlende oder unbekannte Nutzer, Uplinkrichtung,
    abweichendes Frequenzraster, fehlende oder nicht endliche SNR,
    unvollständige HE-Eingaben, NSS-Widerspruch und Paketgröße null. Ein danach
    ausgeführter gültiger Referenzfall weist den unveränderten Vertrag nach.
32. Deterministische ZF-/RZF-Auswahl für orthogonale, korrelierte und
    schlecht konditionierte Kanäle. Der Fall prüft stabile Nutzer-/Spaltenordnung
    unabhängig von der Anforderungsreihenfolge, koeffizientengleiche
    Wiederholung, ZF-Interferenzunterdrückung, endliches explizites RZF,
    Gesamtleistung, Einzelnutzer-MRT-/Phase-4.2-Parität und den 1x1-Grenzfall.
33. Atomare ZF-/RZF-Negativmatrix für mehrere Streams je Nutzer, fehlende
    Nutzer, ungültige Regularisierung oder Rangtoleranz, Überlast, Rangverlust,
    nicht endliche CFR und den noch nicht definierten Mehrantennenempfänger.
    Ein gültiger Aufruf vor und nach allen Ablehnungen liefert identische
    Koeffizienten.

34. Nativer HE-DL-MU-Adapterfall mit umgekehrter Nutzerreihenfolge und von
    Nutzer-IDs verschiedenen STA-IDs. Er prüft zwei MCS, NSS eins und zwei,
    gleiche RU mit analytischer Interferenz und SINR \(5/6\), verschiedene
    RUs mit null Interferenz und SINR 10, nutzerweise PER, Link Budget,
    äquivalente PSD-Leistung sowie den 1x1-Grenzfall.
35. Atomare Adapter-Negativmatrix für unbekannte und nicht adressierte Nutzer,
    falsche Richtung und Präambel, EHT, TX-/RX-/NSS-Dimensionsfehler sowie
    unbekannte Ergebnisnutzer. Ein installierter gültiger MU-Zustand und der
    unveränderte SU-SINR-/PER-Pfad werden vor und nach Ablehnungen verglichen.
36. Echter `HePpdu`-Adapterfall mit AP- und zwei STA-MACs,
    `WIFI_PREAMBLE_HE_MU`, gemeinsamem 2x2-Precoder, unterschiedlichen
    Pfadverlusten und positivem Empfangsgewinn. Die je STA zurückgegebene
    skalare RU-PSD wird mit der zuvor gemeinsam berechneten Matrixreferenz
    verglichen.
37. Korrigierter numerischer RU-Vertrag mit gruppenweiser Precoderleistung,
    ungleichen Spaltennormen und Leistungsbudgets, ausgewählten EESM-Tönen,
    Invarianz außerhalb der Auswahl sowie Ablehnung ungültiger Tonlisten.
38. Unabhängiger Runtime-RU-Fall für 242-, 106- und mittlere 26-Ton-RUs. Er
    prüft Nulltöne, Frequenzzuordnung, fremde RU-Energie mit dem tatsächlichen
    Precoder, das 4:1-Präambelraster und normale Mithörer-/Fremdsenderpfade.
39. Echter Channel-/PHY-Lauf mit zwei adressierten STAs und einem Mithörer.
    Orthogonale Nutzer dekodieren mit analytischer SINR; korrelierte Nutzer
    erreichen die Nutzdatenphase und scheitern. Der Mithörer hält CCA busy,
    erhält aber kein PSDU.

Die eigenständige native Suite `ul-mu-mimo` ergänzt den analytischen
Unterphase-4.7-Fall. Sie prüft den Einzelnutzervergleich mit dem bestehenden
SU-MMSE-/MRC-Kern, orthogonale Nutzer, ZF-Rangverlust, rangdefiniertes MMSE,
den Nah-Fern-Fall mit Referenz-SNRs 100 und 1 sowie getrennte HE-EESM-/PER-

Abbildung für verschiedene MCS und Paketgrößen. ZF liefert im korrelierten Fall 75 und 0,75; MMSE liefert 87,5 und 0,752475247524752.

Die eigenständige native Suite `he-ul-mu-mimo-phy-adapter` ergänzt zwei Unterphase-4.8-Fälle mit echten `CtrlTriggerHeader`- und `HePpdu`-Objekten. Der positive Fall prüft umgekehrte Empfangsreihenfolge, zwei MCS, gemeinsame 242-Ton-RU, analytische orthogonale MMSE-SINRs 4 und 9, PER und äquivalente Empfangsleistung sowie die orthogonale Trennung zweier 106-Ton-RUs. Der Teilnahmefall weist nach, dass ein fehlender und ein um eine Nanosekunde verspäteter Nutzer nicht detektiert und getrennt gemeldet werden. Eine fremde Trigger-UID wird atomar abgelehnt.

Die eigenständige native Suite `sionna-mimo-csi-feedback` ergänzt zwei Fälle für Unterphase 3.4. Der Feedbackfall prüft HE und EHT, beide SU-Codebücher, Bitbreiten und Payloadgrößen, exakt rekonstruierte Grenzindizes, deterministische Wiederholung, den allgemeinen 3x2-Mehrstromfall, Leistungsrenormierung, Phasen-/Subraumfehler sowie die Ablehnung nicht semi-unitärer Wasserfüllungs-Precoder und ungültiger Indizes. Der Alterungsfall prüft Nullalter, getrennte CSI- und Sende-CFR, Rasterung mit Aktualisierungsintervall und Rückmeldedauer, statische und orthogonal gedrehte Kanäle, den daraus folgenden SINR-/PER-Abfall, frische CSI, vollständige Zustandsidentität und den unveränderten 1x1-Fall.

Die sieben reinen Python-Vertragstests prüfen Standard- und explizite Arraykonfiguration, bytegleiche 1x1-Serialisierung, die sc/rx/tx-Reihenfolge einer synthetischen 2x2-CFR, die gemeinsame MIMO-Normalisierung sowie die legacy-kompatible und explizite Knotenorientierung. Zwei Phase-4.1-Fälle ergänzen denselben Zweinutzer-Rundlauf und die Negativmatrix für Identität, Richtung, Zeitpunkt, Frequenzraster und heterogene RX-Geometrien. Diese Tests benötigen weder Raytracing, ZMQ noch eine GPU. Die analytischen Vergleiche des numerischen Kerns verwenden weiterhin eine absolute Toleranz von 1e-10; dies gilt auch für Identitäts-Precoding und synthetische $H[k]W[k]$-Referenzen sowie MRT-/MRC- und SVD-Eigenmode-SINRs. Precoderleistung, Phaseninvarianz, effektiver MRT-Kanal, deterministische SVD-Basis, Leistungsanteile, Wasserstand, KKT-Gleichheit und die neue Feedback-Leistungsidentität werden innerhalb 1e-12 geprüft. Die Link-Budget-/EESM-Vergleiche verwenden ebenfalls 1e-12.
Die Phase-4.2-Parität und Post-MMSE-SINR verwenden ebenfalls 10^{-10}; gewünschte Leistung, Inter-User-Interferenz, Precoderleistung, EESM und ErrorRateModel-Parität werden innerhalb 10^{-12} geprüft.
Phase 4.3 prüft ZF-Interferenz und Precoderleistung innerhalb 10^{-12} sowie Einzelnutzer-/1x1-SINR innerhalb 10^{-10}; deterministische Wiederholungen werden koeffizientengenau verglichen. Phase 4.7 prüft ZF-/MMSE-SINR und Einzelnutzerparität innerhalb 10^{-10} sowie Interferenz, EESM und PER innerhalb 10^{-12}.
Phase 4.4 prüft gleiche-RU-Interferenz, analytische SINR, EESM/PER, Link Budget und äquivalente Leistung innerhalb 10^{-12}. Die direkte HE-MU-PPDU-Integration verwendet relative 10^{-10}. Der unabhängige Runtime-RU-Test vergleicht EESM/Spektrum innerhalb 10^{-4}; diese Grenze bleibt deutlich unter dem Abstand verschiedener Ton- oder RU-Zuordnungen. Exakte Invarianz-, Nullenergie-, Adressierungs- und PHY-Ereignisbedingungen werden separat geprüft.

Die eigenständige native Suite `dl-mu-mimo-runtime` ergänzt zwei schnelle Fälle für Unterphase 4.5. Der Auswahlfall prüft feste und korrelationsbasierte Gruppen, stabile Nutzer-/STA-ID-/RU-/NSS-Zuordnung, lexikographische Gleichstandsauflösung, direkte ZF-/RZF-Parität, Gesamtleistung, Aktualisierungsraster, Rückmeldedauer, CSI-Alter und die exakte Installation im Phase-4.4-Adapter. Der Negativfall prüft EHT, Uplink, falschen AP, fehlende Bindings, unterschiedliche RUs, mehrere Streams, falsche und zukünftige Zeitpunkte, quantisierte MU-Rückmeldung, SU-Precoder-Rekonstruktion, Überlast, Rangverlust, Gruppen-/Zeit-Mismatch und

atomare Ablehnung eines älteren Ersatzes.

Der reale Server-Integrationstest konfiguriert 1x2-PlanarArrays an TX und RX, führt Sionna RT auf der CPU aus und prüft die Form 2 x 2 x 256, 1024 komplexe Werte und mittlere normierte Leistung eins. Die übrigen sieben Servertests decken den unveränderten 1x1-Pfad, Mobilität, Kohärenzzeit und Mehrnutzer-Anfragen ab.

Die ExampleAsTest-Suites enthalten nun zwölf Integrationsprozesse. Die fünf bestehenden Prozesse führen 1x1, Vollrang-2x2, Rang-1-2x2, Identitäts-Precoding und 2x1-Streamauswahl über UDP/SpectrumWifiPhy aus und prüfen widersprüchliches NSS, fehlendes MCS-Beta sowie den CFR-/PHY-Dimensionsguard. Beamforming ist dabei nicht länger pauschal reserviert.

Drei neue positive Prozesse führen das Phasenbeispiel mit idealer Auswahl, grob quantisiertem und drei Millisekunden altem CSI sowie räumlicher Wasserfüllung vollständig auf der CPU aus. Die Referenzlogs vergleichen alle maschinenlesbaren Felder. Vier neue negative Prozesse prüfen fehlende SelectPrecoding-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und einen nicht zum Aktualisierungsraster passenden CSI-Zeitstempel. Sie enden jeweils früh mit Status 2 und einer stabilen configuration_error-Zeile.

7.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 15. August 2026 wurden nach Abschluss der Unterphase 3.5 folgende Prüfungen auf einem CPU-System erfolgreich ausgeführt:

- gezielter Build von Bibliothek, nativer Testsuite, neuem Precoding-Vergleich und bestehendem SU-MIMO-Beispiel unter GCC 13.3
- native ns-3-Suite sionna: 26 von 26 Fällen bestanden. Der neue Helperfall vergleicht MRT und SVD einschließlich Feedback, Alterung und Wasserfüllung direkt mit den bereits vorhandenen wertbasierten Fabriken, prüft die installierten Koeffizienten und Leistungsanteile sowie die Invalidierung einer alten Auswahl. Der bestehende SVD-/Leistungsallokationsfall prüft zwei frequenzselektive diagonale Moden,
unitäre Rotation, gleiche Singularwerte, 1x1-Parität, explizite Stream-/Rangfehler und nicht endliche Eingaben. Wasserfüllung aktiviert bei \(\rho=0{,}5\) nur die starke Mode und bei \(\rho=4\) beide Moden mit den analytischen Anteilen 0,59375 und 0,40625; Leistung, Wasserstand, KKT-Gleichheit und ideale Rate stimmen mit den Referenzen überein. Die bestehenden Phase-1-, Phase-2- und Phase-3.1-/3.2-Fälle bleiben bestanden
- native ns-3-Suite sionna-mimo-csi-feedback: zwei von zwei Fällen bestanden; deterministische HE-/EHT-Quantisierung und Rekonstruktion, Leistungs- und Fehlerdiagnostik, allgemeines 3x2-SU-Feedback sowie CSI-Alterung mit expliziter Zeit- und Zustandsidentität stimmen mit den analytischen Referenzen überein. Der Alterungstest weist zusätzlich den erwarteten SINR-/PER-Verlust bei orthogonal gedrehtem Sendekanal nach
- SISO-Paritätsmatrix: alle 108 Kombinationen stimmen innerhalb 1e-12 mit der direkt aufgerufenen ns-3-Fehlermodellschnittstelle überein
- frequenzselektive Monotonitätsmatrix: alle 312 PER-Werte bestanden
- Reproduzierbarkeit: 20 identische 2x2-Aufrufe mit exakt gleichen Ergebnissen
- ExampleAsTest sionna-example-sumimo-minimal: ein von einem Fall bestanden; Vollrang-2x2 erreicht 134,3904 statt 68,1408 Mbit/s bei 1x1, Rang-1-2x2 erreicht bei PER 1 erwartungsgemäß 0 Mbit/s; Identitäts-Precoding reproduziert die analytischen Vollrangwerte und die 2x1-Auswahl erreicht 23,010300 dB bei voller Gesamtleistung
- ExampleAsTest sionna-example-sumimo-precoding: drei von drei Prozessen bestanden; idealer Vergleich, grob quantisiertes und drei Millisekunden altes CSI sowie Wasserfüllung liefern die vollständigen stabilen Diagnostikzeilen und validation=PASS

- ExampleAsTest sionna-example-sumimo-precoding-negative: vier von vier Prozessen bestanden; fehlende Precoder-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und ein off-grid CSI-Zeitstempel liefern jeweils die erwartete configuration_error-Zeile
- ExampleAsTest sionna-example-mimo-helper-configuration-negative: drei von drei Fällen bestanden; ungültiges NSS, nicht unterstütztes MU-MIMO und fehlendes MCS-Beta brechen jeweils früh mit Status 2 und eindeutiger Meldung ab
- ExampleAsTest sionna-example-mimo-helper-runtime-negative: ein von einem Fall bestanden; eine helperkonfigurierte 2x2-PHY mit 1x1-CFR erreicht den erwarteten Dimensionsguard im echten SpectrumWifiPhy-Laufzeitpfad
- reine Python-Vertragssuite: fünf von fünf Fällen in 0,001 s bestanden; der separate Legacy-Importlauf bestätigte Sionna 1.2.1, Mitsuba 3.7.1, NumPy 1.26.4, TensorFlow im CPU-Modus ohne GPU und ZeroMQ
- gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 153,015 s bestanden; der reale 2x2-Fall lief in 1,6164 s, lieferte 2 x 2 x 256 und erfüllte die Normalisierung innerhalb 1e-6
- P2P-End-to-End-Test example-sionna-sensing im unveränderten 1x1-Modus: drei 1 x 1 x 3072-CFR-Antworten, zwei empfangene UDP-Pakete zu je 1024 Byte und Cache-Hit-Rate 0,993056

7.2 MIGRATION UND VALIDIERUNG MIT SIONNA 2.0.1

Am 20. August 2026 wurde die Python-Serverumgebung auf Sionna 2.0.1 und Sionna RT 2.0.1 aktualisiert. Die reproduzierbare Abhängigkeitsbasis lautet:

- `requirements.txt` fixiert Sionna 2.0.1, Sionna RT 2.0.1, Mitsuba 3.8.0, Dr.Jit 1.3.1 und PyTorch 2.9.1; die von Sionna geforderte NumPy-Untergrenze ist 2.2.6.
- `requirements_cpuonly.txt` wählt zusätzlich das offizielle CPU-Rad PyTorch 2.9.1+cpu.
- TensorFlow wurde aus beiden Requirements und aus dem Server entfernt. Der Server erkennt den tatsächlich von Sionna RT gewählten Mitsuba-Variantennamen und unterscheidet darüber CUDA- und LLVM-Ausführung.
- Der ns-3-Seed wird bei beiden Raytracing-Pfaden nun mit `seed` explizit an `PathSolver` weitergegeben. Damit steuert der Simulationsseed auch die stochastische Strahlabtastung; zuvor blieb dort implizit Sionnas Standardwert aktiv.
- `Paths.cir()`, `Paths.cfr()`, `PlanarArray` und `PathSolver` behalten in Sionna RT 2.0.1 die hier verwendete API. Die CFR-Dimension [num_rx, num_rx_ant, num_tx, num_tx_ant, num_time_steps, num_subcarriers] und damit der bestehende sc/rx/tx-Datenvertrag bleiben unverändert.

Die aktualisierte CPU-Umgebung bestand folgende Prüfungen:

- strikte Importprüfung für Sionna 2.0.1 und Sionna RT 2.0.1 sowie Mitsuba 3.8.0, Dr.Jit 1.3.1, NumPy 2.5.2, PyTorch 2.9.1+cpu und ZeroMQ
- Python-Vertragssuite: fünf von fünf Fällen in 0,003 s bestanden
- gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 155,993 s bestanden; der reale MIMO-Fall lieferte auf dem LLVM-Backend erneut eine endliche, gemeinsam über alle Antennenpaare normalisierte 2 x 2 x 256-CFR
- native ns-3-Suite `sionna`: 26 von 26 Fällen bestanden
- native ns-3-Suite `sionna-mimo-csi-feedback`: zwei von zwei Fällen bestanden

Damit ist die Sionna-RT-Schnittstelle für 1x1 und 2x2 unter 2.0.1 validiert. Der Wechsel von TensorFlow zu PyTorch betrifft den vorhandenen RT-Kanalpfad nicht; TensorFlow ist für ns3sionna nach der Migration keine Runtimeabhängigkeit mehr.

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt: Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen

Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet
ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon
nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

7.3 VALIDIERUNG DER UNTERPHASE 4.1

Am 20. August 2026 wurden nach der Implementierung von Unterphase 4.1
folgende gezielte Prüfungen erfolgreich ausgeführt:

- Build des Ziels `sionna-test` einschließlich neu erzeugter
  Protocol-Buffer-Bindings
- native ns-3-Suite `sionna` mit allen 28 registrierten Fällen; die zwei neuen
  Fälle decken Rundlauf, Sortierung, Identität, Richtung, heterogene
  RX-Dimensionen und die atomare Negativmatrix ab
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,004 s
- gesamte Sionna-CPU-Serversuite mit acht von acht Fällen in 171,865 s,
  einschließlich der vorhandenen Mehrnutzer-, 1x1- und 2x2-Pfade
- unveränderte SU-Suites `sionna-mimo-csi-feedback`,
  `sionna-example-sumimo-minimal` und
  `sionna-example-sumimo-precoding`; alle Fälle bestanden
- `sionna-example-mimo-helper-configuration-negative` mit allen drei Fällen;
  insbesondere bleibt der MU-MIMO-Modus des Helpers abgelehnt und nimmt damit
  Unterphase 4.2 nicht vorweg

7.4 VALIDIERUNG DER UNTERPHASE 4.2

Am 21. August 2026 wurden nach der Implementierung von Unterphase 4.2
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test` mit den vier neuen nativen
  Quelldateien
- native ns-3-Suite `sionna` mit allen 31 registrierten Fällen; die drei
  neuen Fälle decken den festen MU-Precoder, Einzelnutzer- und 1x1-Parität,
  orthogonale und korrelierte Referenzkanäle, Gesamtleistung, nutzerweise
  EESM/PER und die Negativmatrix ab
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,005 s; damit
  bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen
  unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`,
  `sionna-example-sumimo-precoding` mit allen drei positiven Prozessen,
  `sionna-example-sumimo-precoding-negative`,
  `sionna-example-mimo-helper-configuration-negative` und
  `sionna-example-mimo-helper-runtime-negative` erfolgreich
- `git diff --check` ohne Befund

Die Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.2 ändert weder Python noch Protocol Buffers, Serialisierung,
Batching, Cache, ZMQ oder Sionna RT. Die schnelle Python-Vertragssuite und die
nativen Phase-4.1-Fälle prüfen die unveränderten Grenzen direkt.

7.5 VALIDIERUNG DER UNTERPHASE 4.3

Am 21. August 2026 wurden nach der Implementierung von Unterphase 4.3
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test`; die Änderung bleibt auf die
  bestehenden Phase-4.2-Quell-/Headerdateien und die native Testsuite begrenzt
- native ns-3-Suite `sionna` mit allen 33 registrierten Fällen in 0,024 s;
  die zwei neuen Fälle decken orthogonale, korrelierte, schlecht konditionierte,
  überlastete und rangdefiziente Kanäle, ZF-Unterdrückung, endliches
  deterministisches RZF, Leistung, Ordnung, Regularisierung, Mehrstrom-

ablehnung sowie Einzelnutzer-/1x1-/Phase-4.2-Parität ab
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,003 s; damit bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`, `sionna-example-sumimo-precoding` mit allen drei positiven Prozessen, `sionna-example-sumimo-precoding-negative`, `sionna-example-mimo-helper-configuration-negative` und `sionna-example-mimo-helper-runtime-negative` erfolgreich
- `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt: Unterphase 4.3 ändert weder Python noch Protocol Buffers, Serialisierung, Batching, Cache, ZMQ, Sionna RT oder einen Serverpfad. Die schnelle Python-Vertragssuite und die vollständige native Suite prüfen die berührten und unveränderten Grenzen direkt.

7.6 VALIDIERUNG DER UNTERPHASE 4.4

Am 3. September 2026 wurden nach der Implementierung von Unterphase 4.4 folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test`
- direkte native Suite `sionna` mit allen 36 registrierten Fällen in 0,048 s; die drei neuen Fälle decken STA-ID-/Nutzerzuordnung, gleiche und verschiedene RUs, MCS/NSS, analytische SINR/PER, Leistung, Dimensions- und Adressierungsablehnungen, SU-/1x1-Parität sowie einen echten HE-MU-PPDU ab
- `./test.py -s sionna --verbose` mit einem von einem Suite-Lauf in 3,77 s
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,005 s; damit bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` mit einem von einem Lauf in 3,54 s
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal` in 18,69 s, `sionna-example-sumimo-precoding` in 37,52 s, `sionna-example-sumimo-precoding-negative` in 4,28 s, `sionna-example-mimo-helper-configuration-negative` in 3,82 s und `sionna-example-mimo-helper-runtime-negative` in 5,08 s; jeweils ein von einem Lauf erfolgreich
- `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt: Unterphase 4.4 ändert weder Python, Protocol Buffers, Serialisierung, Batching, Cache, ZMQ, Sionna RT noch einen Serverpfad. Die schnelle Python-Vertragssuite, die native Suite und der echte HE-MU-PPDU-Adapterfall prüfen die berührten und unveränderten Grenzen direkt.

7.7 VALIDIERUNG DER UNTERPHASE 4.5

Am 3. September 2026 wurden nach der Implementierung von Unterphase 4.5 folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test`
- neue native Suite `dl-mu-mimo-runtime` mit beiden schnellen Fällen; feste und korrelationsbasierte Auswahl, ZF/RZF, Scheduleridentität, Zeitdiagnostik und die vollständige Negativmatrix bestanden
- native Suite `sionna` mit allen 36 bestehenden Fällen erfolgreich
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`, `sionna-example-sumimo-precoding`,

`sionna-example-sumimo-precoding-negative`,
`sionna-example-mimo-helper-configuration-negative` und
`sionna-example-mimo-helper-runtime-negative` jeweils erfolgreich
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,006 s
- `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.5 ändert weder Python, Protocol Buffers, Serialisierung,
Batching, Rohkanalcache, ZMQ, Sionna RT noch einen Serverpfad.

7.8 KORREKTUR DER FESTEN KOHÄRENZZEIT

Der Server konvertiert `fixed_coherence_time_ms` einmalig an der
Konfigurationsgrenze in `fixed_coherence_time_ns`:

$$T_{fixed,ns} = T_{fixed,ms} * 1\,000\,000$$

Alle drei festen Zeitpfade verwenden diese Nanosekundendauer. Null aktiviert
weiterhin die dynamische Berechnung. Die neue AST-basierte Vertragssuite prüft
die tatsächliche Konfigurationszuweisung sowie alle drei festen und dynamischen
Zweige ohne Raytracing. Die vollständige Python-/CPU-Serversuite bestand am
11. September mit allen 19 Fällen in 229,102 s. Sie umfasst Datenvertrag,
deterministische Mobilität, feste und dynamische Kohärenzzeit sowie reale
1x1-/2x2-CFRs. Die Serverfixtures verwenden 5 ms statt des früheren Testwerts
100000 ms. Diese Korrektur wurde mit Commit `8c112b6` eingecheckt.

7.9 ERNEUTE VALIDIERUNG DER KORRIGIERTEN UNTERPHASE 4.4

Am 12. September 2026 wurden nach Abschluss der Korrekturen folgende Prüfungen
auf derselben CPU-Referenzplattform erfolgreich ausgeführt:

- Build des Ziels `sionna-test` einschließlich der neuen nativen Testdateien
- `dl-mu-mimo-ru`: Spaltenleistungen 1,5/0,5 bei gleichen und ungleichen
RU-Budgets, relative Gewichte innerhalb derselben RU, Ablehnung einer Gruppe
ohne aktive Leistung, der analytische EESM-Gegenfall `[10,0]`, Invarianz
außerhalb der RU sowie ungültige Tonindizes
- `he-dl-mu-spectrum-ru`: unabhängig festgelegte belegte Tonbereiche für
Vollband-242-, getrennte 106- und zweigeteilte mittlere 26-Ton-RUs;
DC-/Null-/Guardausschluss, Frequenzselektivität, Invarianz außerhalb der RU,
tatsächliche `H W`-Energie einer fremden RU, 312,5-/78,125-kHz-Abbildung der
gemeinsamen Präambel, atomare Zustandsablehnung sowie physikalische Pfade
für Mithörer und einen anderen AP
- native Suite `sionna` mit allen 37 Fällen; der neue Fall sendet zwei normale
HE-DL-MU-PPDUs über `MultiModelSpectrumChannel` an zwei adressierte Stationen
und einen Mithörer. Beide Stationen dekodieren den orthogonalen Fall mit der
unabhängig berechneten PHY-SINR, erreichen im korrelierten Fall die
Nutzdatenphase und melden einen Empfangsfehler. Der Mithörer hält CCA busy,
dekodiert aber kein PSDU. Gemeinsame Nicht-HE- und HE-Portionen werden damit
im echten PHY-Ablauf erfasst.
- unveränderte Suite `dl-mu-mimo-runtime`; Phase-4.5-Auswahl, ZF/RZF,
Scheduleridentität und Negativmatrix bestanden
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`,
`sionna-example-sumimo-precoding`,
`sionna-example-sumimo-precoding-negative`,
`sionna-example-mimo-helper-configuration-negative` und
`sionna-example-mimo-helper-runtime-negative` jeweils erfolgreich
- vollständige Python-/CPU-Serversuite mit allen 19 Fällen, einschließlich
realer 2x2-CFR, erfolgreich
- `git diff --check` ohne Befund

Damit ist das Abnahmekriterium der Unterphase 4.4 nach der Korrektur erfüllt. Die Gruppenrechnung wird weiterhin einmal je adressiertem Empfänger ausgeführt; diese bekannte Leistungseinschränkung ändert die funktionale Abnahme nicht. Uplink und wissenschaftliche EESM-Kalibrierung bleiben spätere Unterphasen; der Downlink-Helper und Scheduler gehören seit Unterphase 4.6 zum getesteten CPU-Pfad.

7.10 VALIDIERUNG DER UNTERPHASE 4.6

Am 13. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- `./ns3 build sionna-test ns3_wifi_mumimo ns3_wifi_sumimo_minimal ns3_wifi_sumimo_precoding`
- die nativen Suites `sionna`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und `sionna-mimo-csi-feedback`
- die ExampleAsTest-Suites für SU-Minimal, SU-Precoding, dessen Negativfälle, Helper-Konfiguration, Helper-Laufzeit und `sionna-example-mumimo`; der neue MU-Vergleich bestand in 206,905 s mit dem stabilen Referenzlog
- `./ns3 run "ns3_wifi_mumimo --eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1"`; die Ausgabe endete mit `validation=passed methods=4 channels=2 timings=2`
- `conda run -n sionna-venv python tests/test_imports.py` sowie die Python-Vertragstests für Datenvertrag, deterministische Mobilität, Kohärenzzeit und Server-Unit-Tests. Die Umgebung meldete Sionna/Sionna RT 2.0.1, Mitsuba 3.8.0 mit `llvm_ad_mono_polarized`, PyTorch 2.9.1+cpu und keine CUDA-Laufzeit.

Der MU-Pfad verändert weder Python, Protocol Buffers, Cache-Serialisierung noch den Server. Ein optionaler RTX-3060-Raytracing-Lauf bleibt daher eine Regression des bestehenden Pfads und keine Voraussetzung für die CPU-Abnahme.

7.11 VALIDIERUNG DER UNTERPHASE 4.7

Am 13. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- gezielter Build von `libsionna` und `libsionna-test`; die beiden neuen Uplink-Quelldateien und die eigenständige Testsuite wurden übersetzt und gelinkt
- `./test.py --no-build -s ul-mu-mimo`: eine von einer Suite bestanden. Der analytische Fall prüft beide Detektoren im Einzelnutzergrenzfall, orthogonale Nutzer, ZF-Rangverlust, rangdefiniertes MMSE, Nah-Fern-SNRs 100 und 1 sowie getrennte HE-EESM-/PER-Abbildung
- die Regressionen `sionna`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und `sionna-mimo-csi-feedback`: jeweils eine von einer Suite bestanden, ohne Fehler, Überspringen oder Absturz
- `git diff --check` ohne Befund

Der neue Kern ändert weder Python, Protocol Buffers, Cache, Server, Scheduler noch SpectrumWifiPhy. Diese Pfade bleiben für die koordinierte Uplink-Integration der Unterphase 4.8 unverändert.

7.12 VALIDIERUNG DER UNTERPHASE 4.8

Am 14. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- `./ns3 build sionna-test ns3_wifi_mumimo`; Adapter, Tests und erweitertes Phasenbeispiel wurden übersetzt und gelinkt
- `he-ul-mu-mimo-phy-adapter`: beide Fälle bestanden. Echte HE-Basic-Trigger- und HE-TB-Objekte liefern für eine gemeinsame 242-Ton-RU die analytischen MMSE-SINRs 4 und 9; getrennte 106-Ton-RUs bilden zwei Gruppen. Fehlende und

  um eine Nanosekunde verspätete STAs werden getrennt gemeldet, eine fremde
  Trigger-UID wird abgelehnt.
  - die nativen Regressionen `sionna`, `ul-mu-mimo`, `dl-mu-mimo-runtime`,
  `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und
  `sionna-mimo-csi-feedback`: jeweils eine Suite bestanden, ohne Fehler,
  Überspringen oder Absturz
  - die unveränderten ExampleAsTest-Suites für SU-Minimal, SU-Precoding, dessen
  Negativfälle sowie Helper-Konfiguration und Helper-Laufzeit: jeweils
  erfolgreich
  - `sionna-example-mumimo`: der normale Referenzloglauf bestand in 221,483 s.
  Das ideale orthogonale Uplinkbeispiel liefert bei MCS 5 je Nutzer die
  SINR 10000 und PER null. Sequenzielles SU erreicht 50 Mbit/s, gleichzeitiges
  UL-MU 100 Mbit/s und damit exakt den Faktor 2. Der Nah-Fern-Fall liefert
  75 und 0,75; ZF lehnt Rangverlust ab, MMSE bleibt mit SINR 0,5 definiert.
  - vollständige Python-/CPU-Serversuite mit allen 19 Fällen in 191,753 s,
  einschließlich Datenvertrag, deterministischer Mobilität, Kohärenzzeit und
  realer 2x2-CFR, erfolgreich
  - `git diff --check` ohne Befund

Damit ist das Abnahmekriterium der Unterphase 4.8 erfüllt. Der Adapter bindet
die reale Trigger-/HE-TB-Metadatenebene an den unveränderten Uplinkkern; die
weiterhin idealisierte Synchronisation bleibt in API, Beispielausgabe und
Modellgrenzen ausdrücklich sichtbar.

7.13 PHASE-5-ERGEBNISSE UND VERBLEIBENDE GRENZEN

Die folgenden Werte stammen aus den dokumentierten CPU-Läufen vom
25. September bis 7. Oktober 2026; historische Messdaten bleiben unverändert.

Prüfung	Ergebnis	Gültigkeitsgrenze
5.1: HE MCS 0, BCC, 32 Byte, 1x1, 20 MHz, 800 ns GI	Beta 1,0383687667; Offset −0,4958921206 dB; AWGN-MAE 0,013024, selektive MAE 0,054886	erfüllt 0,05/0,10; selektiv verfehlt 0,05; ideales Timing/perfekte RX-CSI, drei Zweitapkanäle
R2, 7. Oktober: Pilot im tatsächlichen HE-SU-Empfang	Profil-/Kontext-/Fehlermodelltests, flache und selektive PHY-Rezeption, Chunkaufteilung und Ablauf bestanden	explizite Installation, nicht aggregierte 32-Byte-PSDU; keine neue Link-Level-Kalibrierung
5.2.1: Abstand/Orientierung	zwölf 2x2-Fälle, byteidentische Wiederholungen, Rang zwei an allen Tönen	feste Szene/Geometrien, MMSE-/MI-Diagnostik
5.2.2: Samplekonvergenz	alle zwölf 100000/1000000-Paare bestehen; 10000 Samples scheitern bei Fall 03/Seed 43	vier Links, drei Seeds; endliche Referenz, keine allgemeine Konvergenzgarantie
5.2.3: Cache/IPC	64/256/1024 Töne, fünf TX/RX-Formen, 1/16/64 Links; exakte Daten-/RPC-/Payloadprüfungen bestanden	eingefrorene lokale Prozess-IPC, kein Live-RT/PHY-Nachweis
5.2.4/5.2.5: getrennte Abstände/CSI-Alter	42 bzw. 24 initiale RT-Traces; Wiederholungen, Referenzrechnung und statische Kontrollen bestanden	Rohdaten lokal/unversioniert; analytische Rotation belegt keinen physikalischen Doppler
R3: Kopier-/Eigentumsprüfung	großer Matrix-/Loss-Median um 4,81 %/98,87 % verbessert; Matrixziel 20 % verfehlt; kleine Default-Regressionsgrenze 1,10 erfüllt	instrumentierte lokale Baseline, keine universelle Latenzgarantie
R3: Zeit-/Link-/Look-ahead-Sweep	18 Fälle, 54 Prozesspaare, 6912 Epochen, 52200 RPCs; maximale Retention 557056 Byte; ungecacht null	128 Epochen je Prozess; zwei zusätzliche 1024-Epochen-Läufe bestehen
R1, 7. Oktober	sieben frische Seed-Traces, verschobener 20-Dateien-Prüfsatz frisch reproduziert; 129 Studienartefakte verifiziert; strikter Lockcheck	native Conda-CPU-Abnahme, keine Container-/GPU-Freigabe
R4, 7. Oktober	42 frische Prozesspaare, 21 Cachekontrollen, 1890 RPCs und 912 erfolgreiche PHY-Rezeptionen	instrumentiertes lokales LLVM; feste Geometrie und Diagnose-Beta, keine GPU-/PER-Freigabe
CPU-Regressionen, 7. Oktober	57 Sionna- und 35 Wi-Fi-Fälle; 70 Python-Tests bestanden, zwei

optionale Exporttests ausgelassen | drei frische RT-Tests separat; hosted CI/CUDA hier nicht ausgeführt |

Die versionierten Referenzen stehen unter `data/phase5_2_spacing_orientation/`, `data/phase5_2_sample_convergence/`, `data/phase5_2_cache_ipc/` und in den drei `data/he_mcs0_bcc_32_*`-Dateien. Sie enthalten Pläne, Konfigurationen, Rohdaten/Hashes und Validierungsurteile. 5.2.4/5.2.5 sind zusätzlich als vollständige lokale Bundles im Artefaktkatalog zugänglich. Die jüngsten Cache-Messungen liegen unter `/tmp/r3-copy-*`, `/tmp/r3-workload-final` und `/tmp/r3-workload-long`; temporäre Pfade sind kein portables Reproduktionspaket.

R1, R3, R4 und R5 sind im jeweils festgelegten nativen CPU-Umfang abgeschlossen. Phase 5 insgesamt bleibt offen: physikalische Alterung und die CPU/GPU-CI-/Backendabnahme. Zusätzliche MCS, Codearten, Paketgrößen, MIMO-/MU-Profile und physikalischer Doppler besitzen weiterhin keine Freigabe.

Am 7. Oktober 2026 bestanden im bestehenden nativen Conda-CPU-Environment:

- gezielte Neubauten von Tests und beiden betroffenen Benchmarks nach Protobuf-/C++-Konfigurationsänderungen;
- strikte Umgebungsprüfung ohne Pinabweichung und tatsächliches `llvm_ad_mono_polarized`;
- sieben frische 10000-Sample-Traces: wiederholte Tupel (Run 1 und Run 2) liefern byteidentische Kanäle; alternative Runs/Streambereiche leiten andere Seeds ab; Run 1/RT-ID 0 erhält 42/43/44;
- verschobener repräsentativer Bundle: 20 Dateien verifiziert, frisches RT und eingefrorener Replay mit unabhängiger NumPy-Referenz bestanden; der ursprüngliche Szenenpfad war für den RT-Replay nicht verfügbar;
- Bundle der lokalen 5.2.4/5.2.5-Studien: 129 Dateien unverändert verifiziert, je ein nativer Replay bestanden; historische Ausführung bleibt unbekannt;
- etablierte CPU-Regressionen sequenziell: sieben native Suites, 58 Fälle; 98 Python-Tests, davon 96 bestanden und zwei optionale Exporttests ausgelassen.

Die vollständigen lokalen Pakete und unveränderlichen Indexidentitäten stehen im versionierten Artefaktkatalog. Neue Messdaten bleiben außerhalb von Git; historische Messhashes, Datensätze und R2 wurden nicht verändert.

7.14 R3-ABSCHLUSS: BEGRENZTE MOBILITÄTS- UND POOL-SPEICHERUNG

Am 7. Oktober 2026 wurde die verbleibende R3-Abnahme als eine Unterphase im bestehenden nativen Conda-CPU-Environment abgeschlossen:

- 15 vorab festgelegte Fälle: H = 0/1/8/64/1024, WALL/TIME/DISTANCE, ns-3 Run 1, Seeds 42/43/44 und explizite Streams 0/1. Alle 300000 Bewegungsupdates stimmen exakt in Position, Geschwindigkeit, Kontrollzustand und lokalem RNG mit einer historienlosen Referenz überein. 1200 Checkpoints prüfen Slotidentitäten und Zähler; H = 1024 erreicht 49152 Byte und 1024 Paarallokationen, danach 18977 Wiederverwendungen. H = 0 bleibt leer.
- 512 ungecachte native RPCs: eine Stagingallokation, 511 Poolhits, 336 Byte freie Kapazität innerhalb 1048576 Byte; CSI-Belegung bleibt null. Wachstum, kleinere Grenzen, Übergrößen, deaktivierter Pool, fehlerhafte Batchsuffixe und unabhängige Getterwerte bestehen ebenfalls.
- 16-Epochen-Look-ahead liefert mit H = 0/1/1024 identische Antworten; Fehler bei Platzierung/CFR stellen Knotenidentität, Historie und RNG wieder her. Sechs zusätzliche echte LLVM-Traces (2x2, 256 Bins, 10000 Samples, zwei Epochen bei Anfragezeiten 1/12 ms) stimmen rohbytegenau mit der Vor-R3-Implementierung und einer Wiederholung überein.
- beide nativen Benchmarks und Sionna-Tests gezielt neu gebaut; bestehender Dependency-Lock und tatsächlicher LLVM-Backendcheck bestanden.

  - etablierte CPU-Regressionen sequenziell: sieben native Suites mit 60 Fällen;
  122 Python-Tests, davon 120 bestanden und zwei optionale Exporttests ausgelassen.

Neue Pläne, Rohantworten, CSVs, Logs und Eingangsbytes sind lokal im
[R3-Artefaktkatalog](experiments/r3/catalog.json) mit externem Indexhash
zugänglich. Das verschobene 39-Dateien-Paket verifiziert vollständig; ein neuer nativer
CPU-Lauf reproduziert alle deterministischen Speicher-/Trajektorienfelder exakt. Historische
Messhashes und R1/R2 bleiben unverändert. R3 besitzt keine Live-RT/PHY-Latenz-,
physikalische Doppler-, CFR-/Protobuf-Allokator- oder RSS-Freigabe; R4 ist gemäß Abschnitt 7.15
abgeschlossen, R5 ist im festgelegten CPU-Umfang abgeschlossen; R6 bleibt offen.

  7.15 R4-ABSCHLUSS: LIVE-CPU-SERVER-/PHY-SKALIERUNG

Am 7. Oktober 2026 wurde R4 als eine vollständige Unterphase abgeschlossen.
Die feste Abnahme verwendet AMD Ryzen 5 PRO 2500U, Conda/Python 3.12.12,
Sionna/Sionna RT 2.0.1, LLVM und einen Dr.Jit-Thread. Es wurden 14 Einstellungen
mit je drei frischen, sequenziellen Server-/ns-3-Prozesspaaren ausgeführt.

  - 42 Prozesspaare und alle 21 Cachepaarungen bestanden; 336 echte HE-SU-PPDUs
  führten zu 912 erfolgreichen vollständigen PHY-Rezeptionen und null Fehlern.
  - Alle 1890 echten RPCs und 11556 gelieferten Link-/Zeitsnapshots wurden nach
  Identität, Zeit, Shape, Antennen-/Frequenzgitter, Endlichkeit und Normierung
  geprüft. Native SINR-/PER-/Rangwerte und initiale Rohkanäle der Cachepaare
  erfüllen rtol 1e-4/atol 1e-7. Jedes Antwortartefakt bleibt rohbytegehasht.
  - Ungecachte CSI-Belegung, Nutzbytes und Kapazität waren in allen Beobachtungen
  null; größter gecachter Nutzdatenbatch exakt 1671168 Byte. Der getrennte
  strukturelle Stagingpool blieb unter 1 MiB; die Serverhistorien unter acht
  Einträgen pro Knoten, ohne zurückbehaltene Vorhersagezustände.
  - Maximales Linux-VmHWM: Server 485756928 Byte (463,25 MiB), nativer Prozess
  35504128 Byte (33,86 MiB). Das sind getrennte Prozessmaxima einschließlich
  Initialisierung/JIT, keine gleichzeitige Summe oder harte Ressourcenfreigabe.
  - Alle sieben nativen Suites mit 60 Fällen und die vollständige sequenzielle
  Python-Regression bestanden: 142 Tests, davon 140 bestanden und zwei optionale
  ältere Exporttests ausgelassen. Alle 20 neuen R4-Tests, einschließlich der drei
  Prüfungen des vollständigen Live-Exports, wurden ausgeführt. Die CPU-CI baut
  den neuen nativen Zielnamen; hosted CI/CUDA wurden hier nicht ausgeführt.
  - Relative Ausgabe-/Binärpfade werden vor Prozessstart absolut aufgelöst.
  Negative Optionen, fehlerhafte Antwortdaten und echte Prozessfehler testen
  Ausgabeschutz, Fehlerprovenienz, erhaltene Logs und aufgeräumte Peerprozesse.
  Der Regressionrunner setzt außerdem das ns-3-Arbeitsverzeichnis ausdrücklich,
  damit relative Kalibrierfixtures bei Aufruf aus dem Modul gefunden werden.

Die Tabelle aggregiert Folge-Compute-Timer nach Ausschluss der ersten Anfrage
jedes Prozesses; n ist deren Gesamtzahl über drei Prozesse. Median/p95 betreffen
produktive Serverberechnung einschließlich CFR-/Protobufaufbau. T_Run ist der
Median der instrumentierten Simulatorlaufzeit von drei Prozessen. Die übrigen
Stage-/Kalt-/RSS-/Rausch-/PHY-Daten bleiben in den Rohdateien erhalten.

R/A/MHz/D/Samples	Cache	RPCs gesamt	n Folge	Compute Median/p95 [ms]	T_Run [ms]
1/1/20/1/10000	aus	96	93	76.30/86.22	2559.73
1/1/20/1/10000	an	12	9	81.53/93.61	375.74
4/1/20/1/10000	aus	384	381	91.28/104.68	12109.87
4/1/20/1/10000	an	12	9	92.56/96.92	434.72
1/2/40/1/10000	aus	96	93	142.35/149.69	4707.52
1/2/40/1/10000	an	12	9	143.85/149.55	631.91
4/2/40/1/10000	aus	384	381	207.23/229.97	27413.48
4/2/40/1/10000	an	12	9	214.83/308.54	1018.10
1/1/20/4/10000	aus	96	93	90.38/103.13	3075.73
1/1/20/4/10000	an	3	0	keine Folgerechnung	131.37
4/2/40/4/10000	aus	384	381	421.43/468.02	56127.03

4/2/40/4/10000	an	3	0	keine Folgerechnung	539.58
4/2/40/1/100000	aus	384	381	1056.19/1165.72	136742.54
4/2/40/1/100000	an	12	9	1067.37/1156.90	4474.43

Cache reduziert tatsächliche Requests je Prozess von 32 auf 4 beim Einempfänger-Tiefe-1-Fall und von 128 auf 4 beim Vierempfängerfall; Tiefe 4 reduziert sie jeweils auf 1. Aus den gemessenen Medianen folgt zum Beispiel 56,13 s gegenüber 0,54 s für den Vierempfänger-/2x2-/40-MHz-/Tiefe-4-Fall. Das ist eine beobachtete Wirkung des bestehenden Caches auf diesem Host, kein universelles Latenzziel. 100000 statt 10000 Samples erhöht die Folge-Compute-Mediane im Vierempfänger-/2x2-/40-MHz-/Tiefe-1-Fall von ungefähr 207 auf 1056 ms ungecacht. Dies ist ein Kostenvergleich und keine zusätzliche Samplekonvergenzfreigabe.

Der [R4-Artefaktkatalog](experiments/r4/catalog.json) hält das vollständige lokale Bundle, dessen externen Indexhash, Plan, Rohdaten, Laufstatus, Nachprüfungen und die Auswertungs-/Regressionslogs fest. Das Bundle wurde nach Verschiebung mit seinem externen Hash geprüft; Messzeitpunkt-Quellidentitäten und die historischen R1/R2/R3-Artefakte wurden nicht ersetzt. Die vorherigen Pfad-/ACK-Diagnosen und der unvollständige 0-dBm-Lauf bleiben als lokale Fehler-/Kontrolldaten erhalten und werden nicht als bestandene Abnahme gezählt.

R4 ist im festgelegten instrumentierten CPU-Messumfang abgeschlossen. Weitere Szenen, PHY-/MU-/MAC-Lasten, Pools, universelle RSS-/Leistungsgrenzen, R5 schließt die begrenzte physikalische CPU-Alterung ab; GPU-Abnahme bleibt R6.

7.16 R5-ABSCHLUSS: PHYSIKALISCHE CSI-ALTERUNG

Am 8. Oktober 2026 wurde R5 als **eine vollständige Unterphase** im vorab festgelegten CPU-Umfang abgeschlossen. Der [R5-Vertrag](doc/r5-physical-aging.md) definiert den begrenzten Umfang; `tools/benchmark_physical_aging.py` führt ihn mit dem bestehenden nativen Beispiel `ns3_wifi_sumimo_benchmark.cc` aus.

Bewegungs-/Dopplervertrag. `simple_room`, 5180 MHz, 20 MHz mit 256 FFT-Bins, 2x2-Arrays mit 0,5 Lambda, TX/RX bei (3,2,2)/(4,3,1) m und Azimute 90/90 oder 90/0 Grad bleiben fest. RX bewegt sich mit v=(1,0,0) m/s beziehungsweise v=0 als Kontrolle; TX und Szeneobjekte sind stationär. Run 1/Streams 0–1, Seeds 42/43/44, 100000 Samples, Tiefe 3 und ein LLVM-Thread bilden die feste Abnahme. Zwei frische Wiederholungen je Einstellung prüfen Determinismus. Senden erfolgt bei 50 ms; Sounding liegt 0/1/5/20/50 ms davor.

`position` verwendet die produktive `SionnaEnv.compute_cfr`-Berechnung mit konstanter Geschwindigkeit/Richtung und tatsächlicher Position $(p_{rx}(t)=p_{rx}(0)+vt)$ an jedem Anfragezeitpunkt, ohne Wandkontakt. `doppler` berechnet echte Sionna-RT-Pfade an der Ausgangsgeometrie und ruft `Paths.cfr` mit RX-Geschwindigkeit, 1 kHz und 51 Zeitpunkten auf. Für jeden Antennenpfad gilt lokal

$$
 \widetilde H[k,t]=\sum_p a_p(0)
   e^{j2\pi\nu_p t}e^{-j2\pi f_k\tau'_p}.
$$

(ν_p) stammt aus der physikalischen Geschwindigkeitsprojektion der Sionna-Pfade; (τ'_p) verwendet dieselbe Delay-Normierung wie der Server. Bei stationären Streuern/TX gilt $(|\nu_p|\leq |v|f_c/c)$, hier etwa 17,28 Hz. Doppler hält Pfadtopologie, Amplituden, Richtungen und Delays an der Ausgangsgeometrie fest. Seine exportierten Positionen beschreiben die korrespondierende lineare Trajektorie; `configuration.pb` und `paths.npz` halten den tatsächlichen Zeit-null-Anker. Das ist eine lokale physikalische Näherung und wird getrennt von der geometrisch nachgeführten Berechnung berichtet. Beide Modelle werden nur am Zeit-null-Anker auf Gleichheit geprüft.

Jeder Zeit-Snapshot erhält seine eigene Leistungsnormierung und seinen
Wideband-Verlust, ohne künstliche Energie-/Singularwertinvarianz:

$$
q(t)=\frac{1}{4N_{sc}}\sum_{k,r,x}|\widetilde H_{rx}[k,t]|^2,\qquad
H[k,t]=\widetilde H[k,t]/\sqrt{q(t)},\quad L_{wb}(t)=-10\log_{10}q(t).
$$

Getrennte CSI und native Abstraktion. Die neue native Aktion
`physical-aging --input=<Sendesnapshot> --csiInput=<Soundingsnapshot>` prüft
gerichtete IDs 0->1, explizite 2x2-Dimensionen, passende streng aufsteigende
256-Ton-Gitter, endliche Positionen/Koeffizienten/Verluste, Normierung und
Nanosekundenintervalle, bevor sie eine Ausgabe anlegt. Der Runner prüft
zusätzlich das feste Gitter (-128..127)*78125 Hz und tatsächliche Geometrie.
Aus $\hat H[k,t_{csi}]$ erzeugt `MimoCsiAgingModel::Select` gleichleistungs-
SVD $W[k]=V_{csi}[k]$, $\|W[k]\|_F^2=2$. MMSE verwendet ausschließlich
$H[k,t_{tx}]W[k]$ und den Sendekanalverlust $L_{wb}(t_{tx})$:

$$
E[k]=\left(I+\frac{\rho(t_{tx})}{2}
W^H[k]H^H[k,t_{tx}]H[k,t_{tx}]W[k]\right)^{-1},\qquad
\gamma_s[k]=1/E_{ss}[k]-1.
$$

Gesamtleistung 20 dBm, 290 K und 7 dB Rauschzahl bleiben unverändert.
Alle Alter verwenden denselben unveränderten Sendesnapshot. Perfekte RX-CSI
bleibt erhalten. `csi_time_ns`, `transmission_time_ns` und `csi_age_ns` stammen
aus den zwei Rohantworten; `initial_time_ns` bezeichnet in dieser Aktion den
Soundingsnapshot, nicht den Doppleranker. Updateintervall ist explizit null,
Feedbackdauer entspricht dem Alter. Die 5-ms-Cachegültigkeit ist weder
CSI-Alter noch Feedbackdauer und wird hier nicht als Precoder-Ablauf benutzt.
Die bestehende analytische Rotation bleibt als separate Numerikkontrolle erhalten.

Die phaseninvariante Tonkorrelation lautet
$C_k=|\langle\hat H_k,H_k\rangle|/(\|\hat H_k\|_F\|H_k\|_F)$.
Der relative Koeffizientenunterschied verwendet $\|H_k-\hat H_k\|_F/\|H_k\|_F$.
Exportiert werden beide Verluste, Delays, Zeitstempel, Ton-SINRs, Rang,
Singularwerte, MI-SINRs, MI-Effizienz und deskriptive Laufzeiten.

Abnahme. 48 frische Trajektorien ergeben 240 gespeicherte Zeit-Snapshots,
24 vollständige Doppler-Pfadsätze und 240 native Sounding-/Sendeauswertungen.
Alle bestehen: unabhängige NumPy-SVD/MMSE/MI und Sendebudget mit rtol/atol
1e-8, Singularwerte 1e-10; unabhängige Doppler-Pfadsumme mit rtol 2e-4/atol
2e-9; stationäre/Nullalter-Kontrollen, physikalische Geschwindigkeitsgrenze,
Delay-/Geometrie-/Gitterprüfungen, identische rohe Wiederholungskanäle und
deterministische native Metriken. Frische Vollstream-SVD erreicht den separat
geprüften gleichen-Leistungs-Logdetwert; verzögerte MMSE-MI überschreitet ihn
nicht. Positive Bewegungsalter verändern den Kanal nachweislich. Eine monotone
Per-Stream-SINR- oder MI-Abnahme wird weder vorausgesetzt noch behauptet.

Bei 1 m/s und 50 ms Alter ergeben sich folgende Werte. Die Korrelation ist der
Median über 256 Töne. Bereiche betreffen genau drei numerisch gepaarte Seeds,
nur Wiederholung 0; Wiederholung 1 liefert dieselben Werte.

TX/RX-Azimut	Modell	Tonkorrelation Median [Seedbereich]	MI-Verlust Median [Seedbereich], bit/s/Hz
90/90 Grad	position	0.966705 [0.966705, 0.966744]	1.551614 [1.551614, 1.553167]
90/90 Grad	doppler	0.968684 [0.968684, 0.968718]	1.556055 [1.556051, 1.557541]
90/0 Grad	position	0.744658 [0.744544, 0.744658]	0.795591 [0.795219, 0.795592]

| 90/0 Grad | doppler | 0.752550 [0.752435, 0.752550] | 0.733664 [0.733184, 0.733665] |

Die engen Seedbereiche sind Sample-/Solver-Sensitivität bei derselben Geometrie,
keine unabhängigen Szenenrealisationen oder statistischen Konfidenzintervalle.
Der Verlust kann mit dem Alter nichtmonoton verlaufen; die Rohreihen bei
0/1/5/20/50 ms und der Plot erhalten diese Information. 100000 Samples sind
hier ein festes Budget und kein neuer Konvergenznachweis.

Sequenzielle CPU-Regressionen: sieben native Suites mit 60 Fällen sowie
153 Python-Tests, davon 148 bestanden und fünf optionale ältere Exportprüfungen
übersprungen. Alle elf R5-Tests wurden ausgeführt, einschließlich kompletter
Exportabnahme und absichtlich fehlender Pfade, geänderter Konfiguration/Policy
und veränderter/regehashter nativer SINR-Daten. Alte SU-/analytische Alterungs-
und MU-Verträge bleiben durch ihre bestehenden Tests abgedeckt.

Der [R5-Artefaktkatalog](experiments/r5/catalog.json) enthält das lokale
vollständige 574-Dateien-Bundle, seinen externen Indexhash, Rohdaten, Plan,
Quell-/Laufidentitäten und Prüfprotokolle. Die verschobene Kopie verifiziert
vollständig. Das ist Integritätsprüfung, kein neuer portabler Live-RT-Replay.
Der frühe 10000-Sample-Smoke und der wegen ungenauer geerbter Seed-/Zeit-
Metadaten gestoppte erste 100000-Sample-Lauf bleiben separate Kontrolldaten.
Nur `acceptance-final` zählt zur Abnahme; historische R1–R4-Messhashes bleiben
unverändert. Die PDF wird nach der finalen Dokumentationsänderung genau einmal
im vorhandenen CPU-Environment erzeugt und separat validiert.

R5 ist damit im festgelegten physikalischen CPU-Alterungsumfang abgeschlossen.
Automatisches Sounding, MAC-Feedback, produktiver Server-Dopplerdispatch,
bewegte Streuer, neue Abstand-/Elevations-/Roll-/NLOS-Szenen, größere statistische
Studien, PER-/Goodput-Kalibrierung und GPU-Nachweise bleiben spätere Arbeit.
Phase 5 insgesamt bleibt wegen R6 und der dokumentierten Profil-/Szenengrenzen offen.

7.17 R6: BACKENDVERGLEICH UND VOLLSTÄNDIGES REPRODUKTIONSPAKET

R6 wurde als **ein Implementierungsinkrement** mit dem vorab festgelegten
[Backendvertrag](doc/r6-backend-completion.md) umgesetzt. Die lokale CPU-Abnahme
ist erfolgreich; die physische CUDA-Abnahme auf der vom Anwender bereitgestellten
RTX 3060 und eine tatsächliche Ausführung der CUDA-CI stehen noch aus. R6 wird
deshalb ausdrücklich **nicht als vollständig hardwarevalidiert** ausgewiesen.
R1–R5 und deren historische Messhashes bleiben unverändert.

`benchmark_backends.py` startet getrennte frische CPU-/CUDA-Prozesse und setzt
vor dem Import der variantenspezifischen Sionna-Klassen exakt
`llvm_ad_mono_polarized` beziehungsweise `cuda_ad_mono_polarized`.
Eine tatsächliche Dr.Jit-Auswertung muss [2,3] ergeben; ein LLVM-Fallback
erfüllt die CUDA-Abnahme nicht. Beide Seiten verwenden denselben vollständigen
Python-Lock einschließlich CPU-PyTorch: Der CUDA-Raytracer wird durch
Mitsuba/Dr.Jit ausgeführt, die MIMO-Abstraktion weiterhin ausschließlich durch
den nativen ns-3-Kern. Variant, Paketversionen, Lockhash, Treiber-/GPU-Ausgabe,
Konfiguration, Roh-CFR, native Kommandos, Messzeiten und Dateihashes werden
gespeichert. Fehler und Zeitüberschreitungen behalten Logs und einen getrennten
fehlgeschlagenen beziehungsweise unvollständigen Ausführungsstatus.

Die feste Matrix verwendet `simple_room`, 5180 MHz, 20 MHz/256 FFT-Bins,
explizite 2x2-Arrays mit Abstand 0,5 lambda, Positionen (3,2,2)/(4,3,1) m,
TX-/RX-Azimute 90/90 und 90/0 Grad, Seeds 42/43/44, Run 1, Streams 0/1,
100000 Samples, Tiefe 3, einen LLVM-Thread und zwei frische Wiederholungen.
Das sind zwölf Traces pro Backend; andere Budgets bleiben Diagnostik.
Die produktive CFR-Normalisierung erfüllt

$$
  \frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
$$

```
  \sum_k\Vert H_b[k]\Vert_F^2=1,
  \qquad b\in\{\mathrm{CPU},\mathrm{CUDA}\}.
\]

Der gleiche native Open-Loop-Kern berechnet pro Backend bei gleicher
Gesamtsendeleistung und perfekter RX-CSI

\[
 E_b[k]=\left(I+\frac{\rho_b}{2}H_b^H[k]H_b[k]\right)^{-1},
 \quad\gamma_{b,s}[k]=\frac{1}{[E_b[k]]_{ss}}-1,
 \quad\rho_b=\frac{P_\mathrm{tx}10^{-L_b/10}}
 {k_\mathrm{B}TB10^{F/10}}.
\]

\[
 \gamma_{b,s}^{\mathrm{eff}}
 =\exp\!\left(\frac{1}{N_\mathrm{sc}}
   \sum_k\ln(1+\gamma_{b,s}[k])\right)-1,
 \qquad
 \eta_b=\frac{1}{N_\mathrm{sc}}
   \sum_{k,s}\log_2(1+\gamma_{b,s}[k]).
\]

Alle gespeicherten Ergebnisse werden gegen unabhängige NumPy-SVD-/MMSE-/MI-
Referenzen und optional durch erneutes natives Replay geprüft. Gepaarte
CPU-/CUDA-Vergleiche und backendinterne Wiederholungen übernehmen die bestehende
Konvergenzpolicy: Pfadverlustabweichung höchstens 0,5 dB, kein geänderter
Tonrang, Singularwerte mit rtol 0,05/atol 1e-7, Ton-SINR höchstens 1 dB,
effektive SINR höchstens 0,5 dB und MI höchstens 0,2 bit/s/Hz. Vor dem
Logarithmus gilt die SINR-Untergrenze 1e-12. Jeder Ton, Seed und jede
Wiederholung muss bestehen; die Toleranzen werden nach GPU-Messungen nicht
nachträglich verändert. Komplexe CFR-Abweichungen

\[
 \epsilon_H=\frac{\Vert H_\mathrm{CUDA}-H_\mathrm{CPU}\Vert_F}
    {\Vert H_\mathrm{CPU}\Vert_F}
\]

und Bytegleichheit bleiben separate Diagnostik, weil backendabhängige
Pfadentdeckung und Reihenfolge verschieden sein können. Es wird weder eine
allgemeine Samplekonvergenz noch ein Geschwindigkeitsgewinn oder ein neues
PER-/Durchsatzprofil behauptet.

`run_r6_gpu.sh` führt den gesamten Ablauf auf einem Linux-Rechner mit RTX 3060
aus: externe Bundlehashprüfung, sichere Quellenextraktion, isolierte Python-
Umgebung, frische Protobuf-Bindings, nativer Build, sequenzielle CPU-Regression,
echter CUDA-Probe, beide Messungen, unabhängiges und natives Audit, R6-Tests,
Bundleexport, Verschieben und erneutes Replay. Die notwendige einmalige
Installation nativer Systembibliotheken und die konkreten Transferbefehle
stehen im Backendvertrag. Automatische Systemänderungen erfolgen nicht.
Die neue `.github/workflows/cuda.yml` nutzt denselben Ablauf auf einem
Self-Hosted-Runner mit Labels `self-hosted,linux,x64,gpu`. Sie ist ausschließlich
manuell auslösbar; CUDA-Ausfälle werden nicht als erfolgreiche Skips behandelt.
Die vorhandene CPU-CI bleibt bestehen und übernimmt die neuen R6-Tests.

Das vollständige lokale Reproduktionspaket enthält rohe Backendstudien,
Konfigurationen, Szene, Ergebnisse, den vollständigen relevanten lokalen
ns-3-Quellbaum einschließlich PHY-Anpassungen und Modul, Lock, Tests,
Quellinventar/-hashes und Wiederaufbaubefehle. Buildausgaben, Git-Metadaten,
Caches, Plots, PDFs und lokale Experimentverzeichnisse werden ausgeschlossen.
Die Quellen sind ein identifizierter Working-Tree-Snapshot, kein vorgetäuschter
```

sauberer Upstream-Checkout. Historische Studien können zusätzlich unverändert exportiert werden. Bibliotheken und Python-Wheels bleiben externe Installationsvoraussetzungen. Ein außerhalb des Bundles aufbewahrter Indexhash authentifiziert die vollständigen Payloads; `gpu_pending` wird beim Verpacken nicht in eine GPU-Abnahme umgedeutet.

Lokal liefen zwölf echte CPU-Traces mit 3072 Tonauswertungen erfolgreich. Der strikte CPU-Lock-/Systemcheck und sieben native Testsuiten bestanden. Die sequenzielle Python-Regression führte 165 Tests aus: 160 bestanden, fünf optionale ältere Exporte waren nicht ausgewählt. Die abschließende R6-Suite bestand separat alle 13 Tests, einschließlich aller zwölf nativen CPU-Replays, wissenschaftlicher Negativfälle, manipulierten Exporte, Timeout-/Fehlerartefakte und sicherer Remote-Bundleextraktion. Gemessene Initialisierung 2,43–7,51 ms, CFR 658,62–767,89 ms und nativer Subprozess 31,25–36,55 ms sind rein deskriptiv. Die lokale CUDA-Probe schlug auf dem CPU-Rechner explizit fehl. Der [R6-Katalog](experiments/r6/catalog.json) identifiziert Messungen, verschobenes Bundle, Transferarchiv, Audits und die genau einmal nach dieser finalen Dokumentationsänderung erzeugte PDF.

Der Anwender führt das Transferpaket auf der RTX 3060 aus und liefert `r6-results.tar.gz` zurück. Erst nach unabhängiger Prüfung dieser tatsächlichen GPU-Ergebnisse und gesondertem CI-Nachweis kann R6 als hardwarevalidiert abgeschlossen werden. Bis dahin bleibt Phase 5 in diesem Punkt offen.

Die Korrektur vom 12. Oktober 2026 unterstützt nun ausdrücklich bereits installierte Conda-Umgebungen durch `--use-existing-environment`. Dieser Modus verwendet den aktiven Interpreter mit Python >=3.11, erzeugt keine neue venv und installiert keine Pakete. Das ursprüngliche pauschale Python-3.12-Gate und der NumPy-2.5.2-Lock passten nicht zur vorhandenen Python-3.11-/NumPy-2.4.4-Umgebung. Die Korrektur bewahrt die wissenschaftlichen Mess- und Backendkriterien: Sionna/Sionna RT 2.0.1, Mitsuba 3.8.0 und Dr.Jit 1.3.1 bleiben erforderlich; alle aufgelösten Paketversionen, die tatsächliche Python-Version sowie Abweichungen vom ursprünglichen Lock werden dokumentiert. CPU und CUDA müssen dieselben aufgezeichneten Python-/Paketversionen verwenden. CUDA-PyTorch 2.9.1+cu128 ist dadurch zulässig; die Link-Abstraktion bleibt in ns-3. Vendored Distribution-Metadaten werden über dieselbe Versionsauflösung wie beim bisherigen Lockcheck behandelt. Ein bestehendes Environment wird nicht nachträglich als exakte Reproduktion des CPU-Locks bezeichnet.

Das kleine lokale Updatearchiv `experiments/r6/r6-existing-environment-update.tar.gz` enthält drei zusammen zu übertragende Dateien. Nach Prüfung des unveränderten ursprünglichen Bundles kopiert der aktualisierte Launcher Runner und Tests in den extrahierten Quellbaum. `logs/runner-overlay.json` hält Original-/Updatehashes und den verwendeten Interpreter fest. Der Anwender aktiviert auf `hpc3` seine bestehende `sionna2`-Umgebung, entpackt das Update neben `bundle/` und startet `r6-env-update/run_r6_gpu.sh --use-existing-environment` mit einem neuen Ausgabeverzeichnis. Das Originalbundle und dessen externer Indexhash bleiben unverändert. Die Remote-Ausführung übernimmt der Anwender.

Lokal bestanden alle 18 R6-Tests, einschließlich des tatsächlich ausgeführten Shellpfads ohne venv-/Paketinstallation, Python-3.11-/CUDA-PyTorch-Provenienz, Ablehnung verschiedener CPU-/CUDA-Umgebungen, unabhängiger Auswertung vorhandener Exporte und sicherer Bundleextraktion. Zwölf frische CPU-Traces unter dem neuen Modus bestanden erneut native MMSE-/MI- und NumPy-Prüfungen. Der aufgezeichnete Environmentmodus und die Abweichungen bleiben auch beim verlagerten Audit sichtbar. Die vollständigen CPU-Regressionen, das lokale Updatearchiv und die nach dieser abschließenden Dokumentationsänderung einmal erzeugte PDF werden im R6-Katalog mit getrennten Prüfnachweisen festgehalten. Die eigentliche RTX-3060-Messung und CUDA-CI-Abnahme bleiben ausstehend.

Der erste Remote-Regressionslauf im bestehenden Environment zeigte einen unvollständigen Overlay: Die Python-Dateien waren aktualisiert, der im Bundle enthaltene Shell-Launcher jedoch noch unverändert. Dessen erneuter Aufruf im Shellpfad-Test wies `--use-existing-environment` zurück, bevor eine frische CPU-/CUDA-Messung begann. Die Korrektur kopiert alle drei Begleitdateien, einschließlich `run_r6_gpu.sh`, und protokolliert ihre Original-/Updatehashes. Der Regressionstest beginnt mit drei veralteten Dateien und prüft deren vollständige Ersetzung sowie die Hashnachweise. Das lokale Folgearchiv heißt `experiments/r6/r6-runner-overlay-update.tar.gz`. Mit `--source` kann der Anwender den bereits kompilierten, zuvor bundleverifizierten Quellbaum wiederverwenden; ein neues Ausgabeverzeichnis bewahrt die ursprünglichen Fehlerlogs. Die wissenschaftlichen Kriterien und der ursprüngliche Bundleindex bleiben gleich.

Auf Wunsch des Anwenders verwendet `--use-existing-build --source=<ns3-Pfad>` den ursprünglichen vorhandenen ns-3-Quellbaum und dessen kompilierte Programme sowie das aktive Conda-Environment. Dieser Modus extrahiert keine neue Kopie, konfiguriert und kompiliert nicht, regeneriert keine Bindings und installiert keine Pakete. Lediglich die drei Begleitdateien des R6-Launchers werden im gewählten Quellbaum aktualisiert. `logs/build-reuse.json` dokumentiert Pfade und Hashes des vorhandenen Testrunners, der Sionna-Testbibliothek, der drei Benchmarkprogramme und der drei Bindings. Fehlende oder mehrdeutige Artefakte werden ohne automatischen Build abgewiesen. Regressionen, CUDA-Probe, gepaarte Messungen, unabhängige und native Prüfung sowie Paketierung bleiben erforderlich. Die Tests prüfen vorhandene und fehlende Buildartefakte, unveränderte Bindings und das Ausbleiben jeglichen ns-3-Aufrufs. Quell- und Binäridentitäten werden getrennt aufgezeichnet; ein frischer Buildnachweis wird bei Wiederverwendung nicht behauptet. Das lokale Übertragungsarchiv heißt `experiments/r6/r6-existing-ns3-update.tar.gz`.

8. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- SionnaMimoHelper überträgt eine gemeinsame TX- und RX-Geometrie für die jeweilige integrierte Verbindung. Knoten können unterschiedliche Orientierungen besitzen. Der Snapshotvertrag aus Unterphase 4.1 repräsentiert heterogene RX-Geometrien pro Nutzer; die aktuelle Sionna-Raytracing-Konfiguration verwendet weiterhin ein gemeinsames RX-Array.
- Nicht explizit orientierte Knoten behalten die alte Serverorientierung [0, -180, 0]. Explizite Winkel werden in beiden Servermodi an Sionna übertragen.
- Der integrierte MIMO-Pfad unterstützt HE SU ohne Precoding, festes benutzergegebenes \(W[k]\) und die explizite Helperwahl von MRT oder SVD. Nicht-HE-Kontrollrahmen verwenden weiterhin das erste Antennenpaar. SelectPrecoding benötigt eine bereits vorliegende CSI-Matrix und Zeitstempel; automatische Kanalschätzung, Sounding, periodische Simulatorereignisse, MAC-Feedbackframes und per-Paket-Aktualisierung sind nicht implementiert. Unterphase 4.2 stellt den festen gemeinsamen MU-Precoder und die Inter-User-Interferenz als direkte native API bereit. Unterphase 4.3 ergänzt daran die explizit aufgerufenen ZF-/RZF-Fabriken für vollrangige Gruppen mit einer Empfangsantenne und einem Stream je Nutzer. Unterphase 4.4 integriert einen explizit installierten gemeinsamen Zustand für HE-DL-MU direkt in den Spectrumadapter und liest die HE-MU-PHY-Metadaten aus dem TxVector. Unterphase 4.5 ergänzt die wertbasierte Laufzeit- und Scheduler-Schnittstelle; Unterphase 4.6 installiert sie mit `InstallDownlink` am AP und setzt am tatsächlichen Sendezeitpunkt den getrennten Übertragungskanal. Automatisches Sounding und MAC-Feedbackrahmen bleiben offen.
- Der direkte Uplinkkern der Unterphase 4.7 verlangt pro Nutzer eine Sendeantenne und Stream null, einen gemeinsamen AP-Empfänger sowie ideale Zeit- und Frequenzsynchronisation. ZF verlangt zusätzlich \(U\leq N_{\mathrm{rx,AP}}\) und vollen Rang; lineares MMSE bleibt bei Rangverlust definiert. Unterphase 4.8 bindet den Kern an HE-Basic-Trigger

und echte HE-TB-PPDU-Metadaten, gruppiert exakt gleichzeitige Nutzer je RU und meldet fehlende oder nicht gleichzeitige Teilnehmer explizit. Die Gleichzeitigkeit wird über das vom Aufrufer beobachtete Empfangsintervall eingebracht; automatische Trigger-Schedulerinstallation, Zeit-/Frequenzoffsets und mehrere lokale Nutzerstreams bleiben offen.
- MRT bleibt der eindeutige perfekte 2x1-MISO-Ein-Stream-Fall. Quantisierung und positives CSI-Alter gelten nur für gleichleistungs-SVD. Wasserfüllung bleibt auf unquantisiertes aktuelles CSI beschränkt, weil die semantische Winkelrückmeldung einen semi-unitären Gleichleistungs-Precoder verlangt.
- Die Auswahl speichert einen einzelnen unveränderlichen W[k]-Schnappschuss pro Helper. Der Anwender muss die aktuelle Sendekanal-Matrix weiterhin getrennt für SINR und PER bereitstellen und bei neuer CSI SelectPrecoding erneut aufrufen. Das ist kein per-Link- oder per-Paket-Precodercache.
- Die Inter-User-Interferenz der gleichzeitig durch denselben festen MU-Precoder gesendeten Streams geht in Unterphase 4.2 frequenz- und antennenabhängig in die MMSE-Kovarianz ein. Im HE-MU-Pfad der Unterphase 4.4 gilt das ausschließlich innerhalb derselben RU; verschiedene RUs sind orthogonal. Andere zeitvariable externe Interferenz wird weiterhin erst nach der MMSE-/EESM-Abstraktion durch InterferenceHelper skalar berücksichtigt; ihre räumliche Kovarianz ist nicht modelliert.
- SionnaMimoHelper übernimmt die Rauschzahl einmalig und setzt sie identisch am Adapter sowie an beiden PHY-Helpern. Code ohne High-Level-API muss beide Werte weiterhin manuell konsistent halten, da ns-3.40 das PHY-Attribut nicht lesbar macht. Der Phase-4.4-Adapter verwendet deshalb seinen explizit konfigurierten Wert für alle adressierten Empfänger.
- Es existiert absichtlich keine mitgelieferte HE-EESM-Beta-Tabelle. Frequenzselektive absolute PER-Prognosen benötigen vor der wissenschaftlichen Nutzung eine dokumentierte Link-Level-Kalibrierung. Ohne diese Konfiguration bricht das Modell früh ab.
- TableBasedErrorRateModel ist eine AWGN-Referenz. Seine SNR-/PER-Kurven und Paketlängenextrapolation stammen aus ns-3; ihre Grenzen gelten unverändert.
- Perfekte RX-CSI bleibt bewusst idealisiert. Für die Precoderwahl trennt der neue Alterungspfad die geschätzte CFR zum CSI-Zeitpunkt explizit von der CFR zum Sendezeitpunkt und weist Alter, Aktualisierungsintervall und Rückmeldedauer aus; eine automatische Kanalschätzung oder Ablaufsteuerung im Helper existiert noch nicht.
Der 5.2.5-Benchmark entwickelt eingefrorene reale CFRs durch eine analytische unitäre TX-Basisrotation bei fester Geometrie. Seine Alters- und MMSE-/MI-Verluste sind keine physikalisch kalibrierte Doppler- oder Mobilitätsvorhersage; gemeinsame Kapazität bleibt hier invariant.
- Ohne W[k] gilt weiterhin genau ein Stream je TX-Antenne. Mit festem W[k] sind kontrollierte Streamauswahl und \(1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})\) für SU implementiert. Im festen MU-Kern gilt zusätzlich \(N_{s,\mathrm{ges}}\leq N_\mathrm{tx}\) und \(N_{s,u}\leq N_{\mathrm{rx},u}\). Die ZF-/RZF-Auswahl der Unterphase 4.3 ist enger: $N_{s,u}=1$, $N_{\mathrm{rx},u}=1$, $U\leq N_{\mathrm{tx}}$ und voller numerischer Rang auf jedem Subcarrier.
- Die MI-Kompression bleibt eine Diagnosemetrik; nur die explizit konfigurierte EESM-Ausgabe wird an ErrorRateModel übergeben.
- Explizit identifizierte Phase-4.1-Rohkanaleinträge werden nur in ihrer gespeicherten TX-/RX-Richtung getroffen. Nur identitätsfreie Legacy-Einträge behalten die bisherige reziproke Cacheverwendung. Die Precoder-Zustandsidentität enthält ebenfalls gerichtete TX-/RX-Knoten, beide Arraykonfigurationen, Modus, Feedbackkonfiguration und CSI-Zeitpunkt. Der Helper gibt diese Identität maschinenlesbar weiter, bindet sie aber bewusst nicht an einen automatischen Laufzeitcache.
- Der deterministische Beispielkanal ist eine gerichtete, normalisierte Vorwärts-CFR. Das Ad-hoc-UDP-Beispiel setzt deshalb den ARP-Eintrag am Sender statisch; so erzeugt die Infrastruktur keine rückwärts gerichtete HE-Datennachricht mit inkompatiblen 2x1-Dimensionen. Normale Sionna-Cachepfade

sind davon nicht betroffen.

Phase 3 ist mit Unterphase 3.5 abgeschlossen; Unterphase 4.1 ergänzt den gerichteten, atomar validierten Mehrnutzer-Kanalsnapshot. Unterphase 4.2 verwendet ihn nun für einen festen, explizit zugeordneten gemeinsamen Downlink-Precoder, erhält die gesamte AP-Leistung und liefert gewünschte Leistung, Inter-User-Interferenz, Post-MMSE-SINR, EESM und PER je Nutzer. Der Phase-4.3-Kern leitet daraus für den dokumentierten Einantennen-/Ein-Stream-Fall deterministisches ZF oder explizit regularisiertes RZF ab, erhält die Gesamtleistung und lehnt nicht planbare Gruppen ab. Unterphase 4.4 bindet diesen gemeinsamen Zustand für echte HE-DL-MU-PPDUs an STA-ID, RU, MCS und NSS, begrenzt räumliche Interferenz auf dieselbe RU und übergibt erst nach der Matrixrechnung eine skalare PSD je adressiertem Empfänger. Der Helper führt parallel die unveränderten SU-Verträge durch den bisherigen festen-W[k]-Pfad. Einzelnutzer- und 1x1-Grenzfall sowie alle Phase-1-/2-/3-Pfade bleiben kompatibel. Die Laufzeit-CSI-/Scheduler-Schnittstelle der Unterphase 4.5 verwaltet jetzt den neuesten exakten AP-/Gruppen-/RU-/NSS-Zustand, übernimmt Phase-3-Zeiten und installiert ausschließlich denselben gerichteten Snapshot im PHY. Unterphase 4.7 ergänzt getrennte Nutzerleistungen, gemeinsame ZF-/MMSE-Detektion und nutzerweise EESM/PER im direkten Uplink-Numerikkern. Unterphase 4.8 schließt Phase 4 mit dem HE-TB-/Triggeradapter, expliziter Gleichzeitigkeitsauswahl und dem erweiterten CPU-Phasenbeispiel ab. Automatische Sounding-/Feedbackabläufe, Synchronisationsfehler und eine vollständige MAC-Schedulerinstallation bleiben Forschungsgrenzen für Phase 5.

9. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
    ./ns3 configure --enable-examples --enable-tests
    ./ns3 build
    ./test.py -s sionna
    ./test.py -s dl-mu-mimo-ru
    ./test.py -s he-dl-mu-spectrum-ru
    ./test.py -s he-ul-mu-mimo-phy-adapter
    ./test.py -s ul-mu-mimo
    ./test.py -s dl-mu-mimo-runtime
    ./test.py -s sionna-mimo-csi-feedback
    ./test.py -s sionna-example-sumimo-minimal
    ./test.py -s sionna-example-mimo-helper-configuration-negative
    ./test.py -s sionna-example-mimo-helper-runtime-negative
    ./test.py -s sionna-example-sumimo-precoding
    ./test.py -s sionna-example-sumimo-precoding-negative
    ./test.py -s sionna-example-mumimo
    ./ns3 run "ns3_wifi_sumimo_minimal --eesmBetaLinear=2"
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2"
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2 \
    --quantization=coarse --csiUpdateIntervalMs=5 --feedbackDelayMs=2"

    ./ns3 run "ns3_wifi_mumimo --eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1"
```

Phase 5 im dokumentierten CPU-Environment, jeweils mit neuem Ausgabeordner:

```
    ./ns3 build -j 2 test-runner sionna-test ns3_wifi_mimo_awgn_curve \
    ns3_wifi_sumimo_benchmark ns3_sionna_cache_ipc_benchmark
    python contrib/sionna/tools/run_cpu_regressions.py
    python contrib/sionna/tools/calibrate_eesm.py \
    --model-awgn contrib/sionna/data/he_mcs0_bcc_32_ns3_awgn.csv \
    --reference contrib/sionna/data/he_mcs0_bcc_32_matlab_r2026a.json \
    --output=/tmp/he-calibration.json
    python contrib/sionna/tools/benchmark_sumimo.py --output=/tmp/sumimo
```

```
    python contrib/sionna/tools/benchmark_rt_convergence.py --output=/tmp/convergence
    python contrib/sionna/tools/benchmark_cache_ipc.py --output=/tmp/cache-ipc
    python contrib/sionna/tools/benchmark_cache_ipc.py --workload-sweep \
    --output=/tmp/cache-workload

Der Konvergenzrunner bietet `--independent-spacing` und `--csi-aging`,
der IPC-Runner `--copy-profile`, `--storage-steps` und `--workload-steps`.
Jede Variante benötigt ihren eigenen leeren Ausgabeordner; Details und
Defaults stehen unter `--help`. `run_cpu_regressions.py` prüft die versionierten
Referenzen und native Kontrollen. Lokale Exporte werden zusätzlich über
`SUMIMO_BENCHMARK_RESULTS`, `SUMIMO_CONVERGENCE_RESULTS`, `SUMIMO_AGING_RESULTS`,
`CACHE_IPC_BENCHMARK_RESULTS` bzw. `CACHE_IPC_WORKLOAD_RESULTS` ausgewählt.
Frische RT-Läufe und lokale Studien bleiben separate Prüfungen.

Python-Vertrag und Server vom Verzeichnis contrib/sionna/model/ns3sionna:

    python -m pip install --upgrade -r requirements_cpuonly.txt
    python tests/test_imports.py
    conda run -n sionna-venv python -m unittest -v tests.test_mimo_data_contract
    conda run -n sionna-venv python -m unittest -v tests.unit_tests

R3-Abschluss, im vorhandenen Conda-CPU-Environment mit leerem Ausgabeordner:

    ./ns3 build -j 2 sionna-test ns3_wifi_sumimo_benchmark ns3_sionna_cache_ipc_benchmark
    python contrib/sionna/tools/benchmark_mobility_storage.py --native-check \
    --output=/tmp/r3-storage
    python contrib/sionna/tools/experiment_bundle.py export \
    --study=/tmp/r3-storage --output=/tmp/r3-bundle
    python contrib/sionna/tools/run_cpu_regressions.py

Den ausgegebenen Indexhash separat aufbewahren; das vollständige Bundle verschieben
und mit `verify --bundle <neuer-pfad> --index-sha256 <externer-hash>` prüfen.
`storage.csv` enthält alle 256 Schritte einen Checkpoint sowie Anfang/Ende.
Zeitmessungen umfassen gepaarte Bewegung und Validierung und sind rein deskriptiv.

R4-Live-Abnahme, aus dem ns-3-Verzeichnis im dokumentierten CPU-Environment:

    ./ns3 build -j 2 ns3_sionna_live_benchmark sionna-test
    python contrib/sionna/tools/benchmark_live_scaling.py --output=/tmp/r4-live
    LIVE_SCALING_RESULTS=/tmp/r4-live python contrib/sionna/tools/run_cpu_regressions.py
    python contrib/sionna/tools/experiment_bundle.py export \
    --study=/tmp/r4-live --output=/tmp/r4-bundle \
    --input=contrib/sionna/examples/ns3_sionna_live_benchmark.cc \
    --input=contrib/sionna/tools/benchmark_live_scaling.py

`--quick --epochs=2` prüft eine kleine Cachepaarung; eigene Samples/Epochen/
Wiederholungen benötigen neue Ordner und sind nicht die feste volle Abnahme.
Server-/Native-Logs und Protokolldaten werden auch bei Fehlern erhalten.
Den ausgegebenen Bundleindexhash extern aufbewahren und die verschobene Kopie
mit `experiment_bundle.py verify` gegen diesen Hash prüfen. Messgrenzen und
Rohdatenschemata stehen im [R4-Vertrag](doc/r4-live-scaling.md).

R5-Physikalische-Alterungsabnahme im vorhandenen CPU-Environment:

    ./ns3 build -j 2 ns3_wifi_sumimo_benchmark sionna-test
    python contrib/sionna/tools/benchmark_physical_aging.py --output=/tmp/r5-study
    PHYSICAL_AGING_RESULTS=/tmp/r5-study python contrib/sionna/tools/run_cpu_regressions.py
    python contrib/sionna/tools/benchmark_physical_aging.py \
    --output=/tmp/r5-study --validate-export
    python contrib/sionna/tools/experiment_bundle.py export \
    --study=/tmp/r5-study --output=/tmp/r5-bundle \
```

```
    --input=contrib/sionna/tools/benchmark_physical_aging.py \
    --input=contrib/sionna/examples/ns3_wifi_sumimo_benchmark.cc

Andere Samples/Wiederholungen benötigen neue Ordner und sind keine feste Abnahme.
`--validate-export --binary=<nativer-Pfad>` führt zusätzlich native Paarreplays aus.
Den Bundleindexhash separat behalten und die verschobene Kopie mit `verify` prüfen.
Der [R5-Vertrag](doc/r5-physical-aging.md) erklärt die zwei physikalischen Modelle.

PDF neu erzeugen:

    python contrib/sionna/tools/render_model_pdf.py \
    contrib/sionna/ns3sionna_mimo_model.md \
    contrib/sionna/ns3sionna_mimo_model.pdf

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.
```